

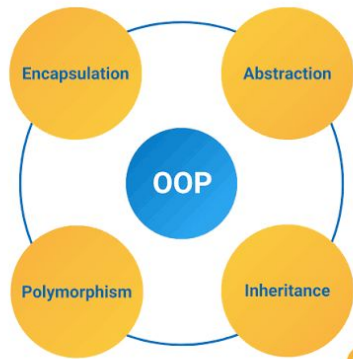
Modélisation et Programmation Orientée Objet

Licence 2
Semestre 3

Plan du cours

Cours à réviser après chaque séance et non uniquement à la fin des séances avant l'examen - susceptible d'évoluer

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		



Principes de base de l'Orientée Objet et introduction à Java

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

La programmation traditionnelle

- Pour résoudre un problème, on commence par le décomposer en sous problèmes élémentaires, que l'on pourra résoudre séparément
 - approche de modélisation "descendante"
- On identifie les données avec lesquelles on va travailler, puis on définit les traitements qu'on leur applique
 - il y a donc séparation des données et des traitements
- Un programme est un assemblage de structures de données et d'algorithmes
- Inconvénients de la programmation procédurale
 - la solution (le programme) est liée au problème, elle n'est pas générique et donc peu réutilisable
 - les différentes portions sont fortement dépendantes

La programmation traditionnelle

- Pour résoudre un problème, on commence par le décomposer en sous problèmes élémentaires, que l'on pourra résoudre séparément
 - approche de modélisation "descendante"
- On identifie les données avec lesquelles on va travailler, puis on définit les traitements qu'on leur applique
 - il y a donc séparation des données et des traitements
- Un programme est un assemblage de structures de données et d'algorithmes
- Inconvénients de la programmation procédurale
 - la solution (le programme) est liée au problème, elle n'est pas générique et donc peu réutilisable
 - les différentes portions sont fortement dépendantes

La programmation traditionnelle

- Pour résoudre un problème, on commence par le décomposer en sous problèmes élémentaires, que l'on pourra résoudre séparément
 - approche de modélisation "descendante"
- On identifie les données avec lesquelles on va travailler, puis on définit les traitements qu'on leur applique
 - il y a donc séparation des données et des traitements
- Un programme est un assemblage de structures de données et d'algorithmes
- Inconvénients de la programmation procédurale
 - la solution (le programme) est liée au problème, elle n'est pas générique et donc peu réutilisable
 - les différentes portions sont fortement dépendantes

La programmation traditionnelle

- Pour résoudre un problème, on commence par le décomposer en sous problèmes élémentaires, que l'on pourra résoudre séparément
 - approche de modélisation "descendante"
- On identifie les données avec lesquelles on va travailler, puis on définit les traitements qu'on leur applique
 - il y a donc séparation des données et des traitements
- Un programme est un assemblage de structures de données et d'algorithmes
- Inconvénients de la programmation procédurale
 - la solution (le programme) est liée au problème, elle n'est pas générique et donc peu réutilisable
 - les différentes portions sont fortement dépendantes

Pourquoi programmer objets ?

- Dans les langages procéduraux, on décompose le programme en unités de code qui interagissent, chacune ayant une fonction clairement définie
 - l'information représentant l'état général du système est partagée par toutes les fonctions → problème : une modification peut entraîner un effet imprévu sur d'autres fonctions
 - pour diminuer les bugs, on va chercher à minimiser l'information partagée

Pourquoi programmer objets ?

- Dans les langages procéduraux, on décompose le programme en unités de code qui interagissent, chacune ayant une fonction clairement définie
 - l'information représentant l'état général du système est partagée par toutes les fonctions → problème : une modification peu entraîner un effet imprévu sur d'autres fonctions
 - pour diminuer les bugs, on va chercher à minimiser l'information partagée
- L'activité de production de logiciels est très répétitive
 - on a toujours plus ou moins les mêmes besoins de base
 - afin de réduire les coûts de développement, on va modéliser en favorisant la réutilisation de composants existants
 - on conçoit ces composants de la façon la plus indépendante du contexte : c'est la problématique de la conception objets
 - on peut définir de nouveaux composants en spécialisant des composants existants

Caractéristiques essentielles de l'OO

- L'abstraction : on définit les caractéristiques essentielles d'un objet, aussi bien au niveau de sa structure que de la façon dont on va l'utiliser

Caractéristiques essentielles de l'OO

- L'abstraction : on définit les caractéristiques essentielles d'un objet, aussi bien au niveau de sa structure que de la façon dont on va l'utiliser
- L'encapsulation : on cache toutes les caractéristiques non essentielles d'un objet ; on sépare l'interface de l'implémentation

Caractéristiques essentielles de l'OO

- L'abstraction : on définit les caractéristiques essentielles d'un objet, aussi bien au niveau de sa structure que de la façon dont on va l'utiliser
- L'encapsulation : on cache toutes les caractéristiques non essentielles d'un objet ; on sépare l'interface de l'implémentation
- La modularité : le programme sera découpé en modules faiblement couplés, c'est-à-dire totalement indépendants les uns des autres (le plus possible)

Caractéristiques essentielles de l'OO

- L'abstraction : on définit les caractéristiques essentielles d'un objet, aussi bien au niveau de sa structure que de la façon dont on va l'utiliser
- L'encapsulation : on cache toutes les caractéristiques non essentielles d'un objet ; on sépare l'interface de l'implémentation
- La modularité : le programme sera découpé en modules faiblement couplés, c'est-à-dire totalement indépendants les uns des autres (le plus possible)
- La hiérarchie : on organise les abstractions entre elles en respectant une relation "est une sorte de"

Points additionnels de l'OO

- Le typage : des objets de types différents ne peuvent être mis en relation que de façon très restrictive
 - plusieurs catégories de typage : faible ou fort, statique ou dynamique

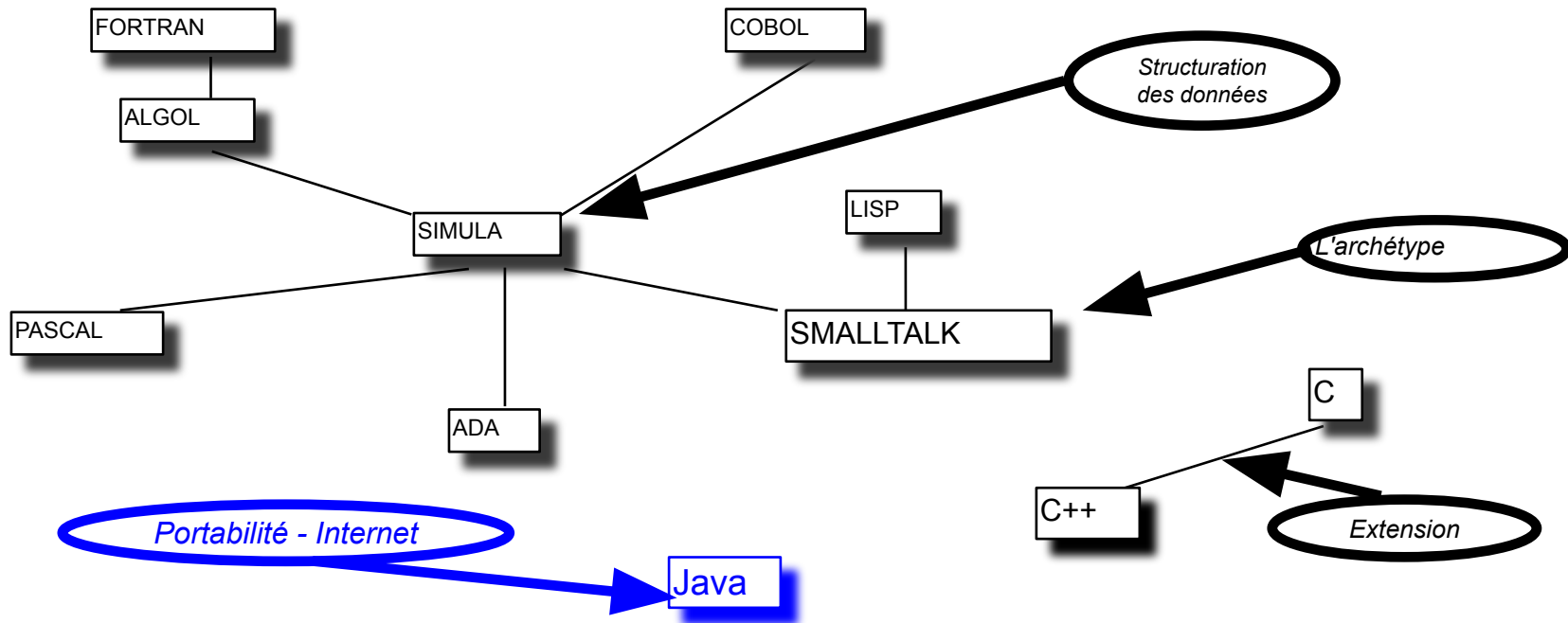
Points additionnels de l'OO

- Le typage : des objets de types différents ne peuvent être mis en relation que de façon très restrictive
 - plusieurs catégories de typage : faible ou fort, statique ou dynamique
- La simultanéité :
 - un élément est activé quand on utilise ses services
 - on distingue les objets actifs de ceux qui ne le sont pas
 - plusieurs objets peuvent être actifs en même temps

Points additionnels de l'OO

- Le typage : des objets de types différents ne peuvent être mis en relation que de façon très restrictive
 - plusieurs catégories de typage : faible ou fort, statique ou dynamique
- La simultanéité :
 - un élément est activé quand on utilise ses services
 - on distingue les objets actifs de ceux qui ne le sont pas
 - plusieurs objets peuvent être actifs en même temps
- La persistance :
 - un élément existe habituellement et est accessible dans une portion de code qu'on appelle sa portée
 - au plus tard, il disparaît à la fin de l'exécution du programme
 - un objet peut persister après l'exécution du programme qui l'a créé, et peut être exploité par plusieurs applications différentes

Historique des langages OO



Notion d'objet

- Un objet est un élément complètement identifié et possédant des caractéristiques bien définies
 - Exemples : ma voiture, cette maison, le dernier de la liste de mes élèves, l'entier i de ma fonction f

Notion d'objet

- Un objet est un élément complètement identifié et possédant des caractéristiques bien définies
 - Exemples : ma voiture, cette maison, le dernier de la liste de mes élèves, l'entier i de ma fonction f
- Les termes instance et objet sont synonymes

Notion d'objet

- Un objet est un élément complètement identifié et possédant des caractéristiques bien définies
 - Exemples : ma voiture, cette maison, le dernier de la liste de mes élèves, l'entier i de ma fonction f
- Les termes instance et objet sont synonymes
- Un objet est caractérisé par
 - un état : les valeurs associées aux attributs
 - Exemple : ma voiture est grise, elle est arrêtée, sa marque est Ford
 - un comportement : les services qu'il peut rendre (qui deviennent des méthodes à invoquer)
 - Exemple : ce cercle peut être déplacé
 - une identité : ce qui le distingue de tous les autres objets
 - Exemple : le point $p1$, l'origine du plan

Notion de classe

- Une classe permet de définir de façon abstraite un ensemble d'objets ayant des états et des comportements similaires : on dit qu'elle en est une abstraction
 - exemple : le type "int" est une abstraction des données numériques entières, sur lesquelles on peut effectuer des opérations arithmétiques

Notion de classe

- Une classe permet de définir de façon abstraite un ensemble d'objets ayant des états et des comportements similaires : on dit qu'elle en est une abstraction
 - exemple : le type "int" est une abstraction des données numériques entières, sur lesquelles on peut effectuer des opérations arithmétiques
- Une classe définit :
 - une structure commune
 - Exemple : toutes les voitures sont caractérisées par leur marque, leur puissance, leur numéro d'immatriculation, ...
 - des opérations : les services rendus par tous les objets
 - Exemple : toutes les voitures permettent de se déplacer, de changer de vitesse, ...

Instanciation

- La relation qui lie un objet à sa classe est une instanciation
- C'est un mécanisme qui permet de créer un objet à partir du modèle ("moule") général qu'est sa classe
 - cela se traduit par la déclaration d'une variable possédant une structure de données précise
- Un objet est l'instance d'une et une seule classe
- Certaines classes correspondent à des concepts auxquels ne correspond aucun objet dans la réalité
 - on peut toutefois définir des services que la classe peut proposer
 - exemple : une figure géométrique, pour laquelle existe le calcul d'un périmètre
 - on distingue ainsi les classes concrètes des classes abstraites (que l'on reverra avec l'héritage)

Typage faible ou fort

- Les classes sont des types, des structures de données
 - un objet est créé à partir d'une classe, donc a une structure prédéfinie
- Certains langages (Lisp, Perl, Python) n'imposent pas le choix d'un type à la déclaration des données
 - ce n'est pas le cas des langages objets
- Suivant les langages, les données peuvent (ou pas) être interprétées de types différents
 - par mécanisme de conversion ou de cast
 - par un mécanisme d'héritage
- On distingue les langages faiblement typés (C, C++, Pascal) de ceux qui sont fortement typés (Java)
 - les langages objets permettent de retrouver le type des objets en exécution (typage dynamique)

Encapsulation

- Consiste à masquer aux utilisateurs les états et le fonctionnement interne d'un objet
 - séparation interface (contrat de services rendus) et implémentation (description de comment ils sont réalisés)
 - ne montrer que l'essentiel : l'interface et on cache ainsi les détails d'implémentation

Encapsulation

- Consiste à masquer aux utilisateurs les états et le fonctionnement interne d'un objet
- L'encapsulation s'appuie sur la notion de visibilité
 - la visibilité d'un attribut ou d'une opération permet de déterminer qui pourra y accéder
 - un attribut privé n'est pas vu de l'extérieur de la classe
 - une méthode publique fait partie de l'interface de la classe
 - on peut définir des opérations privées si elles ne participent qu'au fonctionnement interne de la classe

Encapsulation

- Consiste à masquer aux utilisateurs les états et le fonctionnement interne d'un objet
- L'encapsulation s'appuie sur la notion de visibilité
- Intérêts :
 - sécurité et intégrité des données
 - en offrant des méthodes de modifications ou d'accès des données, on contrôle les manipulations effectuées
 - faciliter les évolutions de services
 - si on garde la même définition de service, on peut changer son implémentation sans impacter les utilisateurs du service
 - exemple : on ajoute à la classe Voiture le niveau d'huile
 - les services existants ne changent pas de définition, seulement d'implémentation
 - se déplacer impact le niveau d'huile
 - on peut enrichir les services offerts avec le service de gestion de l'huile en ajoutant `vidanger()`

Construire et détruire

- Avant d'utiliser un objet informatique, il faut le créer
 - en programmation, cela consiste à allouer de la mémoire associée à une variable
 - chaque objet possède son propre espace de mémoire
 - au moment de la construction de l'objet, on peut en définir des valeurs initiales
 - cela se fait dans une portion de code nommée constructeur
- Quand un objet n'est plus utilisé dans un programme, il doit être détruit, de façon à ne pas consommer de mémoire inutilement
 - en Java, la destruction se fait de façon automatique par un ramasse-miettes : "*garbage collector*"

Construire et détruire

- Avant d'utiliser un objet informatique, il faut le créer
 - en programmation, cela consiste à allouer de la mémoire associée à une variable
 - chaque objet possède son propre espace de mémoire
 - au moment de la construction de l'objet, on peut en définir des valeurs initiales
 - cela se fait dans une portion de code nommée constructeur
- Quand un objet n'est plus utilisé dans un programme, il doit être détruit, de façon à ne pas consommer de mémoire inutilement
 - en Java, la destruction se fait de façon automatique par un ramasse-miettes : "*garbage collector*"

Activation d'un objet

- Les objets communiquent par messages
 - un objet qui reçoit un message devient actif
- Les opérations sont les services qu'on demande à un objet
 - elles déterminent le comportement des instances de la classe
 - une méthode est l'implémentation d'une opération
 - une opération en modélisation = une méthode en Java
 - un message : demande d'exécution d'une opération par l'objet qui le reçoit : appel de méthode sur un objet
- Différents types de messages :
 - synchrone : l'appelant attend la fin de l'exécution de l'opération
 - asynchrone : l'appelant n'attend pas la fin de l'exécution de l'opération et poursuit son traitement, plusieurs objets sont alors actifs en même temps
 - on n'utilise généralement que des messages synchrones

Activation d'un objet

- Les objets communiquent par messages
 - un objet qui reçoit un message devient actif
- Les opérations sont les services qu'on demande à un objet
 - elles déterminent le comportement des instances de la classe
 - une méthode est l'implémentation d'une opération
 - une opération en modélisation = une méthode en Java
 - un message : demande d'exécution d'une opération par l'objet qui le reçoit : appel de méthode sur un objet
- Différents types de messages :
 - synchrone : l'appelant attend la fin de l'exécution de l'opération
 - asynchrone : l'appelant n'attend pas la fin de l'exécution de l'opération et poursuit son traitement, plusieurs objets sont alors actifs en même temps
 - on n'utilise généralement que des messages synchrones

Activation d'un objet

- Les objets communiquent par messages
 - un objet qui reçoit un message devient actif
- Les opérations sont les services qu'on demande à un objet
 - elles déterminent le comportement des instances de la classe
 - une méthode est l'implémentation d'une opération
 - une opération en modélisation = une méthode en Java
 - un message : demande d'exécution d'une opération par l'objet qui le reçoit : appel de méthode sur un objet
- Différents types de messages :
 - synchrone : l'appelant attend la fin de l'exécution de l'opération
 - asynchrone : l'appelant n'attend pas la fin de l'exécution de l'opération et poursuit son traitement, plusieurs objets sont alors actifs en même temps
 - on n'utilise généralement que des messages synchrones

Élément de classe ou d'instance

- Chaque objet possède ses propres caractéristiques, ses propres attributs
 - ce sont des variables d'instance
- Certaines données peuvent être communes à toutes les instances d'une même classe
 - Exemple : on veut compter le nombre d'instances d'une classe
 - cette information n'est pas caractéristique d'un objet précis
 - problématique :
 - un seul moyen pour accéder à un service : envoyer un message à un objet
 - quel objet activer pour connaître le nombre d'instances ?
 - comment connaître ce nombre si aucune instance n'a été créé ?
 - on définit des variables de classe, ainsi que des méthodes de classe qui accèderont à ces données communes
 - on invoque alors la méthode sur la classe, et non plus sur un objet

Élément de classe ou d'instance

- Chaque objet possède ses propres caractéristiques, ses propres attributs
 - ce sont des variables d'instance
- Certaines données peuvent être communes à toutes les instances d'une même classe
 - Exemple : on veut compter le nombre d'instances d'une classe
 - cette information n'est pas caractéristique d'un objet précis
 - problématique :
 - un seul moyen pour accéder à un service : envoyer un message à un objet
 - quel objet activer pour connaître le nombre d'instances ?
 - comment connaître ce nombre si aucune instance n'a été créé ?
 - on définit des variables de classe, ainsi que des méthodes de classe qui accèderont à ces données communes
 - on invoque alors la méthode sur la classe, et non plus sur un objet

Le polymorphisme

- Un objet peut être considéré comme ayant des "formes" différentes
- Cela peut se traduire par différents services proposés
 - Exemple : on peut demander à un cercle de se déplacer en lui transmettant un point, ou bien 2 coordonnées numériques, pour définir le nouveau centre
- Cela peut se traduire par des structures différentes
 - Exemple : une boîte de dialogue possède la structure d'un rectangle, mais aussi celles des composants graphiques qui réagissent à des événements
- Cela peut se traduire par des services dont on ne connaît pas l'implémentation
 - Exemple : les pilotes d'accès à une BD proposent tous la possibilité d'établir une connexion, d'exécuter une requête, mais chaque SGBD propose son implémentation du service
- Cela peut se traduire par des définitions génériques
 - Exemple : un tableau est une structure de données générique, qui possède une taille fixe, dont les composants sont de même nature, et pour lesquels on dispose d'un accès direct
 - on distingue les tableaux de points des tableaux de cercles ...

Le polymorphisme

- Un objet peut être considéré comme ayant des "formes" différentes
- Cela peut se traduire par différents services proposés
 - Exemple : on peut demander à un cercle de se déplacer en lui transmettant un point, ou bien 2 coordonnées numériques, pour définir le nouveau centre
- Cela peut se traduire par des structures différentes
 - Exemple : une boîte de dialogue possède la structure d'un rectangle, mais aussi celles des composants graphiques qui réagissent à des événements
- Cela peut se traduire par des services dont on ne connaît pas l'implémentation
 - Exemple : les pilotes d'accès à une BD proposent tous la possibilité d'établir une connexion, d'exécuter une requête, mais chaque SGBD propose son implémentation du service
- Cela peut se traduire par des définitions génériques
 - Exemple : un tableau est une structure de données générique, qui possède une taille fixe, dont les composants sont de même nature, et pour lesquels on dispose d'un accès direct
 - on distingue les tableaux de points des tableaux de cercles ...

Le polymorphisme

- Un objet peut être considéré comme ayant des "formes" différentes
- Cela peut se traduire par différents services proposés
 - Exemple : on peut demander à un cercle de se déplacer en lui transmettant un point, ou bien 2 coordonnées numériques, pour définir le nouveau centre
- Cela peut se traduire par des structures différentes
 - Exemple : une boîte de dialogue possède la structure d'un rectangle, mais aussi celles des composants graphiques qui réagissent à des événements
- Cela peut se traduire par des services dont on ne connaît pas l'implémentation
 - Exemple : les pilotes d'accès à une BD proposent tous la possibilité d'établir une connexion, d'exécuter une requête, mais chaque SGBD propose son implémentation du service
- Cela peut se traduire par des définitions génériques
 - Exemple : un tableau est une structure de données générique, qui possède une taille fixe, dont les composants sont de même nature, et pour lesquels on dispose d'un accès direct
 - on distingue les tableaux de points des tableaux de cercles ...

Le polymorphisme

- Un objet peut être considéré comme ayant des "formes" différentes
- Cela peut se traduire par différents services proposés
 - Exemple : on peut demander à un cercle de se déplacer en lui transmettant un point, ou bien 2 coordonnées numériques, pour définir le nouveau centre
- Cela peut se traduire par des structures différentes
 - Exemple : une boîte de dialogue possède la structure d'un rectangle, mais aussi celles des composants graphiques qui réagissent à des événements
- Cela peut se traduire par des services dont on ne connaît pas l'implémentation
 - Exemple : les pilotes d'accès à une BD proposent tous la possibilité d'établir une connexion, d'exécuter une requête, mais chaque SGBD propose son implémentation du service
- Cela peut se traduire par des définitions génériques
 - Exemple : un tableau est une structure de données générique, qui possède une taille fixe, dont les composants sont de même nature, et pour lesquels on dispose d'un accès direct
 - on distingue les tableaux de points des tableaux de cercles ...

Le polymorphisme

- Un objet peut être considéré comme ayant des "formes" différentes
- Cela peut se traduire par différents services proposés
 - Exemple : on peut demander à un cercle de se déplacer en lui transmettant un point, ou bien 2 coordonnées numériques, pour définir le nouveau centre
- Cela peut se traduire par des structures différentes
 - Exemple : une boîte de dialogue possède la structure d'un rectangle, mais aussi celles des composants graphiques qui réagissent à des événements
- Cela peut se traduire par des services dont on ne connaît pas l'implémentation
 - Exemple : les pilotes d'accès à une BD proposent tous la possibilité d'établir une connexion, d'exécuter une requête, mais chaque SGBD propose son implémentation du service
- Cela peut se traduire par des définitions génériques
 - Exemple : un tableau est une structure de données générique, qui possède une taille fixe, dont les composants sont de même nature, et pour lesquels on dispose d'un accès direct
 - on distingue les tableaux de points des tableaux de cercles ...

La modélisation objet

- Un modèle est une représentation du système informatique que l'on veut réaliser
- Au cours de la phase de modélisation, on construit différents modèles successifs en les affinant
 - on part des éléments de la réalité du domaine qu'on veut informatiser
 - on aboutit à des éléments informatiques qu'on pourra réaliser directement dans un environnement et dans un langage
- Démarche : à l'inverse de procédural (approche descendante), on construit le système de façon ascendante, comme un assemblage de composants unitaires
- Le formalisme graphique UML
 - ce n'est pas une méthode de modélisation
 - normalisé par l'OMG (organisme de normalisation)
 - définit différents diagrammes (statiques ou dynamiques)
 - la description du formalisme est l'objet d'un cours de S5

La modélisation objet

- Un modèle est une représentation du système informatique que l'on veut réaliser
- Au cours de la phase de modélisation, on construit différents modèles successifs en les affinant
 - on part des éléments de la réalité du domaine qu'on veut informatiser
 - on aboutit à des éléments informatiques qu'on pourra réaliser directement dans un environnement et dans un langage
- Démarche : à l'inverse de procédural (approche descendante), on construit le système de façon ascendante, comme un assemblage de composants unitaires
- Le formalisme graphique UML
 - ce n'est pas une méthode de modélisation
 - normalisé par l'OMG (organisme de normalisation)
 - définit différents diagrammes (statiques ou dynamiques)
 - la description du formalisme est l'objet d'un cours de S5

La modélisation objet

- Un modèle est une représentation du système informatique que l'on veut réaliser
- Au cours de la phase de modélisation, on construit différents modèles successifs en les affinant
 - on part des éléments de la réalité du domaine qu'on veut informatiser
 - on aboutit à des éléments informatiques qu'on pourra réaliser directement dans un environnement et dans un langage
- Démarche : à l'inverse de procédural (approche descendante), on construit le système de façon ascendante, comme un assemblage de composants unitaires
- Le formalisme graphique UML
 - ce n'est pas une méthode de modélisation
 - normalisé par l'OMG (organisme de normalisation)
 - définit différents diagrammes (statiques ou dynamiques)
 - la description du formalisme est l'objet d'un cours de S5

La modélisation objet

- Un modèle est une représentation du système informatique que l'on veut réaliser
- Au cours de la phase de modélisation, on construit différents modèles successifs en les affinant
 - on part des éléments de la réalité du domaine qu'on veut informatiser
 - on aboutit à des éléments informatiques qu'on pourra réaliser directement dans un environnement et dans un langage
- Démarche : à l'inverse de procédural (approche descendante), on construit le système de façon ascendante, comme un assemblage de composants unitaires
- Le formalisme graphique UML
 - ce n'est pas une méthode de modélisation
 - normalisé par l'OMG (organisme de normalisation)
 - définit différents diagrammes (statiques ou dynamiques)
 - la description du formalisme est l'objet d'un cours de S5

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

La technologie Java

- Java est un environnement de programmation orienté objet : un langage, une machine virtuelle (JVM), un ensemble d'APIs d'outils (jdb, javadoc, ...)
- Java est entièrement orienté objet : tout est une classe
- Portable : tout programme est traduit en un langage intermédiaire le *Java bytecode*
- Indépendant de l'architecture : le code, indépendant de la cible, est interprété par un interpréteur dans une plateforme Java
- Simple : sa syntaxe ressemble à celle de C/C++, mais en évite les principales difficultés (pointeurs, héritage multiple, ...)
 - Avantage/difficulté : un ensemble d'APIs très riche :
<http://download.oracle.com/javase/7/docs/api/index.html>
- Haute performance et allocation de mémoire réduite (taille de la JVM paramétrable)
- Robuste : pas de pointeurs, récupérateur de mémoire, compilateur très strict, exceptions
- Édition de lien dynamique : l'interpréteur retrouve dynamiquement les classes utilisées
- Multithread, distribué, sécurisé

Cycle de développement

- Une application est un ensemble de modules (packages)
- Chaque fichier peut contenir une ou plusieurs classes, mais une seule est déclarée **public** dans ce fichier
 - son nom doit être exactement le même que celui de la classe public
 - l'extension est **.java**
 - il contient la déclaration complète des classes (c'est-à-dire que l'implémentation des méthodes est incluse)
- Le compilateur crée un fichier par classe (extension **.class**) ; ces fichiers contiennent du bytecode qui pourra être utilisé quelque soit la JVM
- L'interpréteur charge, analyse syntaxiquement et exécute chaque instruction de bytecode
- L'interpréteur charge dynamiquement les classes demandées
- Les variables d'environnement : PATH, CLASSPATH, JAVA_HOME
 - elles sont automatiquement initialisées si on utilise un IDE

Premier exemple

```
class PremierExemple
{
    // La méthode principale
    public static void main(String [ ] args)
    {
        int sum = 0 ;
        for ( int i = 1 ; i < 10 ; i++)
            sum += i ;
        System.out.println("Somme des 9 : " + sum) ;
    }
}
```

Variables

- Ces entités du langage ont deux composants : le nom et le type
 - le nom d'une variable est constitué d'une série de caractères Unicode
 - par convention, leur **nom commence par une minuscule**. Si on utilise plus d'un mot, chaque mot après le premier commence par une majuscule, par exemple "`int monCompteur=0;` »
 - ne peuvent être définies que dans une classe (pas globale)
 - les numériques sont initialisées par défaut à 0, les objets à **null**
- La portée d'une variable dépend de la position de sa déclaration : il s'agit de la portion de code pendant laquelle la variable est accessible
 - quatre catégories : variable d'instance, variable locale, paramètre de méthode, paramètre de gestionnaire d'exception
- Les variables locales ou d'instance peuvent être initialisées à leur déclaration : le type de la valeur affectée doit correspondre à celui de la variable
- La valeur d'une variable "**final**" ne peut plus changer après son initialisation
 - une variable locale "**final**" déclarée mais pas encore initialisée est appelée une variable "blank final".

Variables

- Ces entités du langage ont deux composants : le nom et le type
 - le nom d'une variable est constitué d'une série de caractères Unicode
 - par convention, leur **nom commence par une minuscule**. Si on utilise plus d'un mot, chaque mot après le premier commence par une majuscule, par exemple "`int monCompteur=0;` »
 - ne peuvent être définies que dans une classe (pas globale)
 - les numériques sont initialisées par défaut à 0, les objets à **null**
- La portée d'une variable dépend de la position de sa déclaration : il s'agit de la portion de code pendant laquelle la variable est accessible
 - quatre catégories : variable d'instance, variable locale, paramètre de méthode, paramètre de gestionnaire d'exception
- Les variables locales ou d'instance peuvent être initialisées à leur déclaration : le type de la valeur affectée doit correspondre à celui de la variable
- La valeur d'une variable "**final**" ne peut plus changer après son initialisation
 - une variable locale "**final**" déclarée mais pas encore initialisée est appelée une variable "blank final".

Variables

- Ces entités du langage ont deux composants : le nom et le type
 - le nom d'une variable est constitué d'une série de caractères Unicode
 - par convention, leur **nom commence par une minuscule**. Si on utilise plus d'un mot, chaque mot après le premier commence par une majuscule, par exemple "`int monCompteur=0;` »
 - ne peuvent être définies que dans une classe (pas globale)
 - les numériques sont initialisées par défaut à 0, les objets à **null**
- La portée d'une variable dépend de la position de sa déclaration : il s'agit de la portion de code pendant laquelle la variable est accessible
 - quatre catégories : variable d'instance, variable locale, paramètre de méthode, paramètre de gestionnaire d'exception
- Les variables locales ou d'instance peuvent être initialisées à leur déclaration : le type de la valeur affectée doit correspondre à celui de la variable
- La valeur d'une variable "**final**" ne peut plus changer après son initialisation
 - une variable locale "**final**" déclarée mais pas encore initialisée est appelée une variable "blank final".

Variables

- Ces entités du langage ont deux composants : le nom et le type
 - le nom d'une variable est constitué d'une série de caractères Unicode
 - par convention, leur **nom commence par une minuscule**. Si on utilise plus d'un mot, chaque mot après le premier commence par une majuscule, par exemple "`int monCompteur=0;` »
 - ne peuvent être définies que dans une classe (pas globale)
 - les numériques sont initialisées par défaut à 0, les objets à **null**
- La portée d'une variable dépend de la position de sa déclaration : il s'agit de la portion de code pendant laquelle la variable est accessible
 - quatre catégories : variable d'instance, variable locale, paramètre de méthode, paramètre de gestionnaire d'exception
- Les variables locales ou d'instance peuvent être initialisées à leur déclaration : le type de la valeur affectée doit correspondre à celui de la variable
- La valeur d'une variable "**final**" ne peut plus changer après son initialisation
 - une variable locale "**final**" déclarée mais pas encore initialisée est appelée une variable "blank final".

Types de données

- Détermine les valeurs qu'une variable peut contenir et les opérations qu'on peut lui appliquer

Mot-clé	Description	Taille/Format
<i>(integers)</i>		
byte	Entier sur un octet	8-bit two's complement
short	Entier court	16-bit two's complement
int	Entier	32-bit two's complement
long	Entier long	64-bit two's complement
<i>(nombres réels)</i>		
float	Flottant en précision simple	32-bit IEEE 754
double	Flottant en précision double	64-bit IEEE 754
<i>(other types)</i>		
char	Caractère simple	16-bit Unicode character
boolean	Valeur booléenne	true or false

Expressions, instructions et blocs

- Une expression est une combinaison de variables, opérateurs et appels de méthodes (construites suivant la syntaxe du langage) et qui s'évalue en une valeur unique
- Une instruction est une unité d'exécution complète et est terminée par un point-virgule
- On peut regrouper zéro ou plusieurs instructions ensemble dans un bloc encadré d'accolades

Opérateurs

- **Opérateurs arithmétiques**

`+`, `-`, `*`, `/`, et `%` (modulo)

versions unaires `+` et `-`

opérateurs raccourcis d'incrémentation/décrémentation `++` et `--`

- Opérateurs relationnels and conditionnels

`>`, `>=`, `<`, `<=`, `==`, `!=`

`&&`, `||`, `!`, `&`, `|`, `^`

- Opérateurs de décalage et logiques : `>>`, `<<`, `>>>`

- Opérateurs sur les bits : `&`, `|`, `^`, `~`

- Opérateurs d'affectation : `=`, `+=`, `-=`, ...

- Autres opérateurs

`?:`, `[]`, `..`, `()`, `(type)`, `new`, `instanceof`

Opérateurs

- **Opérateurs arithmétiques**

$+$, $-$, $*$, $/$, et $\%$ (modulo)

versions unaires $+$ et $-$

opérateurs raccourcis d'incrément/décroissement $++$ et $--$

- **Opérateurs relationnels et conditionnels**

$>$, $>=$, $<$, $<=$, $==$, $!=$

$\&\&$, $||$, $!$, $\&$, $|$, $^$

- Opérateurs de décalage et logiques : $>>$, $<<$, $>>>$

- Opérateurs sur les bits : $\&$, $|$, $^$, $\sim$

- Opérateurs d'affectation : $=$, $+=$, $-=$, ...

- Autres opérateurs

$?:$, $[]$, $..$, $()$, $(type)$, new , $instanceof$

Opérateurs

- **Opérateurs arithmétiques**

$+$, $-$, $*$, $/$, et $\%$ (modulo)

versions unaires $+$ et $-$

opérateurs raccourcis d'incrément/décrémentation $++$ et $--$

- **Opérateurs relationnels et conditionnels**

$>$, $>=$, $<$, $<=$, $==$, $!=$

$\&\&$, $||$, $!$, $\&$, $|$, $^$

- **Opérateurs de décalage et logiques : $>>$, $<<$, $>>>$**

- Opérateurs sur les bits : $\&$, $|$, $^$, $\sim$

- Opérateurs d'affectation : $=$, $+=$, $-=$, ...

- Autres opérateurs

$?:$, $[]$, $..$, $()$, $(type)$, new , $instanceof$

Opérateurs

- **Opérateurs arithmétiques**

$+$, $-$, $*$, $/$, et $\%$ (modulo)

versions unaires $+$ et $-$

opérateurs raccourcis d'incrément/décrémentation $++$ et $--$

- **Opérateurs relationnels et conditionnels**

$>$, $>=$, $<$, $<=$, $==$, $!=$

$\&\&$, $||$, $!$, $\&$, $|$, $^$

- **Opérateurs de décalage et logiques : $>>$, $<<$, $>>>$**

- **Opérateurs sur les bits : $\&$, $|$, $^$, $\sim$**

- **Opérateurs d'affectation : $=$, $+=$, $-=$, ...**

- **Autres opérateurs**

$?:$, $[]$, $..$, $()$, $(type)$, new , $instanceof$

Opérateurs

- **Opérateurs arithmétiques**

`+`, `-`, `*`, `/`, et `%` (modulo)

versions unaires `+` et `-`

opérateurs raccourcis d'incrémentation/décrémentation `++` et `--`

- **Opérateurs relationnels et conditionnels**

`>`, `>=`, `<`, `<=`, `==`, `!=`

`&&`, `||`, `!`, `&`, `|`, `^`

- **Opérateurs de décalage et logiques : `>>`, `<<`, `>>>`**

- **Opérateurs sur les bits : `&`, `|`, `^`, `~`**

- **Opérateurs d'affectation : `=`, `+=`, `-=`, ...**

- **Autres opérateurs**

`?:`, `[]`, `.`, `()`, `(type)`, `new`, `instanceof`

Priorité des opérateurs

Comment estimer l'opération $op=1-2-3$?

Priorité des opérateurs

Comment estimer l'opération $op=1-2-3$?

Résultat -4 ou 2 ?

Priorité des opérateurs

Comment estimer l'opération $op=1-2-3$?

Résultat -4 ou 2 ?

Doit-on évaluer l'expression de gauche à droite et commencer par $1 - 2$ ($op=-4$) ou de droite à gauche et commencer par $2 - 3$ ($op=2$) ?

Priorité des opérateurs

Comment estimer l'opération $op=1-2-3$?

Résultat -4 ou 2 ?

Doit-on évaluer l'expression de gauche à droite et commencer par $1 - 2$ ($op=-4$) ou de droite à gauche et commencer par $2 - 3$ ($op=2$) ?

C'est la règle d'associativité qui dicte cela. Cette règle définit comment des expressions dont tous les opérateurs ont le même niveau de priorité sont évaluées.

Priorité des opérateurs

Comment estimer l'opération $op=1-2-3$?

Résultat -4 ou 2 ?

Doit-on évaluer l'expression de gauche à droite et commencer par $1 - 2$ ($op=-4$) ou de droite à gauche et commencer par $2 - 3$ ($op=2$) ?

C'est la règle d'associativité qui dicte cela. Cette règle définit comment des expressions dont tous les opérateurs ont le même niveau de priorité sont évaluées.

L'opérateur de soustraction est associatif de gauche à droite, il faut donc évaluer l'expression précédente de la première manière ; sa valeur est donc -4.

Priorité des opérateurs

Opérateurs postfix	[] . (params) expr++ expr--	Gauche à droite
Opérateurs unaires	++expr --expr +expr -expr ~ !	Gauche à droite
Création ou cast	new (type)expr	Gauche à droite
Multiplicatifs	* / %	Gauche à droite
Additifs	+ -	Gauche à droite
Décalage	<< >> >>>	Gauche à droite
Relationnel	< > <= >= instanceof	Gauche à droite
Egalité	== !=	Gauche à droite
Et bit à bit	&	Gauche à droite
Ou bit à bit	^	Gauche à droite
Ou inclusif bit à bit		Gauche à droite
Et logique	&&	Gauche à droite
Ou logique		Gauche à droite
Conditionnel	? :	Gauche à droite
Affectation	= += -= *= /= %= &= ^= = <<= >>= >>>=	Droite à gauche

Instructions de contrôle

- Décision :
 - if-else : exécution d'instructions suivant la valeur d'une condition
 - switch-case : exécution d'instructions conditionnée par la valeur d'une expression entière
- Boucle :
 - while, do-while : exécution continue d'une instruction tant qu'une condition reste vraie
 - for : itération sur un ensemble de valeurs
- Branchement :
 - break : la version sans étiquette interrompt l'instruction switch, for, while, ou do-while la plus imbriquée ; la version avec étiquette interrompt une instruction plus englobante identifiée par une étiquette
 - continue : (avec ou sans étiquette) saute l'itération courant d'une boucle for, while, ou do-while
 - return : pour sortir de la méthode courant
- Gestion d'exception :
 - try : identifie un bloc dans lequel une exception peut être lancée
 - catch : identifie un bloc d'instructions gérant un type particulier d'exception
 - throw : pour lancer une exception

Tableaux

- Structure de données de taille fixe, dont les composants sont tous de même type ; tout type de données est possible, y compris tableau

Tableaux

- Structure de données de taille fixe, dont les composants sont tous de même type ; tout type de données est possible, y compris tableau
- Création
 - déclaration d'une variable référençant un tableau : `int[] tab;`
 - création du tableau :
 - en utilisant l'opérateur `new` : `unTableau = new int[10];`
 - en utilisant un initialiseur : `int[] tab = {1, 3, 5, 2, 10};`

Tableaux

- Structure de données de taille fixe, dont les composants sont tous de même type ; tout type de données est possible, y compris tableau
- Création
 - déclaration d'une variable référençant un tableau : `int[] tab;`
 - création du tableau :
 - en utilisant l'opérateur `new` : `unTableau = new int[10];`
 - en utilisant un initialiseur : `int[] tab = {1,3,5,2,10};`
- Utilisation
 - accès à un élément : `tab[i]`
 - l'indice commence à 0 et va jusqu'à la taille du tableau moins 1
 - obtenir la taille d'un tableau : `tab.length`

Tableaux

- Structure de données de taille fixe, dont les composants sont tous de même type ; tout type de données est possible, y compris tableau
- Création
 - déclaration d'une variable référençant un tableau : `int[] tab;`
 - création du tableau :
 - en utilisant l'opérateur `new` : `unTableau = new int[10];`
 - en utilisant un initialiseur : `int[] tab = {1,3,5,2,10};`
- Utilisation
 - accès à un élément : `tab[i]`
 - l'indice commence à 0 et va jusqu'à la taille du tableau moins 1
 - obtenir la taille d'une tableau : `tab.length`
- Tableau de tableaux : `uneMatrice[i][j]`

Tableaux

- Structure de données de taille fixe, dont les composants sont tous de même type ; tout type de données est possible, y compris tableau
- Création
 - déclaration d'une variable référençant un tableau : `int[] tab;`
 - création du tableau :
 - en utilisant l'opérateur `new` : `unTableau = new int[10];`
 - en utilisant un initialiseur : `int[] tab = {1,3,5,2,10};`
- Utilisation
 - accès à un élément : `tab[i]`
 - l'indice commence à 0 et va jusqu'à la taille du tableau moins 1
 - obtenir la taille d'une tableau : `tab.length`
- Tableau de tableaux : `uneMatrice[i][j]`
- Copie de tableaux : méthode `arraycopy` de la classe `System`
`System.arraycopy(copyFrom, 2, copyTo, 0, 7);`

String

- Une string peut être créée à partir d'un littéral chaîne de caractères

```
String uneString = "Dot saw I was Tod";
```

String

- Une string peut être créée à partir d'un littéral chaîne de caractères

```
String uneString = "Dot saw I was Tod";
```

- ou d'un tableau de caractères

```
char[] tableauHello = {'h','e','l','l','o'};  
String stringHello = new String(tableauHello);
```

String

- Une string peut être créée à partir d'un littéral chaîne de caractères

```
String uneString = "Dot saw I was Tod";
```

- ou d'un tableau de caractères

```
char[] tableauHello = {'h','e','l','l','o'};  
String stringHello = new String(tableauHello);
```

- Obtenir la longueur d'une string

```
int len = aString.length();
```


String

- Une string peut être créée à partir d'un littéral chaîne de caractères

```
String uneString = "Dot saw I was Tod";
```

- ou d'un tableau de caractères

```
char[] tableauHello = {'h','e','l','l','o'};  
String stringHello = new String(tableauHello);
```

- Obtenir la longueur d'une string

```
int len = aString.length();
```

- Obtenir un caractère par son indice dans la string

```
char aChar = aString.charAt(3);
```

String

- Une string peut être créée à partir d'un littéral chaîne de caractères

```
String uneString = "Dot saw I was Tod";
```

- ou d'un tableau de caractères

```
char[] tableauHello = {'h','e','l','l','o'};  
String stringHello = new String(tableauHello);
```

- Obtenir la longueur d'une string

```
int len = aString.length();
```

- Obtenir un caractère par son indice dans la string

```
char aChar = aString.charAt(3);
```

- Concaténation : l'opérateur +

```
String s1 = "hello";  
s1 += "world";
```

```
String s2 = s1;  
s1 += 45;
```

La méthode main

- Un programme Java est constitué de plusieurs classes
 - uniquement de classes, pas de méthode globale
 - une seule d'entre elles possède la méthode principale (**main** en anglais)
 - le programme est une instance de cette classe
 - cette méthode est la première invoquée, automatiquement
- La signature de la méthode principale comprend 3 "modificateurs"
 - **public** qui indique qu'elle peut être appelée par n'importe quel objet
 - **static** qui indique que c'est une méthode de classe
 - **void** qui indique que cette méthode ne retourne pas de résultat
- La méthode principale n'a qu'un seul argument, un tableau de chaînes de caractères (String)
 - en lançant l'application, on peut préciser des arguments en ligne de commande
 - l'environnement d'exécution Java ne transmet pas le nom de la classe invoquée
 - on peut connaître le nombre d'arguments transmis grâce à l'attribut length du tableau :
`nombreArguments = args.length;`

La méthode main

- Un programme Java est constitué de plusieurs classes
 - uniquement de classes, pas de méthode globale
 - une seule d'entre elles possède la méthode principale (**main** en anglais)
 - le programme est une instance de cette classe
 - cette méthode est la première invoquée, automatiquement
- La signature de la méthode principale comprend 3 "modificateurs"
 - **public** qui indique qu'elle peut être appelée par n'importe quel objet
 - **static** qui indique que c'est une méthode de classe
 - **void** qui indique que cette méthode ne retourne pas de résultat
- La méthode principale n'a qu'un seul argument, un tableau de chaînes de caractères (String)
 - en lançant l'application, on peut préciser des arguments en ligne de commande
 - l'environnement d'exécution Java ne transmet pas le nom de la classe invoquée
 - on peut connaître le nombre d'arguments transmis grâce à l'attribut length du tableau :
`nombreArguments = args.length;`

La méthode main

- Un programme Java est constitué de plusieurs classes
 - uniquement de classes, pas de méthode globale
 - une seule d'entre elles possède la méthode principale (**main** en anglais)
 - le programme est une instance de cette classe
 - cette méthode est la première invoquée, automatiquement
- La signature de la méthode principale comprend 3 "modificateurs"
 - **public** qui indique qu'elle peut être appelée par n'importe quel objet
 - **static** qui indique que c'est une méthode de classe
 - **void** qui indique que cette méthode ne retourne pas de résultat
- La méthode principale n'a qu'un seul argument, un tableau de chaînes de caractères (String)
 - en lançant l'application, on peut préciser des arguments en ligne de commande
 - l'environnement d'exécution Java ne transmet pas le nom de la classe invoquée
 - on peut connaître le nombre d'arguments transmis grâce à l'attribut length du tableau :
`nombreArguments = args.length;`

Entrées et sorties

- Écrire une information :

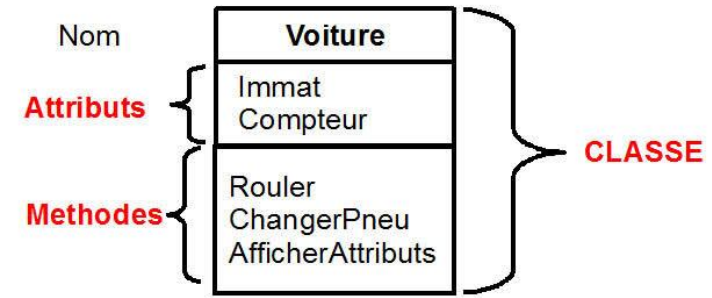
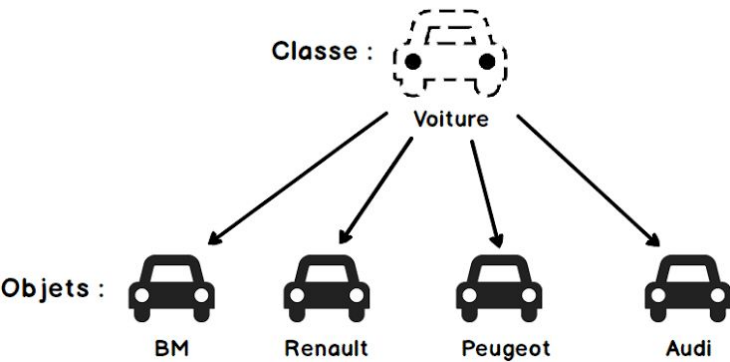
```
System.out.println(La chaîne que vous voulez écrire);
```

- Pour lire un caractère : **import** java.io.IOException;

```
char c=0;  
try  
{  
    c=(char) System.in.read();  
}  
catch (IOException e) { }
```

- Pour lire des valeurs de type primitif
 - faire une boucle de lecture d'un caractère et le stocker dans une String (fin de ligne : 10)
 - utiliser les conversions de String vers un type primitif

```
int i = Integer.parseInt(s) où s est un objet String
```



Classes, objets et classes utilitaires

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

```
public class Point
{
    private int x , y;

    public void init(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public void affiche() {
        System.out.println("x : "+x+", y : " +y);
    }
}
```


Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

public class Point ← Déclaration de classe

```
{  
    private int x , y;  
  
    public void init(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public void affiche() {  
        System.out.println("x : "+x+", y : " +y);  
    }  
}
```

Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

```
public class Point
```

```
{
```

```
    private int x , y;
```

```
    public void init(int x, int y) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
    }
```

```
    public void affiche() {
```

```
        System.out.println("x : "+x+", y : " +y);
```

```
    }
```

```
}
```



Corps de la classe

Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

```
public class Point
{
    private int x , y;

    public void init(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public void affiche() {
        System.out.println("x : "+x+", y : " +y);
    }
}
```

← Pas de ; pour finir la déclaration de la classe

Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

```
public class Point
{
    private int x , y;    ← Variables
    public void init(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public void affiche() {
        System.out.println("x : "+x+", y : " +y);
    }
}
```

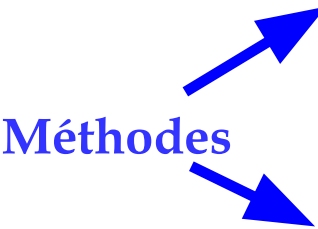
Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"

```
public class Point
{
    private int x , y;

    public void init(int x, int y) {
        this.x = x;
        this.y = y;
    }

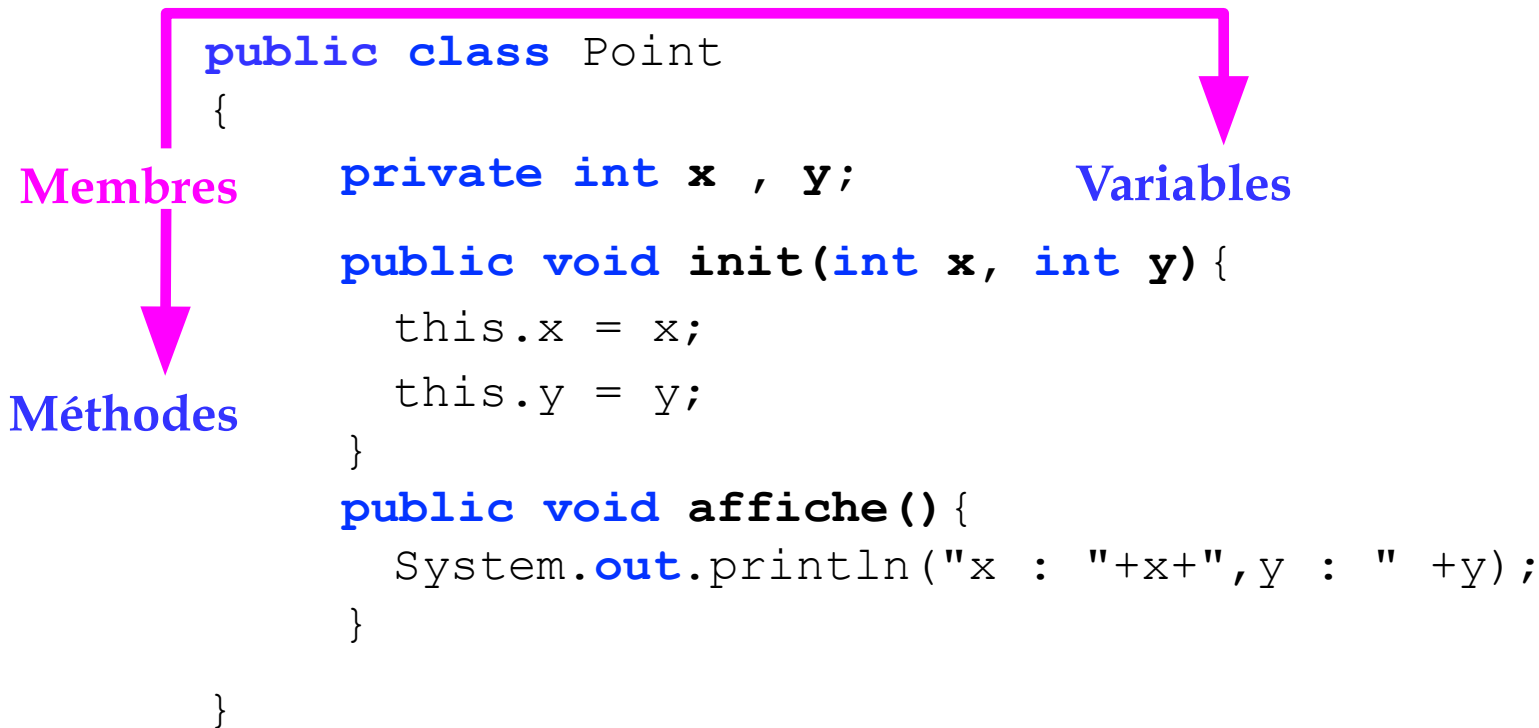
    public void affiche() {
        System.out.println("x : "+x+", y : " +y);
    }
}
```



Méthodes

Définition de classes

Une classe est un prototype d'objets : elle définit un nouveau "type"



Objets

- On déclare une variable qui fera référence à un objet

```
Point unPoint ;
```

Objets

- On déclare une variable qui fera référence à un objet

```
Point unPoint ;
```

- On instancie un objet en utilisant l'opérateur new

```
unPoint = new Point();
```


Objets

- On déclare une variable qui fera référence à un objet

```
Point unPoint ;
```

- On instancie un objet en utilisant l'opérateur new

```
unPoint = new Point();
```

- **Initialisation**

L'opérateur **new** est suivi d'un appel au constructeur

Objets

- On déclare une variable qui fera référence à un objet

```
Point unPoint ;
```

- On instancie un objet en utilisant l'opérateur new

```
unPoint = new Point();
```

- Initialisation

L'opérateur **new** est suivi d'un appel au constructeur

- On utilise les objets

- en faisant référence à ses variables

```
unPoint.x
```

- en invoquant ses méthodes

```
unPoint.init(3, 4);
```

```
unPoint.affiche();
```

Constructeurs

- **Un constructeur est un moyen d'initialiser les variables d'instance**
 - il ne construit pas l'objet, il l'initialise
 - il porte le même nom que la classe
 - il n'a pas de type de retour
 - il n'est pas obligatoire de fournir un constructeur : il en existe un par défaut pour toute classe qui n'a pas de constructeur
 - si un constructeur est défini, plus de comportement par défaut
- **Java supporte la surcharge des noms de constructeurs**
 - à la création d'un objet, on choisit le constructeur approprié
- **On utilise un spécificateur d'accès pour préciser quels autres objets ont le droit de créer des instances**
 - **private** : aucune autre classe ne peut instancier cette classe
 - **protected** : uniquement les classes dérivées ou définies dans le même paquetage
 - **public** : toute classe peut créer une instance de cette classe
 - pas de spécificateur : seules les classes du paquetage ont accès au constructeur

Constructeurs

- **Un constructeur est un moyen d'initialiser les variables d'instance**
 - il ne construit pas l'objet, il l'initialise
 - il porte le même nom que la classe
 - il n'a pas de type de retour
 - il n'est pas obligatoire de fournir un constructeur : il en existe un par défaut pour toute classe qui n'a pas de constructeur
 - si un constructeur est défini, plus de comportement par défaut
- **Java supporte la surcharge des noms de constructeurs**
 - à la création d'un objet, on choisit le constructeur approprié
- **On utilise un spécificateur d'accès pour préciser quels autres objets ont le droit de créer des instances**
 - **private** : aucune autre classe ne peut instancier cette classe
 - **protected** : uniquement les classes dérivées ou définies dans le même paquetage
 - **public** : toute classe peut créer une instance de cette classe
 - pas de spécificateur : seules les classes du paquetage ont accès au constructeur

Constructeurs

- **Un constructeur est un moyen d'initialiser les variables d'instance**
 - il ne construit pas l'objet, il l'initialise
 - il porte le même nom que la classe
 - il n'a pas de type de retour
 - il n'est pas obligatoire de fournir un constructeur : il en existe un par défaut pour toute classe qui n'a pas de constructeur
 - si un constructeur est défini, plus de comportement par défaut
- **Java supporte la surcharge des noms de constructeurs**
 - à la création d'un objet, on choisit le constructeur approprié
- **On utilise un spécificateur d'accès pour préciser quels autres objets ont le droit de créer des instances**
 - **private** : aucune autre classe ne peut instancier cette classe
 - **protected** : uniquement les classes dérivées ou définies dans le même paquetage
 - **public** : toute classe peut créer une instance de cette classe
 - pas de spécificateur : seules les classes du paquetage ont accès au constructeur

Constructeur : exemple

```
public class Point {  
  
    private int x , y;  
  
    public Point() {  
        this.x = this.y = 0;  
    }  
  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println(x + "-" +  
y);  
    }  
}
```

Constructeur : exemple

Constructeur sans argument

```
public class Point {  
  
    private int x , y;  
  
    public Point() {  
        this.x = this.y = 0;  
    }  
  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println(x + "-" +  
y);  
    }  
}
```

Constructeur : exemple

```
public class Point {
```

```
    private int x , y;
```

```
    public Point() {  
        this.x = this.y = 0;  
    }
```

Constructeur avec arguments

```
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }
```

```
    public void affiche() {  
        System.out.println(x + "-" +  
y);  
    }
```


Constructeur : exemple

```
public class Point {  
    private int x , y;  
  
    public Point() {  
        this.x = this.y = 0;  
    }  
  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println(x + "-" + y);  
    }  
}  
  
public class Exemple {  
    public static void main(String[] args){  
        Point p1=new Point();  
        p1.affiche();  
  
        Point p2=new Point(1,2);  
        p2.affiche();  
    }  
}
```

Constructeur : exemple

```
public class Point {  
  
    private int x , y;  
  
    public Point() {  
        this.x = this.y = 0;  
    }  
  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println(x + "-" +  
y);  
    }  
}
```

```
! public class Exemple {  
!  
! public static void main(String[] args){  
!  
!     Point p1=new Point();  
!     p1.affiche();  
!  
!     Point p2=new Point(1,2);  
!     p2.affiche();  
!  
!     }  
!  
! }
```

Constructeur : exemple

```
public class Point {
```

```
    private int x , y;
```

```
    public Point() {  
        this.x = this.y = 0;  
    }
```

```
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }
```

```
    public void affiche() {
```

```
        System.out.println(x + "-" +
```

```
y);  
    }
```

```
! public class Exemple {
```

```
! . public static void main(String[] args){
```

```
! . Point p1=new Point();  
! . p1.affiche();
```

```
! . Point p2=new Point(1,2);  
! . p2.affiche();
```

```
! . }
```

```
! }
```

```
! .
```

```
! }
```

Constructeur : exemple

```
public class Point {  
  
    private int x , y;  
  
    public Point() {  
        this.x = this.y = 0;  
    }  
  
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println(x + "-" +  
y);  
    }  
}
```

```
! public class Exemple {  
!  
! public static void main(String[] args){  
!  
!     Point p1=new Point();  
!     p1.affiche();  
!  
!     Point p2=new Point(1,2);  
!     p2.affiche();  
!  
!     }  
!  
! }
```

Constructeur : exemple

```
public class Point {
```

```
    private int x , y;
```

```
    public Point() {  
        this.x = this.y = 0;  
    }
```

```
    public Point(int x, int y){  
        this.x = x;  
        this.y = y;  
    }
```

```
    public void affiche() {
```

```
        System.out.println(x + "-" +
```

```
y);  
    }
```

```
public class Exemple {
```

```
    public static void main(String[] args){
```

```
        Point p1=new Point();
```

```
        p1.affiche();
```

```
        Point p2=new Point(1,2);
```

```
        p2.affiche();
```

```
    }
```

```
}
```

Variable d'instance ou de classe

- **Variable d'instance**

- à la création d'un objet, une copie de chaque variable d'instance est créée
- les méthodes d'instance travaillent sur les variables d'instance de l'objet courant (celui sur lequel la méthode a été invoquée), mais peuvent aussi accéder aux variables de la classe

Variable d'instance ou de classe

- **Variable d'instance**

- à la création d'un objet, une copie de chaque variable d'instance est créée
- les méthodes d'instance travaillent sur les variables d'instance de l'objet courant (celui sur lequel la méthode a été invoquée), mais peuvent aussi accéder aux variables de la classe

- **Variable de classe**

- allouée une seule fois par classe, la première fois que le système a affaire à la classe. Toutes les instances partagent la même copie des variables de classe
- les méthodes de classe n'ont pas accès aux variables d'instances déclarées dans la même classe
- les méthodes de classe sont invoquées sur la classe
`NomDeClasse.methode(arg)`

Déclaration des variables

- On peut spécifier un niveau d'accès, que la variable soit de classe ou d'instance ou que ce soit une constante
- Spécificateurs d'accès
 - **private** : accessible uniquement dans la classe où elle est déclarée
 - **protected** : autorise l'accès aux membres spécifiés à la classe courante, ses classes dérivées et toutes les classes du même paquetage
 - **public** : toute classe a accès aux membres publiques d'une classe
 - pas de spécificateur : toutes les classes du même paquetage ont accès à ces membres
- Le mot-clé **static** déclare un membre de classe
- Le mot-clé **final** indique que la valeur ne peut pas changer
- On peut utiliser des initialiseurs pour initialiser les variables (instance ou classe)

Déclaration de méthode

- **La signature d'une méthode**
 - à la déclaration, on précise le nom de la méthode et le type de la valeur qu'elle retourne
 - mot-clé **void** : la méthode ne retourne pas de valeur
 - on déclare le nom et le type de chaque argument dans la signature : tout type (classe) Java peut être passé en argument
 - spécif. d'accès + **static** + **abstract** + **final** + **return** type + nom + liste de paramètres + clause **throws**
- **Le résultat d'une méthode**
 - on utilise l'instruction **return** pour retourner une valeur ; le type de l'expression retournée doit être compatible avec le type du résultat déclaré dans la signature de la méthode
- Il n'y a pas de séparation entre la déclaration et la définition
- **Java supporte la surcharge des méthodes : elles sont différenciées par le nombre et le type des arguments qui lui seront passés**

Portée des variables

- Les arguments sont passés par valeur aux méthodes
 - si l'argument est de type primitif, la méthode n'en change pas la valeur
 - si l'argument est une référence d'objet, la méthode ne peut pas changer cette référence d'objet, mais, en invoquant les méthodes sur cet objet, elle peut éventuellement en modifier certaines données
- Variables locales
 - on peut déclarer des variables dans le corps d'une méthode, qui seront locales à cette méthode ; ces variables existent tant que le contrôle reste dans la méthode
- L'argument d'une méthode peut avoir le même nom qu'une variable d'instance
 - on utilise le mot-clé **this** dans la méthode pour faire référence à l'objet sur lequel la méthode a été invoquée

Variables : exemple

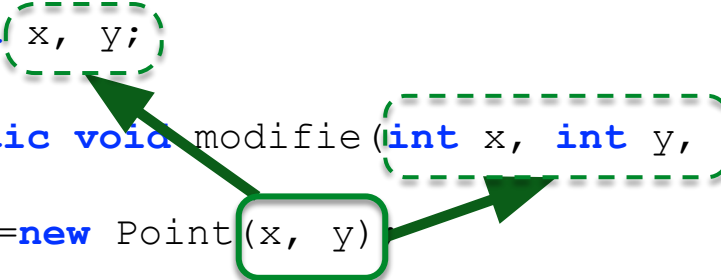
```
public class Point{  
    private int x, y;  
  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

Variables : exemple

```
public class Point{  
    private int x, y;  
  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

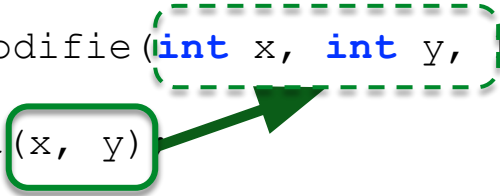
Variables : exemple

```
public class Point{  
    private int x, y;  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y)  
        p=p1;  
    }  
}
```



Variables : exemple

```
public class Point{  
    private int x, y;  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y)  
        p=p1;  
    }  
}
```



méthode static

=> pas d'accès direct aux variables membres

Variables : exemple

```
public class Point{  
    private int x, y;  
  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

```
public class Exemple {  
  
    public static void main(String[] args){  
        Point p = new Point(1,1);  
        Point.modifie(5,5, p);  
        p.affiche();  
    }  
}
```

Variables : exemple

```
public class Point{  
    private int x, y;  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

Affichage ?

```
public class Exemple {  
  
    public static void main(String[] args){  
        Point p = new Point(1,1);  
        Point.modifie(5,5, p);  
        p.affiche();  
    }  
}
```


Variables : exemple

```
public class Point{  
    private int x, y;  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

x : 6 , y : 1

```
public class Exemple {  
  
    public static void main(String[] args){  
        Point p = new Point(1,1);  
        Point.modifie(5,5, p);  
        p.affiche();  
    }  
}
```

Variables : exemple

```
public class Point{  
    private int x, y;  
  
    ...  
    public static void modifie(int x, int y, Point p){  
        p.x=x+1;  
        Point p1=new Point(x, y);  
        p=p1;  
    }  
}
```

x : 6 , y : 1

```
public class Exemple {  
  
    public static void main(String[] args){  
        Point p = new Point(1,1);  
        Point.modifie(5,5, p);  
        p.affiche();  
    }  
}
```

Variables : exemple

```
public class Point{
    private int x, y;
    ...
    public static void modifie(int x, int y, Point p){
        p.x=x+1;
        //Point p1=new Point(x, y);
        //p=p1;
    }
}
```

```
public class Exemple {

    public static void main(String[] args){
        Point p = new Point(1,1);
        Point.modifie(5,5, p);
        p.affiche();
    }
}
```

Composition d'objets

- Les variables d'instance peuvent être des instances d'une classe

```
public class Cercle {  
    private Point centre;  
    private float rayon;  
    public Cercle(int cx, int cy, float r)  
    {  
        centre = new Point(cx, cy);  
        rayon = r;  
    }  
}
```

- Une instance de Cercle possède une instance de Point référencée par l'identificateur centre
- Le constructeur de Cercle prend en charge l'allocation de l'instance centre, en appelant le constructeur de la classe Point
- Lorsque le garbage-collector détruira une instance de Cercle, il détruira tous les sous-objets du cercle, donc le point centre

Classes enveloppes

- Integer, Double, Float, Boolean, Byte,...
- On les utilisera lorsqu'un objet est attendu plutôt qu'un type primitif
- Leurs constructeurs attendent un argument du type primitif correspondant
- Méthodes génériques :
 - xxxValue où xxx est le nom du type primitif correspondant
Ex: intValue invoqué sur un Integer retourne sa valeur entière
 - parseXxx (méthode de classe) convertit un objet String en une donnée de type primitif xxx
Ex:

```
String s = new String("122");  
int i = Integer.parseInt(s);
```

La classe Character

- `Character(char)` : une fois créée, la valeur ne peut pas changer
- `compareTo(Character)` : retourne un entier qui indique si le caractère de l'objet courant est plus petit, égal, ou plus grand que la valeur de l'argument. Une lettre est plus grande qu'une autre si sa valeur numérique est plus grande
- `equals(Object)` : compare la valeur contenue dans l'objet courant avec celle contenue dans un autre
- `toString()` : convertit l'objet en un objet `String`
- `charValue()` : retourne la valeur de type `char` contenue dans l'objet `Character`
- `isUpperCase(char)` : détermine si la valeur primitive est une majuscule

La classe String

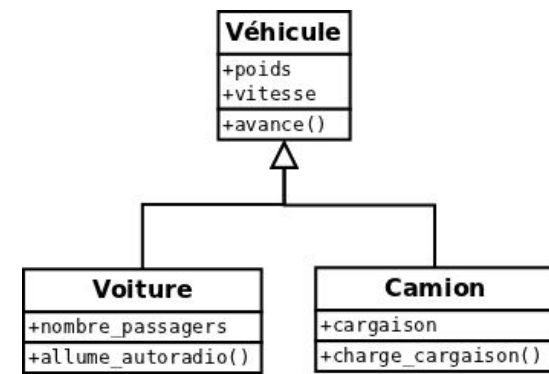
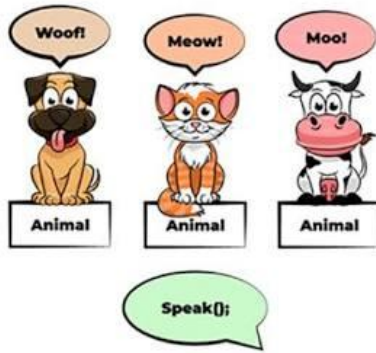
- **Pour des chaînes de caractères dont la valeur ne changera pas**
- **Méthodes**
 - `length` : pour obtenir la longueur d'un objet `String` ou `StringBuffer`
 - `charAt` : pour connaître la valeur du caractère à un index dans un objet `String` ou `StringBuffer`
 - `String substring(int)` : pour trouver une sous-chaîne d'un objet `String` ou `StringBuffer`
 - `String substring(int, int)`
 - `indexOf` : trouve la position dans la chaîne d'un caractère spécifié
 - `lastIndexOf` : cherche en sens inverse à partir de la fin de la chaîne
 - `toUpperCase` : retourne un objet `String` dont les caractères sont tous en majuscule

La classe `StringBuffer`

- Pour des chaînes qui peuvent être modifiées
- On doit utiliser l'opérateur `new` pour créer un objet `StringBuffer`
- **Méthodes :**
 - `append` : pour ajouter un caractère à la fin d'un objet d'un `StringBuffer`
 - `capacity` : retourne la quantité d'espace allouée plutôt que celle utilisée (`length`)
 - `insert` : insère une donnée au milieu
 - `setCharAt` : remplace le caractère à une position spécifiée

Conversions

- D'un objet vers un `String`
 - `toString` : défini dans la classe `Object`
- D'un objet `String` vers des nombres
 - les classes enveloppes (`Integer`, `Double`, `Float` et `Long`) fournissent une méthode de classe nommée `valueOf` qui convertit un objet `String` en objet de la classe enveloppe



Héritage, généricité et polymorphisme

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Intérêt de l'héritage

- Conséquence des principes de base du génie logiciel
- On cherche à favoriser la réutilisation de code existant
 - on veut utiliser des bibliothèques stables et fiables, d'où gain de temps en production de logiciel et code plus sûr
 - on se concentre sur un seul apport tout en bénéficiant d'une série de fonctionnalités standard
 - le code est plus facile à maintenir grâce à l'ajout de nouvelles classes
 - démarche OO : composition d'application à partir de composants existants
- On cherche à privilégier la clarté d'écriture afin d'améliorer la maintenance
 - démarche inspirée de l'observation de notre fonctionnement cognitif
 - organisation et encodage économique de la connaissance
 - l'héritage favorise la simplification de la représentation de la réalité

Intérêt de l'héritage

- Conséquence des principes de base du génie logiciel
- On cherche à favoriser la réutilisation de code existant
 - on veut utiliser des bibliothèques stables et fiables, d'où gain de temps en production de logiciel et code plus sûr
 - on se concentre sur un seul apport tout en bénéficiant d'une série de fonctionnalités standard
 - le code est plus facile à maintenir grâce à l'addition de nouvelles classes
 - démarche OO : composition d'application à partir de composants existants
- On cherche à privilégier la clarté d'écriture afin d'améliorer la maintenance
 - démarche inspirée de l'observation de notre fonctionnement cognitif
 - organisation et encodage économique de la connaissance
 - l'héritage favorise la simplification de la représentation de la réalité

Intérêt de l'héritage

- Conséquence des principes de base du génie logiciel
- On cherche à favoriser la **réutilisation** de code existant
 - on veut utiliser des bibliothèques stables et fiables, d'où **gain** de **temps** en production de logiciel et code plus sûr
 - on se concentre sur un seul apport tout en bénéficiant d'une série de fonctionnalités standard
 - le code est plus facile à maintenir grâce à l'addition de nouvelles classes
 - démarche OO : composition d'application à partir de composants existants
- On cherche à privilégier la **clarté** d'écriture afin **d'améliorer** la **maintenance**
 - démarche inspirée de l'observation de notre fonctionnement cognitif
 - organisation et encodage économique de la connaissance
 - l'héritage favorise la simplification de la représentation de la réalité

Quand utilise-t-on l'héritage ?

- Il n'apparaît généralement pas en début de modélisation
- Quand on identifie des classes qui ont des fonctionnements comparables
 - on procède à une factorisation d'information (attribut ou méthode)
- Quand on dispose de classes qui offrent des services semblables
- La relation d'héritage dépend généralement du contexte du problème, mais il faut élargir le point de vue de façon à envisager des réutilisations futures
 - l'effort en modélisation ne porte ses fruits que si le programme s'enrichit de nouvelles ressources

Quand utilise-t-on l'héritage ?

- Il n'apparaît généralement pas en début de modélisation
- Quand on identifie des classes qui ont des fonctionnements comparables
 - on procède à une factorisation d'information (attribut ou méthode)
- Quand on dispose de classes qui offrent des services semblables
- La relation d'héritage dépend généralement du contexte du problème, mais il faut élargir le point de vue de façon à envisager des réutilisations futures
 - l'effort en modélisation ne porte ses fruits que si le programme s'enrichit de nouvelles ressources

Quand utilise-t-on l'héritage ?

- Il n'apparaît généralement pas en début de modélisation
- Quand on identifie des classes qui ont des fonctionnements comparables
 - on procède à une factorisation d'information (attribut ou méthode)
- Quand on dispose de classes qui offrent des services semblables
- La relation d'héritage dépend généralement du contexte du problème, mais il faut élargir le point de vue de façon à envisager des réutilisations futures
 - l'effort en modélisation ne porte ses fruits que si le programme s'enrichit de nouvelles ressources

Quand utilise-t-on l'héritage ?

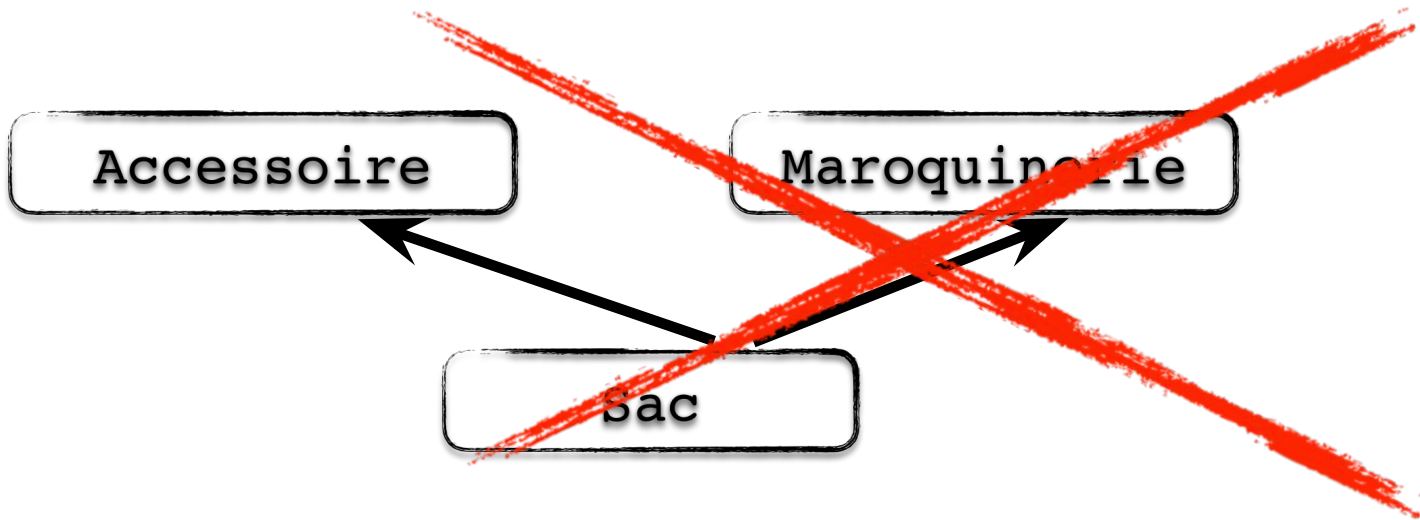
- Il n'apparaît généralement pas en début de modélisation
- Quand on identifie des classes qui ont des fonctionnements comparables
 - on procède à une factorisation d'information (attribut ou méthode)
- Quand on dispose de classes qui offrent des services semblables
- La relation d'héritage dépend généralement du contexte du problème, mais il faut élargir le point de vue de façon à envisager des réutilisations futures
 - l'effort en modélisation ne porte ses fruits que si le programme s'enrichit de nouvelles ressources

Héritage : Règles et Structure

- Java impose trois règles :

Héritage : Règles et Structure

- Java impose trois règles :
 - une classe hérite toujours d'**une seule classe** ;



Héritage : Règles et Structure

- Java impose trois règles :
 - une classe hérite toujours d'**une seule classe** ;
 - par défaut une classe hérite de la classe **Object** ;

Héritage : Règles et Structure

- Java impose trois règles :
 - une classe hérite toujours d'**une seule classe** ;
 - par défaut une classe hérite de la classe **Object** ;
 - les cycles sont interdits.



Héritage : Règles et Structure

- Java impose trois règles :
 - une classe hérite toujours d'**une seule classe** ;
 - par défaut une classe hérite de la classe **Object** ;
 - les cycles sont interdits.

→ Les classes forment donc un arbre d'héritage unique

Héritage : Règles et Structure

- Java impose trois règles :
 - une classe hérite toujours d'**une seule classe** ;
 - par défaut une classe hérite de la classe **Object** ;
 - les cycles sont interdits.

→ Les classes forment donc un arbre d'héritage unique

Définition

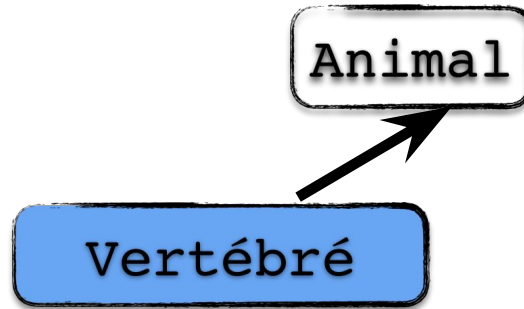
Un **arbre** est un graphe non orienté connexe et acyclique.

Héritage : Règles et Structure

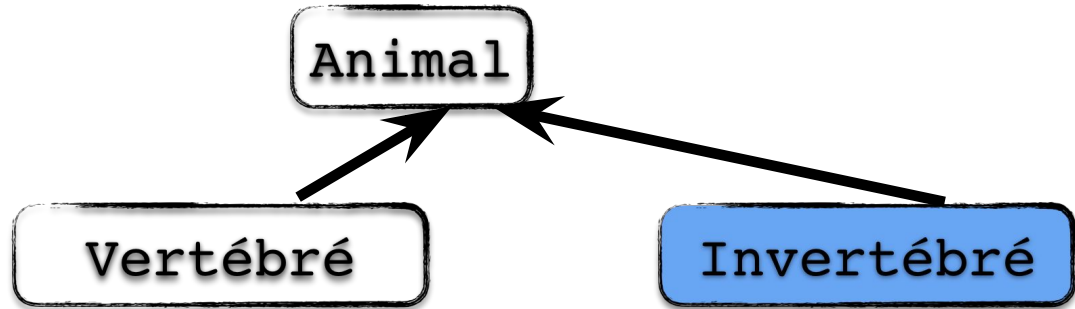


Animal

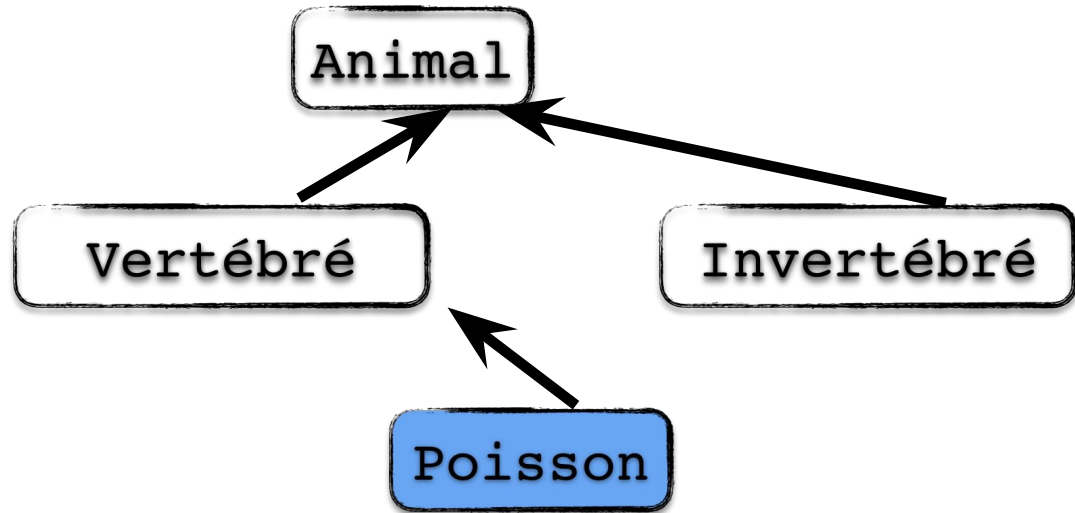
Héritage : Règles et Structure



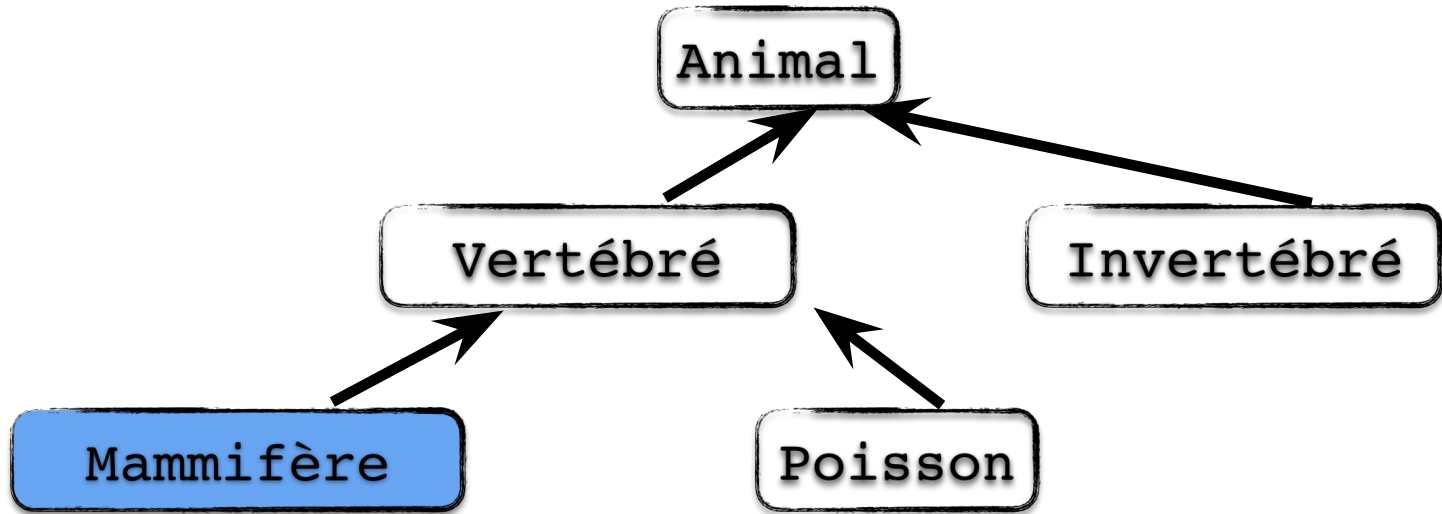
Héritage : Règles et Structure



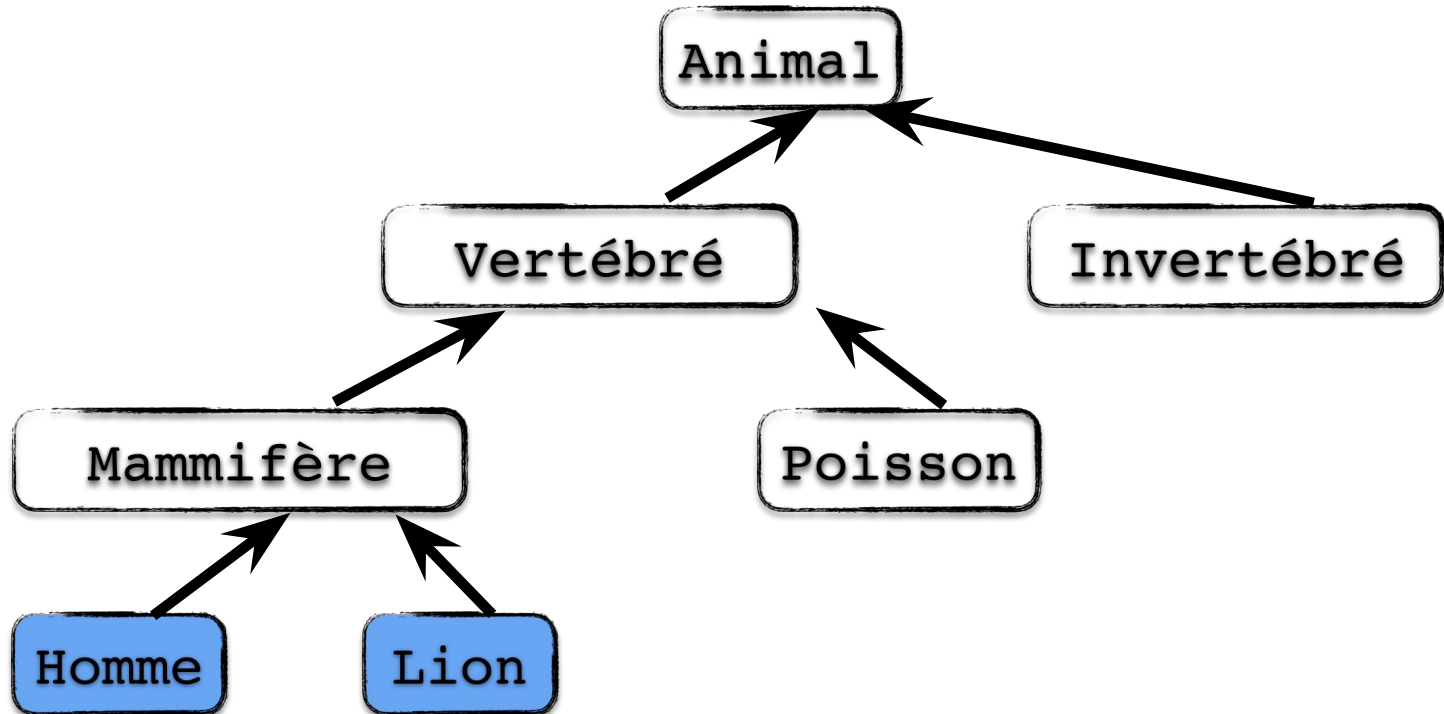
Héritage : Règles et Structure



Héritage : Règles et Structure

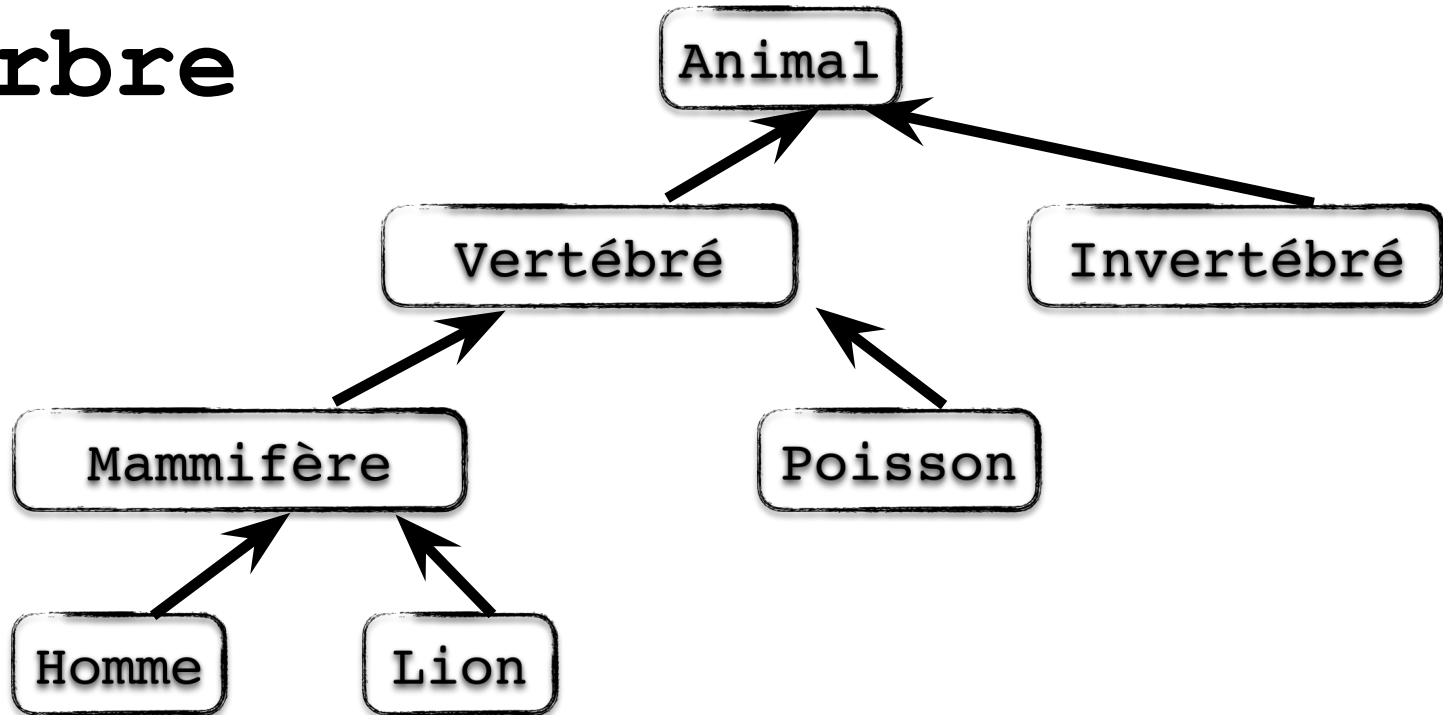


Héritage : Règles et Structure

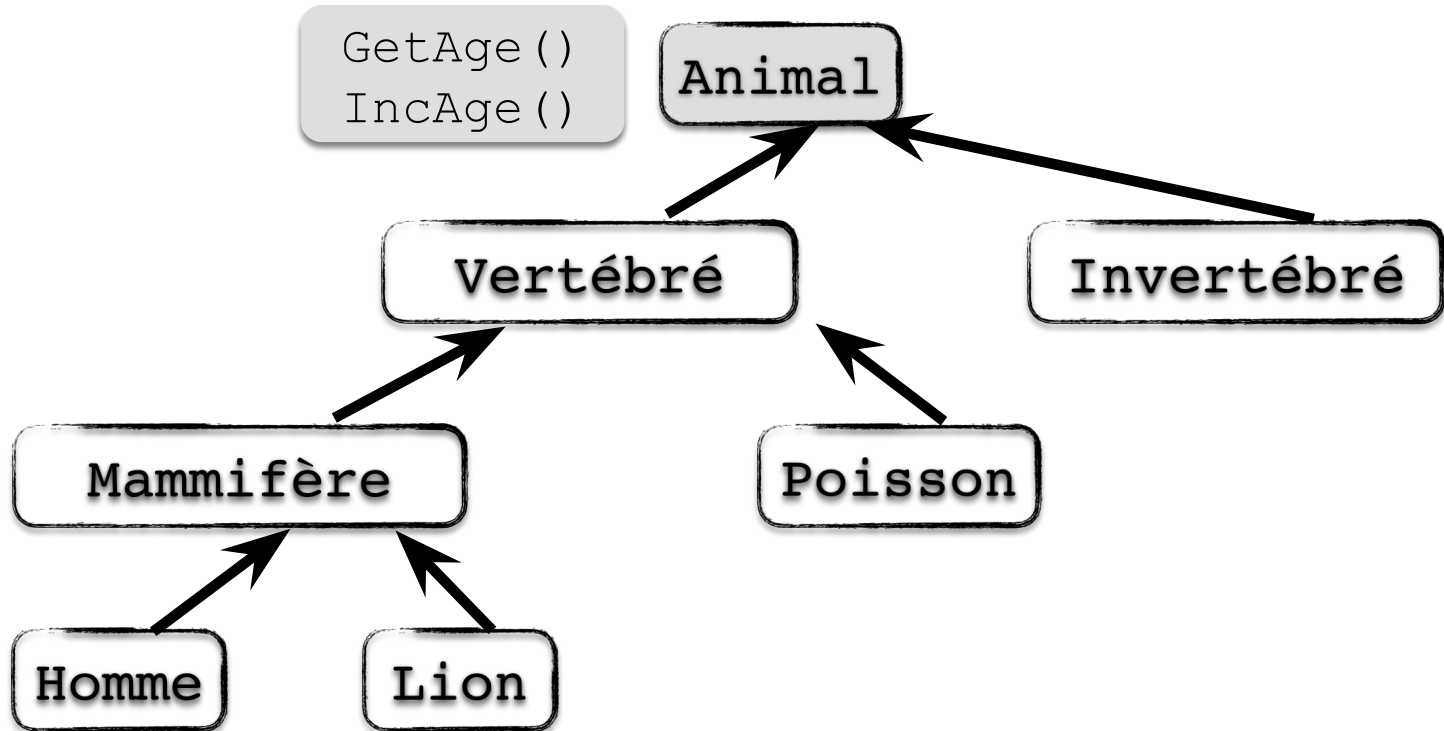


Héritage : Règles et Structure

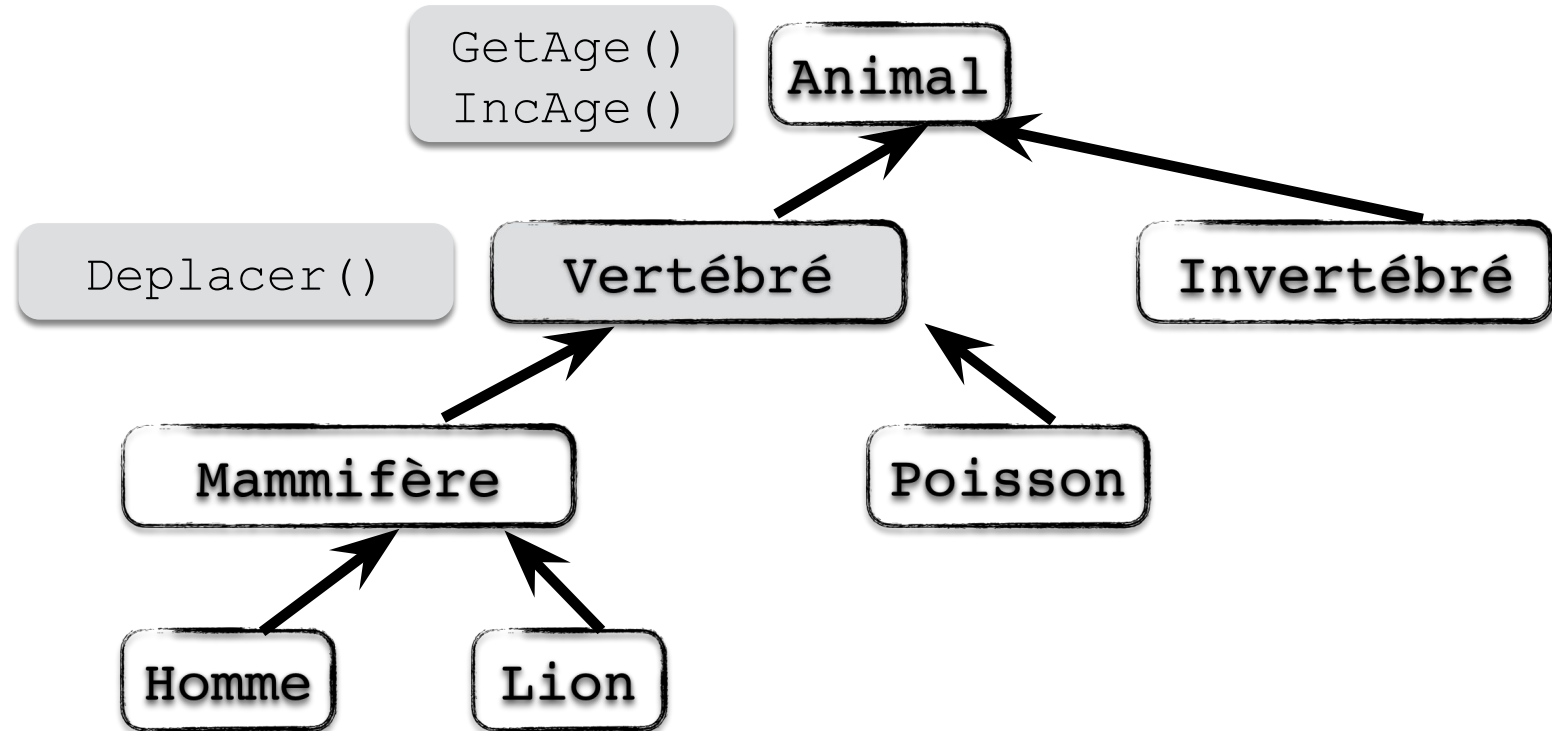
Arbre



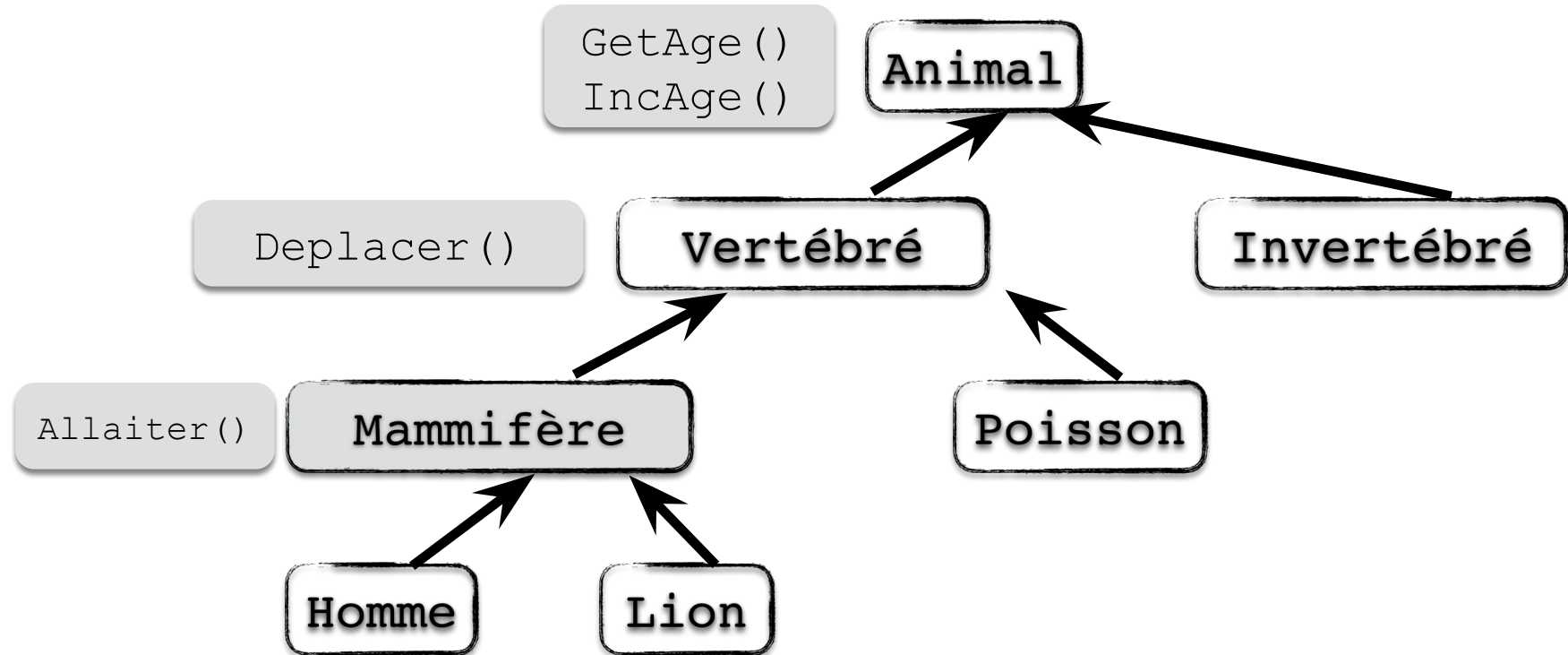
Héritage : Règles et Structure



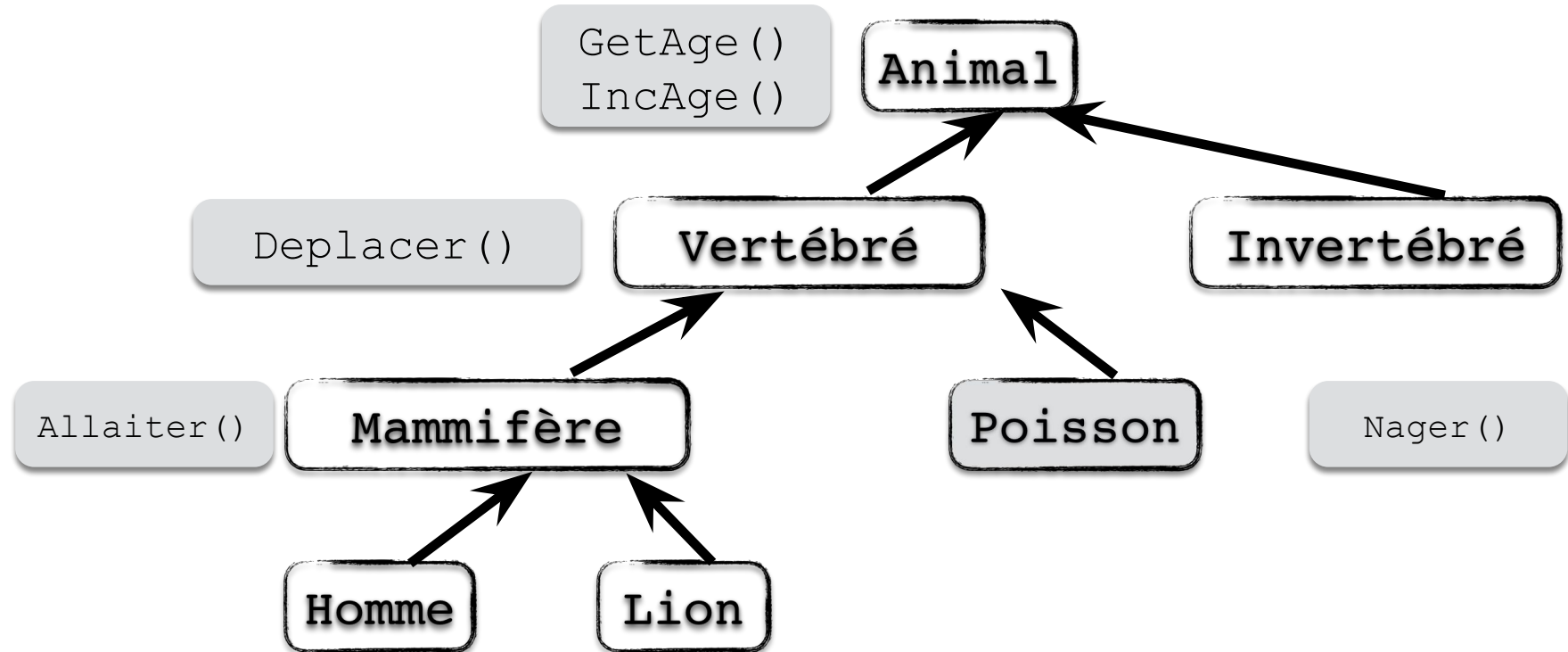
Héritage : Règles et Structure



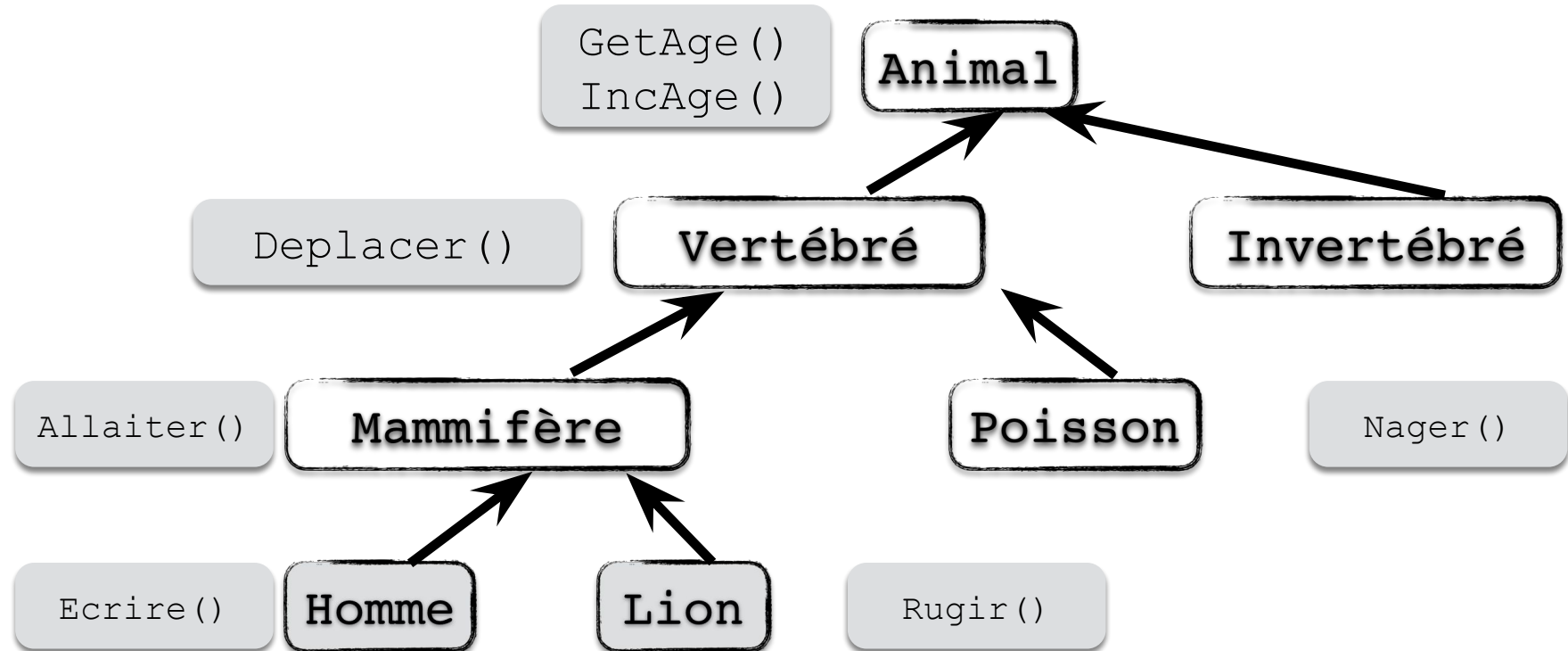
Héritage : Règles et Structure



Héritage : Règles et Structure



Héritage : Règles et Structure



Héritage : Règles et Structure

Une autre super classe ?

GetAge()
IncAge()

Animal

Deplacer()

Vertébré

Invertébré

Allaiter()

Mammifère

Poisson

Nager()

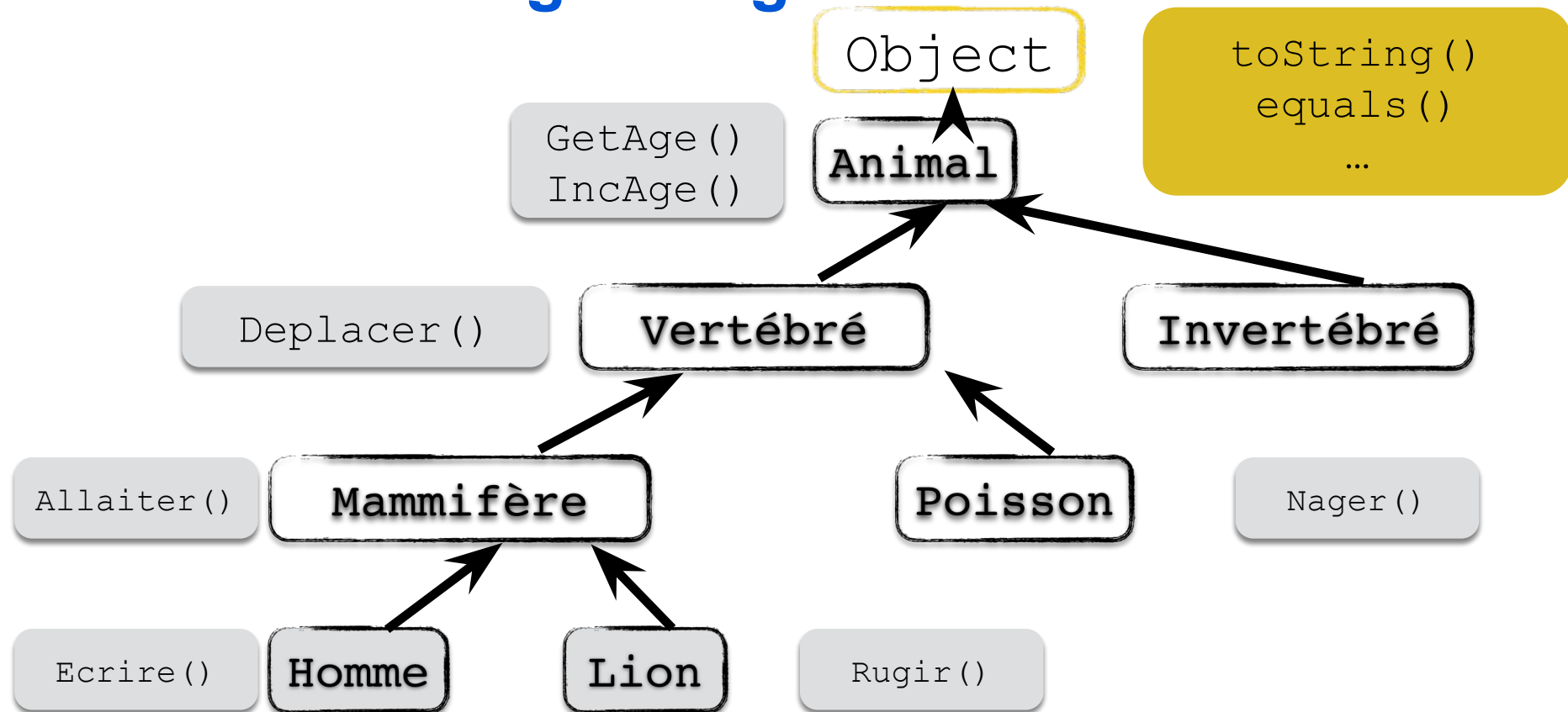
Ecrire()

Homme

Lion

Rugir()

Héritage : Règles et Structure



Héritage des membres

```
public class Animal {  
    final public String nom;  
    private int age;  
    public int getAge() { return age; }  
}
```

Héritage des membres

```
public class Animal {  
    final public String nom;  
    private int age;  
    public int getAge() { return age; }  
}  
  
public class Vertebre extends Animal {  
    final public int nbVertebres;  
    public void deplacerVertebres(int nb) {...}  
}
```

Héritage des membres

```
public class Animal {  
    final public String nom;  
    private int age;  
    public int getAge() { return age; }  
}  
  
public class Vertebre extends Animal {  
    final public int nbVertebres;  
    public void deplacerVertebres(int nb) {...}  
}
```

Les membres de la classe Vertebre sont :

attributs : nom (hérité), age (hérité) et nbVertebres

méthodes : getAge() (hérité) et deplacerVertebres().

Instance d'une classe avec héritage

```
public class Homme extends Mammifère {  
    // attributs propres aux hommes uniquement  
    private boolean droitier;  
    ...  
  
    public void écrire() {  
        if droitier {...}  
        else {...}  
    }  
}
```

Instance d'une classe avec héritage

```
public class Homme extends Mammifère {  
    // attributs propres aux hommes uniquement  
    private boolean droitier;  
    ...  
  
    public void écrire() {  
        if droitier {...}  
        else {...}  
    }  
}
```

```
Homme unHomme = new Homme(...);  
// appel d'un service propre aux hommes unHomme.écrire();  
// appel d'un service commun aux vertébrés (hérité)  
unHomme.deplacerVertebres(3);  
// appel d'un service commun aux animaux (hérité)  
unHomme.setNom("Adam");
```

Une double relation

- Généralisation : consiste à factoriser les propriétés communes d'un ensemble de classes dans une classe plus générale et hériter de celle ci
- Spécialisation : consiste à ajouter une nouvelle classe en ajoutant de nouvelles propriétés et de nouveaux comportements à une classe existante
 - il ne s'agit pas uniquement de l'ajout d'attributs : on se base principalement sur les opérations des nouveaux objets
 - on peut spécialiser un comportement défini plutôt qu'en ajouter
- Comment identifier que D hérite de B ? Il faut pouvoir dire : « D est une sorte de B », c'est-à-dire que tout ce qui est vrai pour B l'est aussi pour D

Une double relation

- Généralisation : consiste à factoriser les propriétés communes d'un ensemble de classes dans une classe plus générale et hériter de celle ci
- Spécialisation : consiste à ajouter une nouvelle classe en ajoutant de nouvelles propriétés et de nouveaux comportements à une classe existante
 - il ne s'agit pas uniquement de l'ajout d'attributs : on se base principalement sur les opérations des nouveaux objets
 - on peut spécialiser un comportement défini plutôt qu'en ajouter
- Comment identifier que D hérite de B ? Il faut pouvoir dire : « D est une sorte de B », c'est-à-dire que tout ce qui est vrai pour B l'est aussi pour D

Une double relation

- Généralisation : consiste à factoriser les propriétés communes d'un ensemble de classes dans une classe plus générale et hériter de celle ci
- Spécialisation : consiste à ajouter une nouvelle classe en ajoutant de nouvelles propriétés et de nouveaux comportements à une classe existante
 - il ne s'agit pas uniquement de l'ajout d'attributs : on se base principalement sur les opérations des nouveaux objets
 - on peut spécialiser un comportement défini plutôt qu'en ajouter
- Comment identifier que D hérite de B ? Il faut pouvoir dire : « D est une sorte de B », c'est-à-dire que tout ce qui est vrai pour B l'est aussi pour D

Vocabulaire

- Principe : un objet spécialisé bénéficie des caractéristiques de l'objet le plus général (il en hérite) auquel il rajoute ses éléments propres
- La classe associée au concept le plus général est appelée **classe de base**, **classe mère** ou **super-classe**
- Pour chaque concept spécialisé, on dérive une classe du concept de base : la nouvelle classe est dite classe dérivée, classe fille ou sous-classe
- On dit que la classe dérivée, qui est une version spécialisée de sa classe de base, l'étend
- Une classe peut avoir un nombre illimité de sous classes
- L'héritage peut se faire sur plusieurs niveaux
- En cas d'héritage multiple, la classe a plusieurs super-classes

Conséquences de l'héritage

- On peut hiérarchiser les concepts en différents niveaux d'abstraction afin d'utiliser celui qui convient, de façon flexible, selon le contexte
 - on peut parler à un objet à différents niveaux d'abstraction, sans connaître sa nature exacte, ce qui facilite l'écriture du code
 - on privilégie un des niveaux, souvent le niveau de base, car c'est celui qui capture le plus d'informations sur ce qu'on attribue à l'objet
- Principe de substitution : principe clé de la pratique orientée objet
 - partout où une instance d'une superclasse apparaît, on peut substituer une instance d'une sous-classe
 - tout message compris par un objet d'une superclasse le sera obligatoirement par tous les objets issus des sous-classes
- Interprétation ensembliste
 - la superclasse, comme « ensemble », regroupe tous les éléments, y compris tous ceux que regroupe l'ensemble sous-classe
 - tout objet d'une sous-classe est également objet de sa superclasse, et on peut passer d'un type sous-classe à un type superclasse sans difficulté
 - l'inverse n'est évidemment plus vrai : les sous-classes étant plus spécifiques, elles se permettent des choses que ne peuvent pas les superclasses

Conséquences de l'héritage

- On peut hiérarchiser les concepts en différents niveaux d'abstraction afin d'utiliser celui qui convient, de façon flexible, selon le contexte
 - on peut parler à un objet à différents niveaux d'abstraction, sans connaître sa nature exacte, ce qui facilite l'écriture du code
 - on privilégie un des niveaux, souvent le niveau de base, car c'est celui qui capture le plus d'informations sur ce qu'on attribue à l'objet
- Principe de substitution : principe clé de la pratique orientée objet
 - partout où une instance d'une superclasse apparaît, on peut substituer une instance d'une sous-classe
 - tout message compris par un objet d'une superclasse le sera obligatoirement par tous les objets issus des sous-classes
- Interprétation ensembliste
 - la superclasse, comme « ensemble », regroupe tous les éléments, y compris tous ceux que regroupe l'ensemble sous-classe
 - tout objet d'une sous-classe est également objet de sa superclasse, et on peut passer d'un type sous-classe à un type superclasse sans difficulté
 - l'inverse n'est évidemment plus vrai : les sous-classes étant plus spécifiques, elles se permettent des choses que ne peuvent pas les superclasses

Conséquences de l'héritage

- On peut hiérarchiser les concepts en différents niveaux d'abstraction afin d'utiliser celui qui convient, de façon flexible, selon le contexte
 - on peut parler à un objet à différents niveaux d'abstraction, sans connaître sa nature exacte, ce qui facilite l'écriture du code
 - on privilégie un des niveaux, souvent le niveau de base, car c'est celui qui capture le plus d'informations sur ce qu'on attribue à l'objet
- Principe de substitution : principe clé de la pratique orientée objet
 - partout où une instance d'une superclasse apparaît, on peut substituer une instance d'une sous-classe
 - tout message compris par un objet d'une superclasse le sera obligatoirement par tous les objets issus des sous-classes
- Interprétation ensembliste
 - la superclasse, comme « ensemble », regroupe tous les éléments, y compris tous ceux que regroupe l'ensemble sous-classe
 - tout objet d'une sous-classe est également objet de sa superclasse, et on peut passer d'un type sous-classe à un type superclasse sans difficulté
 - l'inverse n'est évidemment plus vrai : les sous-classes étant plus spécifiques, elles se permettent des choses que ne peuvent pas les superclasses

Conversion explicite ou implicite

- La conversion (casting) permet à une variable typée d'une certaine façon lors de sa création d'adopter un autre type le temps d'une opération
 - en programmation classique, le compilateur l'accepte si on a recours à une conversion explicite
- Principe de substitution : «casting implicite»
 - on fait passer un objet d'une sous-classe pour un objet d'une superclasse
- En revanche, faire passer un objet d'une superclasse pour celui d'une sous-classe demande un « casting explicite »

Héritage ou composition

- L'allocation mémoire d'un objet dérivé se compose, d'abord, de l'allocation d'un objet de type superclasse, puis de la mémoire pour les attributs qui lui sont propres
 - cette allocation ressemble à celle d'un objet qui serait en partie composé d'un autre objet
 - si on ne s'intéresse qu'à l'allocation de mémoire, pas de grande différence entre une composition et un héritage
- La communication avec un objet contenu dans un autre passe par un envoi de messages de l'objet englobant à celui contenu
 - dans le cas de l'héritage, il n'y a pas d'envoi de message entre un objet de base et un dérivé, car la méthode est propre à la classe elle-même, même si héritée d'une autre

Héritage ou composition

- L'allocation mémoire d'un objet dérivé se compose, d'abord, de l'allocation d'un objet de type superclasse, puis de la mémoire pour les attributs qui lui sont propres
 - cette allocation ressemble à celle d'un objet qui serait en partie composé d'un autre objet
 - si on ne s'intéresse qu'à l'allocation de mémoire, pas de grande différence entre une composition et un héritage
- La communication avec un objet contenu dans un autre passe par un envoi de messages de l'objet englobant à celui contenu
 - dans le cas de l'héritage, il n'y a pas d'envoi de message entre un objet de base et un dérivé, car la méthode est propre à la classe elle-même, même si héritée d'une autre

Recherche de méthode

- Quand on invoque une méthode, celle-ci est d'abord recherchée dans la classe de l'objet
- Si la méthode ne s'y trouve pas, en raison de l'héritage, elle peut se trouver plus haut, dans une superclasse
- La recherche de la méthode se fait donc en remontant, de superclasse en superclasse, jusqu'à retrouver la première méthode qui convienne

Encapsulation protégée

- Principe d'encapsulation : maximiser la partie privée des classes au détriment de leur partie publique
- L'encapsulation protégée permet d'adapter l'encapsulation à l'héritage
 - un attribut (ou une méthode) déclaré protégé dans une classe reste privé mais devient accessible dans toutes les sous-classes
 - une méthode protégée devrait être redéfinie, en faisant appel aux méthodes originelles de la superclasse
- La charte de la bonne programmation OO déconseille l'utilisation de « protected »
 - même par rapport à toutes ses sous-classes, la superclasse se doit de préserver son intégrité

Encapsulation protégée

- Principe d'encapsulation : maximiser la partie privée des classes au détriment de leur partie publique
- L'encapsulation protégée permet d'adapter l'encapsulation à l'héritage
 - un attribut (ou une méthode) déclaré protégé dans une classe reste privé mais devient accessible dans toutes les sous-classes
 - une méthode protégée devrait être redéfinie, en faisant appel aux méthodes originelles de la superclasse
- La charte de la bonne programmation OO déconseille l'utilisation de « protected »
 - même par rapport à toutes ses sous-classes, la superclasse se doit de préserver son intégrité

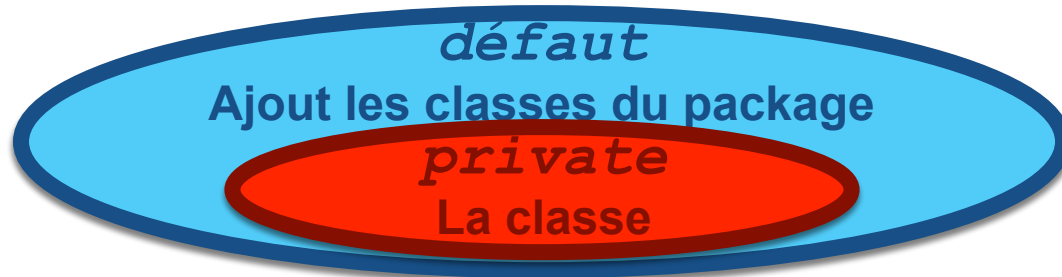
Encapsulation protégée

- Principe d'encapsulation : maximiser la partie privée des classes au détriment de leur partie publique
- L'encapsulation protégée permet d'adapter l'encapsulation à l'héritage
 - un attribut (ou une méthode) déclaré protégé dans une classe reste privé mais devient accessible dans toutes les sous-classes
 - une méthode protégée devrait être redéfinie, en faisant appel aux méthodes originelles de la superclasse
- La charte de la bonne programmation OO déconseille l'utilisation de « protected »
 - même par rapport à toutes ses sous-classes, la superclasse se doit de préserver son intégrité

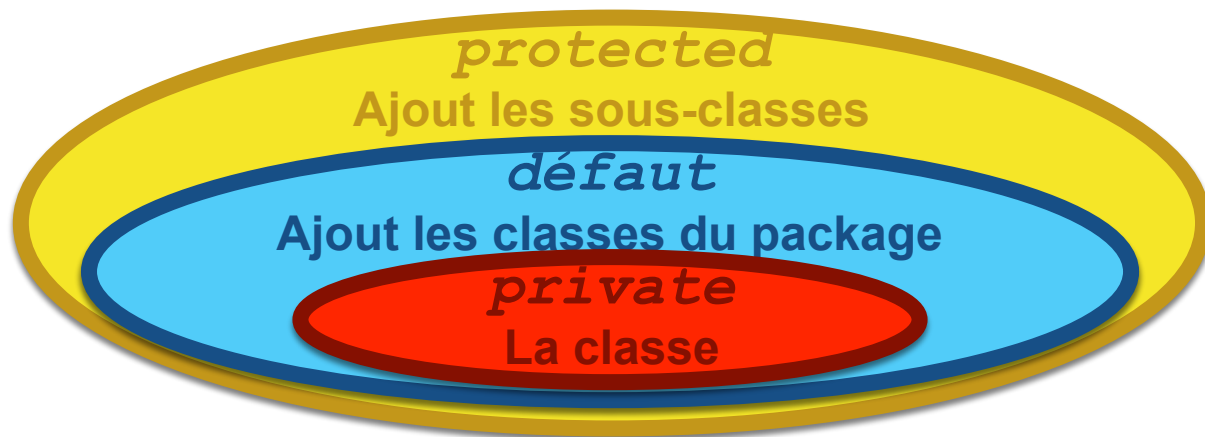
Encapsulation et visibilité



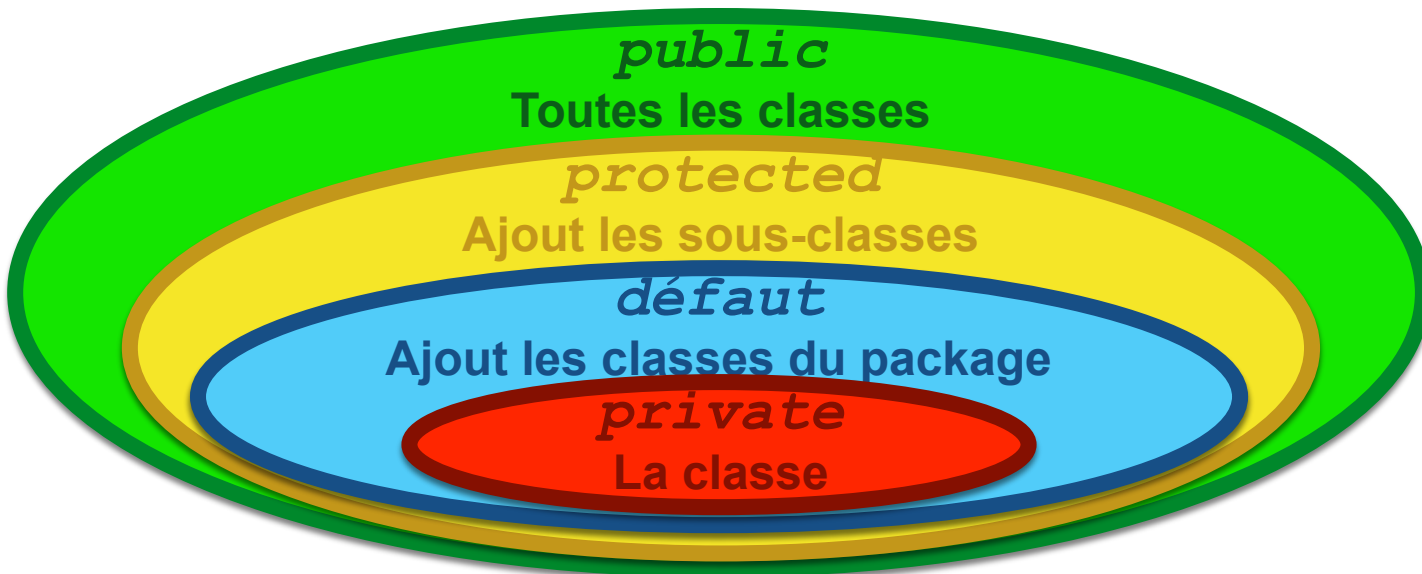
Encapsulation et visibilité



Encapsulation et visibilité

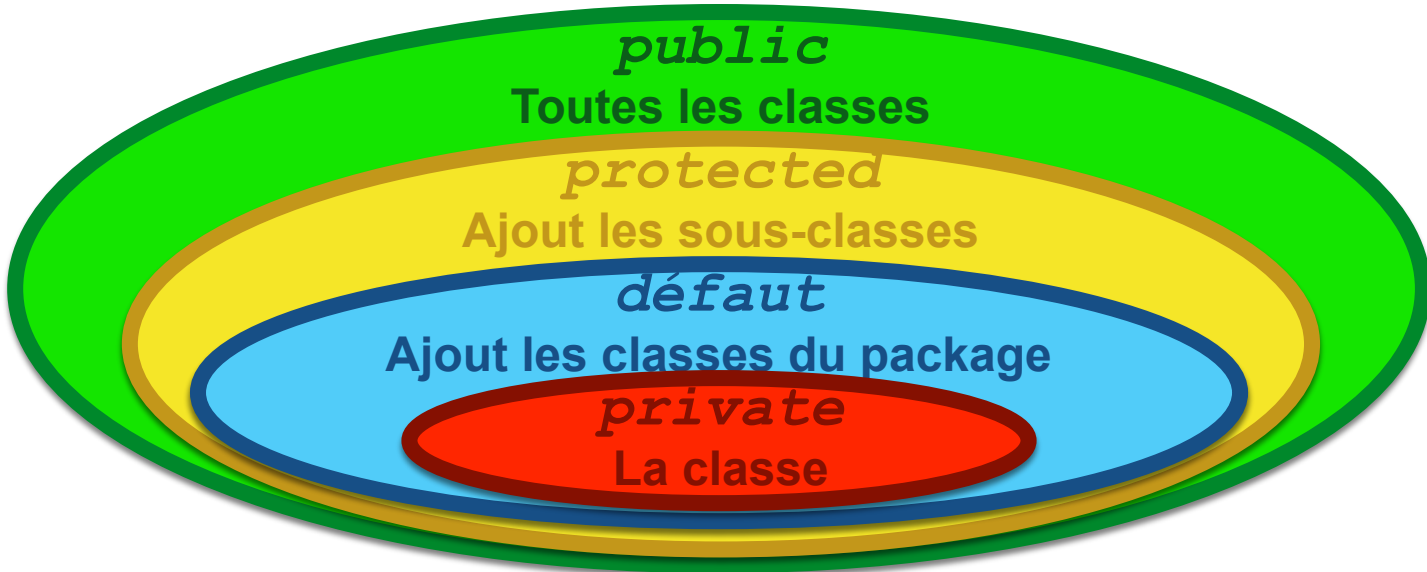


Encapsulation et visibilité



Encapsulation et visibilité

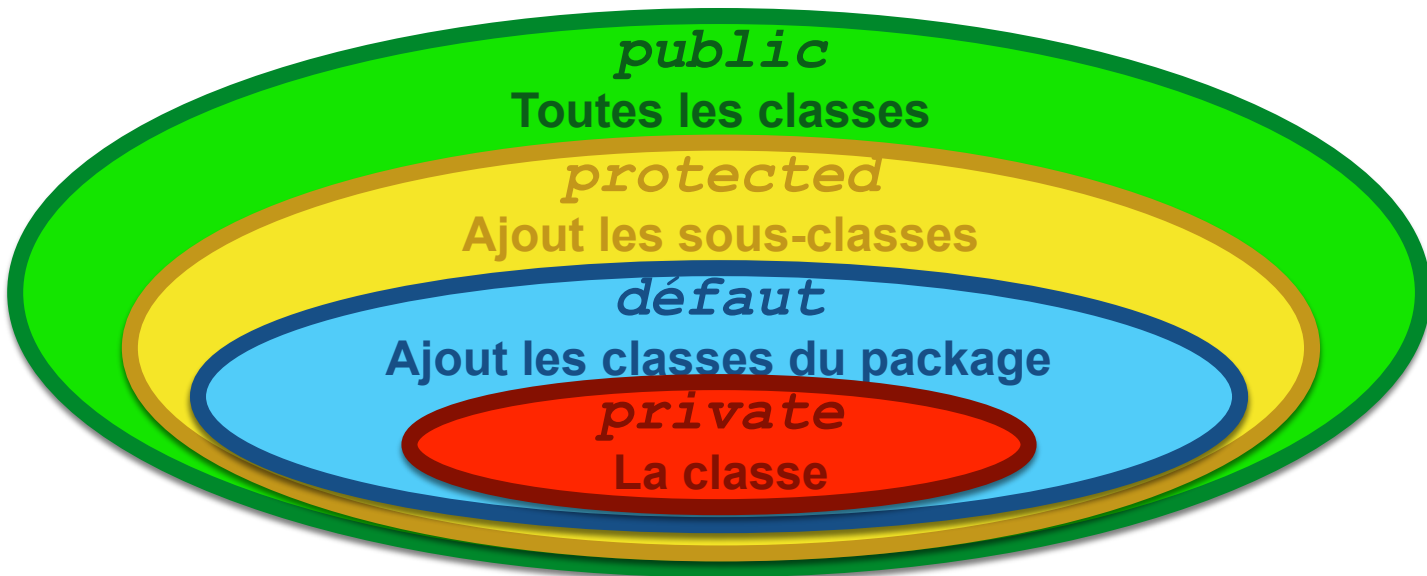
On peut étendre la visibilité aux sous-classes avec le mot clé : `protected`.



Encapsulation et visibilité

On peut étendre la visibilité aux sous-classes avec le mot clé : `protected`.

Attention : on n'étend pas la visibilité `private` mais la visibilité par défaut.

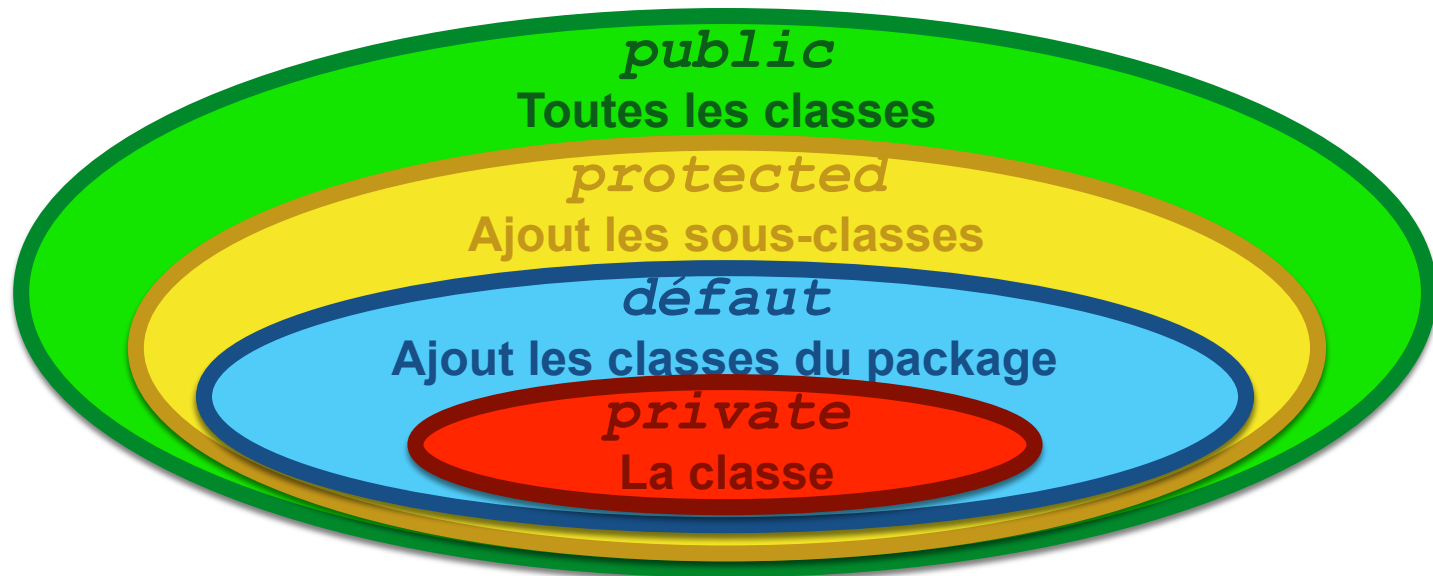


Encapsulation et visibilité

On peut étendre la visibilité aux sous-classes avec le mot clé : **protected**.

Attention : on n'étend pas la visibilité **private** mais la visibilité par **défaut**.

⇒ l'héritage s'ajoute aux packages



Encapsulation et visibilité

On peut étendre la visibilité aux sous-classes avec le mot clé : **protected**.

Attention : on n'étend pas la visibilité **private** mais la visibilité par défaut.

⇒ l'héritage s'ajoute aux packages
⇒ **protected** ne protège pas mais augmente la visibilité !



Encapsulation et visibilité

On peut étendre la visibilité aux sous-classes avec le mot clé : **protected**.

Attention : on n'étend pas la visibilité **private** mais la visibilité par défaut.

⇒ l'héritage s'ajoute aux packages
⇒ **protected** ne protège pas mais augmente la visibilité !

	private	défaut	protected	public
<i>La classe elle même</i>	X	X	X	X
<i>une sous-classe, même paquetage</i>		X	X	X
<i>pas sous-classe, même paquetage</i>		X	X	X
<i>une sous-classe, paquetage différent</i>			X	X
<i>pas sous-classe, paquetage différent</i>				X

La surcharge et la surdéfinition

- Un même identificateur peut désigner plusieurs méthodes
 - avec des signatures différentes
 - avec la même signature en cas d'héritage
 - une méthode d'une sous-classe peut porter le même nom et avoir la même signature que celle de la super classe
 - la méthode de la super classe est encore accessible mais préfixée
- La surcharge (polymorphisme statique) consiste à proposer différentes signatures de la même méthode
- La redéfinition (polymorphisme dynamique) se produit dans l'héritage par redéfinition de la méthode mère avec une méthode fille
 - c'est ce qu'en anglais on désigne comme la pratique `override`
 - la surcharge est aussi possible, mais elle ne masque pas une méthode de la super classe

La surcharge et la surdéfinition

- Un même identificateur peut désigner plusieurs méthodes
 - avec des signatures différentes
 - avec la même signature en cas d'héritage
 - une méthode d'une sous-classe peut porter le même nom et avoir la même signature que celle de la super classe
 - la méthode de la super classe est encore accessible mais préfixée
- La surcharge (polymorphisme statique) consiste à proposer différentes signatures de la même méthode
- La redéfinition (polymorphisme dynamique) se produit dans l'héritage par redéfinition de la méthode mère avec une méthode fille
 - c'est ce qu'en anglais on désigne comme la pratique `override`
 - la surcharge est aussi possible, mais elle ne masque pas une méthode de la super classe

La surcharge et la surdéfinition

- Un même identificateur peut désigner plusieurs méthodes
 - avec des signatures différentes
 - avec la même signature en cas d'héritage
 - une méthode d'une sous-classe peut porter le même nom et avoir la même signature que celle de la super classe
 - la méthode de la super classe est encore accessible mais préfixée
- La surcharge (polymorphisme statique) consiste à proposer différentes signatures de la même méthode
- La redéfinition (polymorphisme dynamique) se produit dans l'héritage par redéfinition de la méthode mère avec une méthode fille
 - c'est ce qu'en anglais on désigne comme la pratique `override`
 - la surcharge est aussi possible, mais elle ne masque pas une méthode de la super classe

Redéfinition et encapsulation

- Une méthode redéfinie dans une sous-classe possédera la même signature que celle définie dans la super classe avec, comme unique différence autorisée, un mode d'accès moins restrictif
 - on ne peut pas rendre l'accès à la méthode dans la sous-classe plus strict qu'il ne l'est dans la superclasse (respect du principe de substitution)
- Par conséquent, une méthode définie protected ou public dans la super classe sera redéfinie protected ou public dans la sous-classe
- Certains langages OO considèrent qu'une méthode déclarée private dans la super classe, donc non accessible par la sous-classe, ne peut pas être redéfinie

Héritage et constructeur

- Si la sous-classe ajoute des attributs par rapport à la super classe, toute instance de la sous-classe possède deux ensembles d'attributs, ceux qui lui sont propres et ceux hérités de la base
- Chaque classe s'occupe de l'initialisation de ses propres attributs
 - le constructeur de la super classe s'occupe d'initialiser ses attributs et celui de la sous-classe les siens
 - le constructeur de la sous-classe fait appel (et il doit le faire en premier) au constructeur de la super classe
- Plusieurs cas de figure
 - la super classe ne définit pas de constructeur : Java en propose un par défaut, qui initialise tous les attributs à des valeurs par défaut : 0 ou **null**
 - la super classe définit un constructeur sans argument : celui-ci se substitue à celui par défaut
 - la super classe ne définit aucun constructeur sans argument : la sous-classe doit appeler le constructeur de la super classe avec les valeurs initiales des attributs qui le concerne

Héritage et constructeur

- Si la sous-classe ajoute des attributs par rapport à la super classe, toute instance de la sous-classe possède deux ensembles d'attributs, ceux qui lui sont propres et ceux hérités de la base
- Chaque classe s'occupe de l'initialisation de ses propres attributs
 - le constructeur de la super classe s'occupe d'initialiser ses attributs et celui de la sous-classe les siens
 - le constructeur de la sous-classe fait appel (et il doit le faire en premier) au constructeur de la super classe
- Plusieurs cas de figure
 - la super classe ne définit pas de constructeur : Java en propose un par défaut, qui initialise tous les attributs à des valeurs par défaut : 0 ou `null`
 - la super classe définit un constructeur sans argument : celui-ci se substitue à celui par défaut
 - la super classe ne définit aucun constructeur sans argument : la sous-classe doit appeler le constructeur de la super classe avec les valeurs initiales des attributs qui le concerne

Héritage et constructeur

- Si la sous-classe ajoute des attributs par rapport à la super classe, toute instance de la sous-classe possède deux ensembles d'attributs, ceux qui lui sont propres et ceux hérités de la base
- Chaque classe s'occupe de l'initialisation de ses propres attributs
 - le constructeur de la super classe s'occupe d'initialiser ses attributs et celui de la sous-classe les siens
 - le constructeur de la sous-classe fait appel (et il doit le faire en premier) au constructeur de la super classe
- Plusieurs cas de figure
 - la super classe ne définit pas de constructeur : Java en propose un par défaut, qui initialise tous les attributs à des valeurs par défaut : 0 ou **null**
 - la super classe définit un constructeur sans argument : celui-ci se substitue à celui par défaut
 - la super classe ne définit aucun constructeur sans argument : la sous-classe doit appeler le constructeur de la super classe avec les valeurs initiales des attributs qui le concerne

Le multihéritage

- L'héritage direct de plusieurs classes induit 2 types de problèmes
 - parce que toute classe dérivée réunit des caractéristiques différentes provenant de plusieurs superclasses, elle les additionne
 - Les deux superclasses nomment de la même manière un attribut et une méthode ambiguïté lors de l'appel de la méthode : solution : spécifier l'attribut et la méthode en attachant au nom de la méthode celui de la classe
- En cas d'héritage en losange :
 - l'héritage ressemble à une certaine forme de composition, puisque l'objet de la sous-classe possède un objet de la super classe
 - tout objet se composera deux fois d'un objet de la classe
 - C++ élimine le problème avec l'héritage virtuel
- Java interdit le multihéritage de classes mais permet le multihéritage d'interfaces : une sous-classe peut hériter d'une classe et d'autant d'interfaces que l'on veut

Interfaces

- «Classe» ne spécifiant que des opérations sans jamais en donner l'implémentation
- Représente un «contrat de service» que doivent implémenter les sous-classes
- Plusieurs interfaces, héritées par une même sous-classe, peuvent posséder des signatures de méthodes communes
 - l'interface se limite à la liste des signatures de méthode et donc élimine les problèmes d'ambiguïté de l'héritage multiple de classes
 - la classe n'implémente qu'une fois les méthodes communes
- Une interface ne peut pas avoir d'attributs propres
 - Java autorise uniquement des constantes de classe (attributs publics, finaux et statiques)

Interfaces

- «Classe» ne spécifiant que des opérations sans jamais en donner l'implémentation
- Représente un «**contrat** de service» que doivent **implémenter** les sous-classes
- Plusieurs interfaces, héritées par une même sous-classe, peuvent posséder des signatures de méthodes communes
 - l'interface se limite à la liste des signatures de méthode et donc élimine les problèmes d'ambiguïté de l'héritage multiple de classes
 - la classe n'implémente qu'une fois les méthodes communes
- Une interface ne peut pas avoir d'attributs propres
 - Java autorise uniquement des constantes de classe (attributs publics, finaux et statiques)

Interfaces

- «Classe» ne spécifiant que des opérations sans jamais en donner l'implémentation
- Représente un «**contrat** de service» que doivent **implémenter** les sous-classes
- Plusieurs interfaces, héritées par une même sous-classe, peuvent posséder des signatures de méthodes communes
 - l'interface se limite à la liste des signatures de méthode et donc élimine les problèmes d'ambiguïté de l'héritage multiple de classes
 - la classe n'implémente qu'une fois les méthodes communes
- Une interface ne peut pas avoir d'attributs propres
 - Java autorise uniquement des constantes de classe (attributs publics, finaux et statiques)

Interfaces

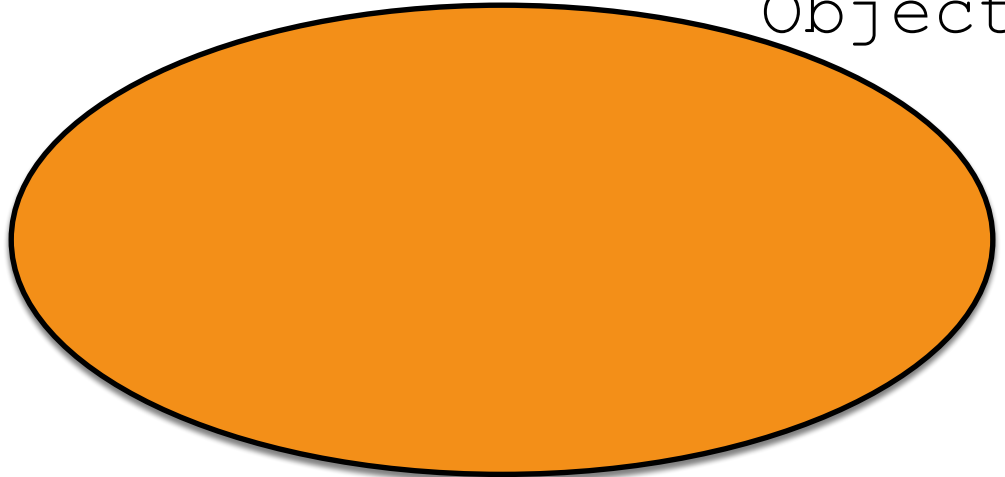
- «Classe» ne spécifiant que des opérations sans jamais en donner l'implémentation
- Représente un «**contrat** de service» que doivent **implémenter** les sous-classes
- Plusieurs interfaces, héritées par une même sous-classe, peuvent posséder des signatures de méthodes communes
 - l'interface se limite à la liste des signatures de méthode et donc élimine les problèmes d'ambiguïté de l'héritage multiple de classes
 - la classe n'implémente qu'une fois les méthodes communes
- Une interface ne peut pas avoir d'attributs propres
 - Java autorise uniquement des constantes de classe (attributs publics, finaux et statiques)

Le polymorphisme

- Capacité des objets d'une même hiérarchie à répondre différemment à la même demande (opération)
 - les classes filles peuvent implémenter différemment les opérations de la classe mère, et ainsi spécialiser un comportement
- On va pouvoir gérer des listes d'objets qui répondront au même message chacun avec son comportement spécifique
 - la classe qui demande l'exécution du service ne modifie pas son comportement quand on ajoute une sous-classe
 - d'où simplicité de conception, économie d'écriture et évolution sans problème

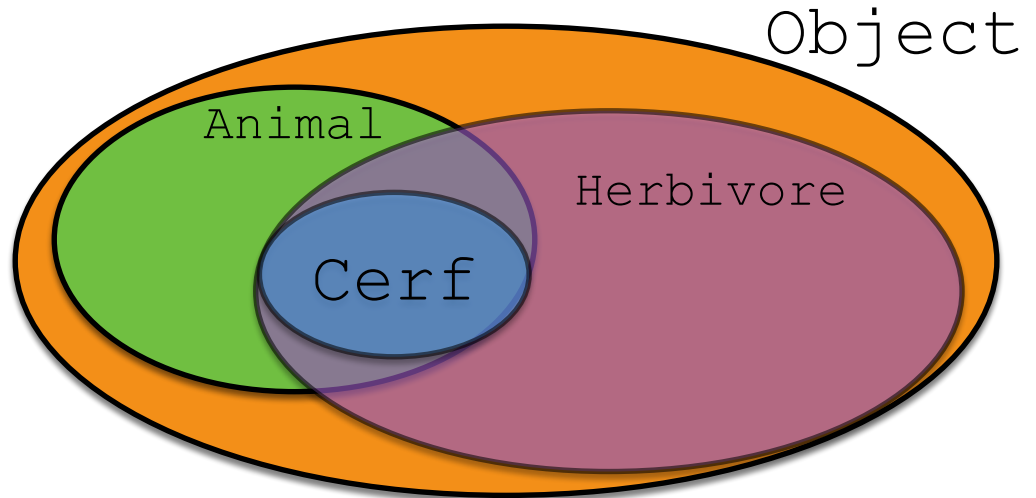
Le polymorphisme

Object



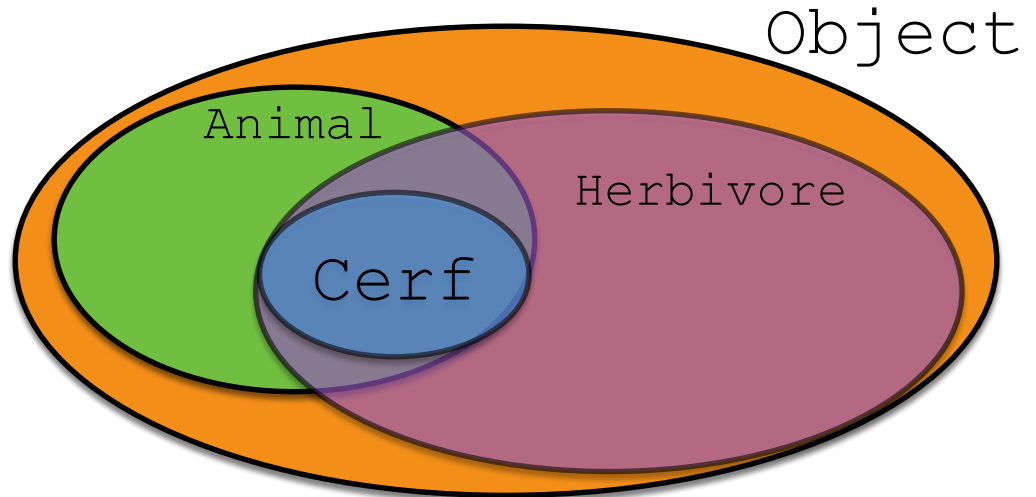
Le polymorphisme

```
public interface Herbivore{}
```



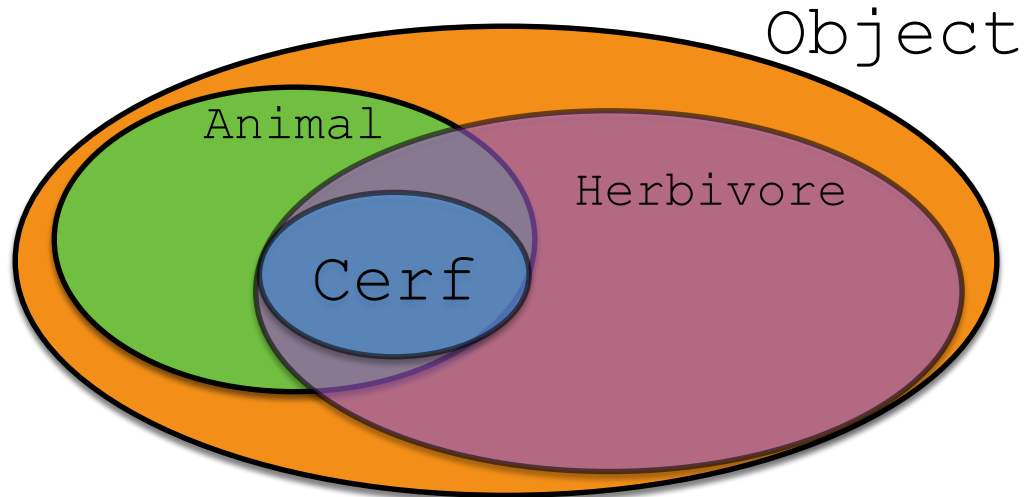
Le polymorphisme

```
public interface Herbivore{}  
public class Animal{}
```



Le polymorphisme

```
public interface Herbivore{}  
public class Animal{}  
public class Cerf extends Animal implements Herbivore{}
```

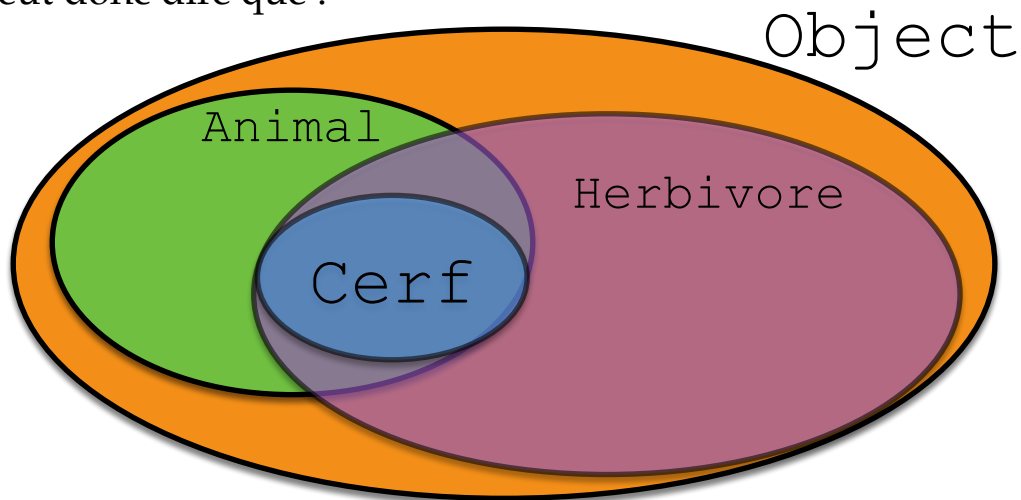


Le polymorphisme

```
public interface Herbivore{}  
public class Animal{}  
public class Cerf extends Animal implements Herbivore{}
```

- La classe Cerf est considérée comme polymorphique puisqu'elle hérite de plusieurs classes (héritage + extension). On peut donc dire que :

- Un Cerf IS-A Animal
- Un Cerf IS-A Herbivore
- Un Cerf IS-A Cerf
- Un Cerf IS-A Object



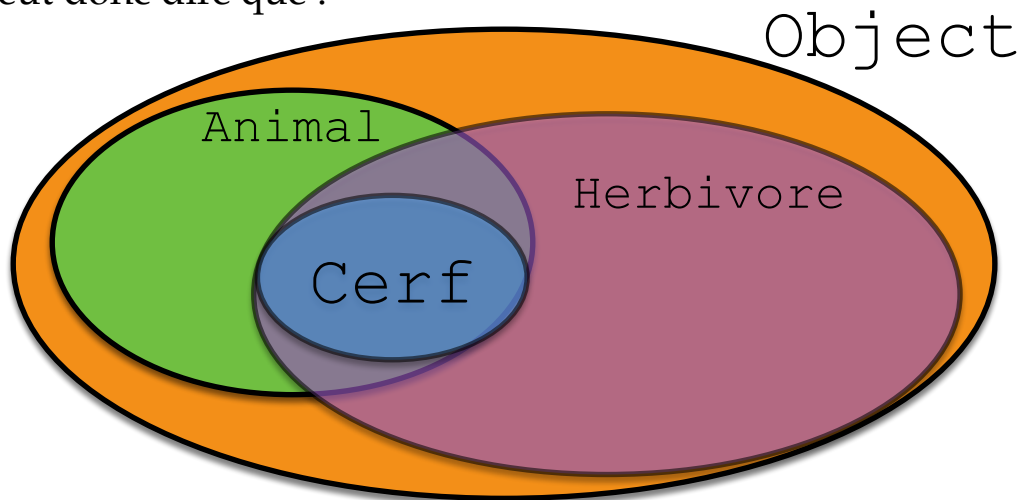
Le polymorphisme

```
public interface Herbivore{}  
public class Animal{}  
public class Cerf extends Animal implements Herbivore{}
```

- La classe Cerf est considérée comme polymorphique puisqu'elle hérite de plusieurs classes (héritage + extension). On peut donc dire que :

- Un Cerf IS-A Animal
- Un Cerf IS-A Herbivore
- Un Cerf IS-A Cerf
- Un Cerf IS-A Object

- On peut alors déclarer :
Cerf d = **new** Cerf();
Animal a = d;
Herbivore v = d;
Object o = d;



Typage statique ou dynamique

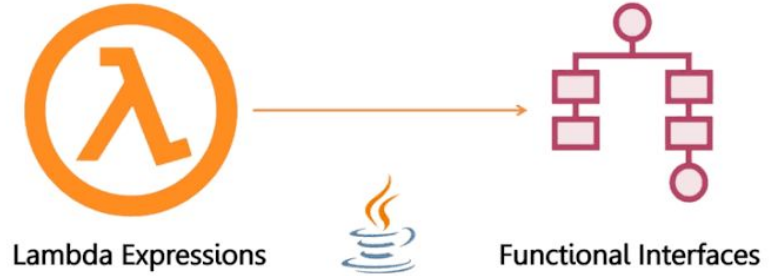
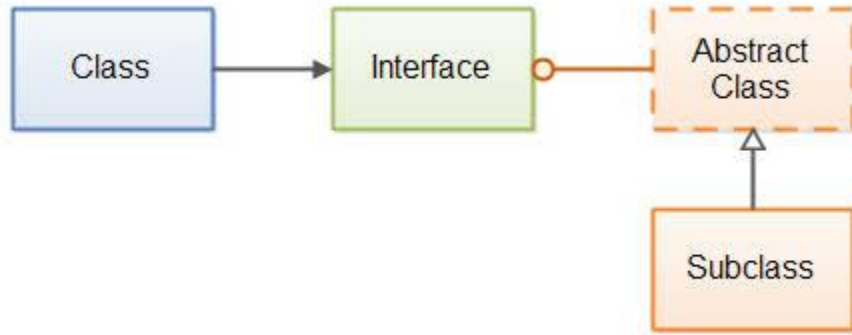
- Une conséquence du polymorphisme : le recours au «casting» pour trouver les fonctionnalités propres à la sous-classe
- Typage statique (comportement par défaut de C++) : le compilateur a décidé du type de l'objet et donc des méthodes qui seront invoquées
- Typage dynamique (comportement par défaut de Java, qui peut être mis en place en C++) : on diffère jusqu'à l'exécution le fait d'identifier le type réel de l'objet sur lequel la fonction est invoquée

Les classes abstraites

- Classe qui comme unique rôle de **factoriser** des méthodes et des attributs communs aux sous-classes
 - à un niveau très général, on peut être incapable de donner la définition de certaines méthodes => on parle de méthodes abstraites
 - en conséquence, on **ne peut pas instancier** une classe abstraite !
 - les instances sont obligatoirement des instances d'une de ses sous-classes
 - au contraire des super classes concrètes, les super classes abstraites **obligent à redéfinir** les méthodes abstraites dans leurs sous-classes
 - tant que la méthode abstraite n'est pas redéfinie, chacune de ces sous-classes reste abstraite
- Une classe abstraite peut implémenter des opérations
- Une classe abstraite peut avoir des attributs

La généricité

- Idée partagée par **tous** les **langages OO**
- Permet de définir un code pour des fonctionnalités universelles, applicables sur tout type d'objet, et de récupérer ces fonctionnalités pour un type donné
- En C++, géré par le compilateur qui applique un code générique pour générer un code à appliquer à un type particulier
 - lors de l'exécution, la fonctionnalité ne sera plus générique mais bien instanciée pour le type en question
- Java (et C#) recourait jusqu'à présent à la classe de base de tous les objets. Lors de l'exécution, les objets sont « castés »
 - approche plus intuitive et plus simple à mettre en oeuvre, mais plus vulnérable en raison de « casting »
 - actuellement un mécanisme de généricité très proche de l'esprit du C++ afin d'éviter les erreurs d'exécution qui peuvent découler du casting



Interface et Lambda

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Héritage

```
class Point{

    protected int x , y;

    Point(int x, int y) {
        this.x=x;
        this.y=y;
    }

    public void affiche() {
        System.out.println(x+"-"+y);
    }

    public void setX(int x) {
        this.x=x;
    }

}
```

Héritage

```
class Point{  
  
    protected int x , y;  
  
    Point(int x, int y){  
        this.x=x;  
        this.y=y;  
    }  
  
    public void affiche(){  
        System.out.println(x+"-"+y);  
    }  
  
    public void setX(int x){  
        this.x=x;  
    }  
}
```

```
class ColoredPoint extends Point{  
    private byte couleur ;  
    ColoredPoint(int x, int y,int c){  
        super(x,y);  
        couleur = c;  
    }  
  
    public void affiche(){  
        super.affiche();  
        System.out.println(  
            " Couleur : " + couleur );  
    }  
  
    public void setCouleur(int c){  
        couleur = c;  
    }  
}
```

Héritage

`(class Point{)`

`protected int x , y;`

```
Point(int x, int y) {  
    this.x=x;  
    this.y=y;  
}
```

```
public void affiche() {  
    System.out.println(x+"-"+y);  
}
```

```
public void setX(int x) {  
    this.x=x;  
}
```

`class ColoredPoint extends Point{`

`private byte couleur ;`

```
ColoredPoint(int x, int y,int c) {  
    super(x,y);  
    couleur = c;  
}
```

```
public void affiche() {  
    super.affiche();  
    System.out.println(  
        " Couleur : " + couleur );  
}
```

```
public void setCouleur(int c) {  
    couleur = c;  
}
```

Héritage

```
class Point{  
  
    protected int x , y;  
  
    ( Point(int x, int y) {  
        this.x=x;  
        this.y=y;  
    }  
  
    public void affiche() {  
        System.out.println(x+"-"+y);  
    }  
  
    public void setX(int x) {  
        this.x=x;  
    }  
}
```

```
class ColoredPoint extends Point{  
    private byte couleur ;  
    ColoredPoint(int x, int y,int c) {  
        super(x,y);  
        couleur = c;  
    }  
  
    public void affiche() {  
        super.affiche();  
        System.out.println(  
            " Couleur : " + couleur );  
    }  
  
    public void setCouleur(int c) {  
        couleur = c;  
    }  
}
```

Héritage

```
class Point{  
  
    protected int x , y;  
  
    Point(int x, int y){  
        this.x=x;  
        this.y=y;  
    }  
  
    (public void affiche()){  
        System.out.println(x+"-"+y);  
    }  
  
    public void setX(int x){  
        this.x=x;  
    }  
}
```

```
class ColoredPoint extends Point{  
    private byte couleur ;  
    ColoredPoint(int x, int y,int c){  
        super(x,y);  
        couleur = c;  
    }  
  
    public void affiche() {  
        super.affiche();  
        System.out.println(  
            " Couleur : " + couleur );  
    }  
  
    public void setCouleur(int c){  
        couleur = c;  
    }  
}
```

Ancêtre et descendant

```
Point p = new Point ( 3 , 4 ) ;  
ColoredPoint cp = new ColoredPoint (4 , 5 , 1 ) ;  
  
p.affiche() // x : 3 , y : 4  
cp.affiche() // x : 4 , y : 5, Couleur : 1  
  
cp.setCouleur ( 2 ) ;  
cp.setX ( 4 ) ;  
  
p = cp ;    //affection de références  
  
p.setCouleur ( 1 )
```

Ancêtre et descendant

```
Point p = new Point ( 3 , 4 ) ;  
ColoredPoint cp = new ColoredPoint (4 , 5 , 1 ) ;
```

```
p.affiche() // x : 3 , y : 4  
cp.affiche() // x : 4 , y : 5, Couleur : 1
```

```
cp.setCouleur ( 2 ) ;  
cp.setX ( 4 ) ;
```

```
p = cp ; //affection de références
```

```
p.setCouleur ( 1 ) // rejeté par le compilateur
```

Ancêtre et descendant

```
Point p = new Point ( 3 , 4 ) ;
ColoredPoint cp = new ColoredPoint (4 , 5 , 1 ) ;

p.affiche() // x : 3 , y : 4
cp.affiche() // x : 4 , y : 5, Couleur : 1

cp.setCouleur ( 2 ) ;
cp.setX ( 4 );

p = cp ;    //affection de références

p.setCouleur ( 1 ) // rejeté par le compilateur
((ColoredPoint) p).setCouleur ( 1 ) ; //solution=cast
```


Surdéfinition de méthodes

- A noter que les constructeurs ne sont pas considérés comme des méthodes donc ne sont pas hérités
- Le type du résultat, le nom de la méthode, le nombre et le type des paramètres d'une méthode qui "surdéfinit" doit correspondre exactement à ceux de la méthode de base
- Le spécificateur d'accès de la nouvelle méthode peut permettre un accès plus large, mais jamais plus strict
- La méthode surdéfinie peut toujours être invoquée en utilisant le mot-clé **super**
- Une sous-classe ne peut pas surdéfinir une méthode
 - déclarée final dans la classe de base
 - déclarée static dans la classe de base
 - une classe dérivée peut masquer une méthode de classe (**static**) de la classe de base en déclarant une méthode statique de même signature

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets
- Les méthodes `equals` et `hashCode`
 - La méthode `equals` fournie utilise la fonction d'identité pour déterminer si deux objets sont égaux, c'est-à-dire si ils font référence au même objet ; cependant, on peut la redéfinir pour que deux objets soient considérés comme égaux si ils contiennent les mêmes informations

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets
- Les méthodes `equals` et `hashCode`
 - La méthode `equals` fournie utilise la fonction d'identité pour déterminer si deux objets sont égaux, c'est-à-dire si ils font référence au même objet ; cependant, on peut la redéfinir pour que deux objets soient considérés comme égaux si ils contiennent les mêmes informations
- La méthode `toString` retourne un objet `String` représentation de l'objet sous forme de chaîne de caractères
 - nom de la classe + une représentation hexadécimale (non signée) du hashcode de l'objet
 - utile pour l'affichage des informations sur un objet

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets
- Les méthodes `equals` et `hashCode`
 - La méthode `equals` fournie utilise la fonction d'identité pour déterminer si deux objets sont égaux, c'est-à-dire si ils font référence au même objet ; cependant, on peut la redéfinir pour que deux objets soient considérés comme égaux si ils contiennent les mêmes informations
- La méthode `toString` retourne un objet `String` représentation de l'objet sous forme de chaîne de caractères
 - nom de la classe + une représentation hexadécimale (non signée) du hashcode de l'objet
 - utile pour l'affichage des informations sur un objet
- La méthode `getClass` retourne un objet de la classe `Class` qui est une représentation en exécution de la classe de l'objet

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets
- Les méthodes `equals` et `hashCode`
 - La méthode `equals` fournie utilise la fonction d'identité pour déterminer si deux objets sont égaux, c'est-à-dire si ils font référence au même objet ; cependant, on peut la redéfinir pour que deux objets soient considérés comme égaux si ils contiennent les mêmes informations
- La méthode `toString` retourne un objet `String` représentation de l'objet sous forme de chaîne de caractères
 - nom de la classe + une représentation hexadécimale (non signée) du hashcode de l'objet
 - utile pour l'affichage des informations sur un objet
- La méthode `getClass` retourne un objet de la classe `Class` qui est une représentation en exécution de la classe de l'objet
- La méthode `finalize` nettoie un objet avant l'opération de récupération mémoire
 - garbage collection : l'environnement d'exécution supprime les objets lorsqu'il peut déterminer qu'ils ne seront plus utilisés
 - on peut implémenter la méthode `finalize` pour libérer des ressources spécifiques

La classe Object

- Cette classe définit l'état de base et le comportement que tout objet doit avoir
 - c'est la classe de base de toute la hiérarchie des objets
- Les méthodes `equals` et `hashCode`
 - La méthode `equals` fournie utilise la fonction d'identité pour déterminer si deux objets sont égaux, c'est-à-dire si ils font référence au même objet ; cependant, on peut la redéfinir pour que deux objets soient considérés comme égaux si ils contiennent les mêmes informations
- La méthode `toString` retourne un objet `String` représentation de l'objet sous forme de chaîne de caractères
 - nom de la classe + une représentation hexadécimale (non signée) du hashcode de l'objet
 - utile pour l'affichage des informations sur un objet
- La méthode `getClass` retourne un objet de la classe `Class` qui est une représentation en exécution de la classe de l'objet
- La méthode `finalize` nettoie un objet avant l'opération de récupération mémoire
 - garbage collection : l'environnement d'exécution supprime les objets lorsqu'il peut déterminer qu'ils ne seront plus utilisés
 - on peut implémenter la méthode `finalize` pour libérer des ressources spécifiques
- La méthode `clone` est utilisée pour créer un objet à partir d'un autre existant
 - la classe doit implémenter l'interface `Cloneable`
 - ne pas utiliser l'opérateur `new` ou un constructeur pour créer un clone, mais appeler `super.clone`

Classes finales

- Utile pour des besoins de sécurité ou de conception
- Une classe finale est une classe dont il n'est pas possible de définir de classe dérivée
- Ajouter le mot-clé **final** devant la déclaration de classe
- Méthodes finales
 - On peut utiliser le mot-clé **final** dans une déclaration de méthode pour indiquer au compilateur que la méthode ne pourra pas être surdéfinie dans les sous-classes

Classes abstraites

- Une classe abstraite représente un concept abstrait et ne peut donc pas être instanciée
- On utilise le mot-clé **abstract** devant le mot-clé `class` au moment de la déclaration de la classe
- Une classe abstraite peut contenir des méthodes abstraites, c'est-à-dire des méthodes dont on ne donne pas l'implémentation
 - dans ce cas, la classe abstraite laisse à ses classes dérivées le soin de donner l'implémentation de ces méthodes

Interfaces

- Une interface définit un contrat de service (protocole de communication) qui peut être implémenté dans n'importe quelle classe
 - on y définit un ensemble de méthodes que l'on n'implémente pas
- Une interface ne implémenter que des méthodes **default** ou **static**
- Une interface peut aussi contenir des déclarations de constantes
 - une constante définie dans une interface est implicitement public, static, et final.
- Déclaration d'interface
 - public (ou rien) + **interface** + nom + **extends** + superinterface
- Le corps de l'interface contient les déclarations de toutes les méthodes abstraites
 - la déclaration d'une méthode est suivie d'une point-virgule
 - toute méthode sans corps déclarée est implicitement publique et abstraite

Implémentation d'interface

- Une classe qui implémente une interface doit implémenter toutes les méthodes déclarées dans cette interface

```
interface Affichable{  
    void affiche() ;  
}
```

Implémentation d'interface

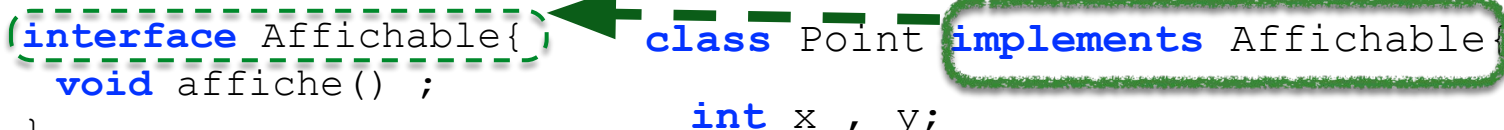
- Une classe qui implémente une interface doit implémenter toutes les méthodes déclarées dans cette interface

```
interface Affichable{  
    void affiche() ;  
}  
  
class Point implements Affichable{  
    int x , y;  
  
    Point(int x, int y){  
        this.x=x;  
        this.y=y;  
    }  
  
    void affiche(){  
        System.out.println(  
            "x : "+x+" , y : "+y);  
    }  
}
```

Implémentation d'interface

- Une classe qui implémente une interface doit implémenter toutes les méthodes déclarées dans cette interface

```
interface Affichable{  
    void affiche() ;  
}  
  
class Point implements Affichable{  
    int x , y;  
  
    Point(int x, int y){  
        this.x=x;  
        this.y=y;  
    }  
  
    void affiche(){  
        System.out.println(  
            "x : "+x+" , y : "+y);  
    }  
}
```



Implémentation d'interface

- Une classe qui implémente une interface doit implémenter toutes les méthodes déclarées dans cette interface

```
interface Affichable{  
    void affiche() ;  
}
```

```
class Point implements Affichable{  
    int x , y;
```

```
    Point(int x, int y){  
        this.x=x;  
        this.y=y;  
    }
```

```
    void affiche(){  
        System.out.println(  
            "x : "+x+" , y : "+y);  
    }
```

```
}
```

Définition de méthode - interface

Que se passe-t-il si on modifie à posteriori une interface en y ajoutant un nouveau service (méthode) ?

```
interface IContact {  
  
    List<String> standardistes = Arrays.asList("Pierre", "Franck", "Jeremy");  
    int idClass = 23 ;  
    String mailSecours = "secours@info.net" ;  
  
    static int getId() {  
        return idClass ;  
    }  
  
    public void envoieMessage(String message, String destinataire);  
  
}
```

Définition de méthode - interface

Une classe implémentant une interface doit fournir une implémentation pour chaque méthode définie par l'interface ou hériter de l'implémentation d'une superclasse.

Problème : Si les concepteurs d'une bibliothèque mettent à jour une interface en ajoutant une nouvelle méthode.

Par exemple, l'API Java 8 introduit de nombreuses nouvelles méthodes sur les interfaces existantes (`sort` ou `replaceAll` sur l'interface `List`).

=> **Modifier** toutes les classes implémentant l'interface ([List](#)) pour fournir une implémentation pour les nouvelles méthodes.

Définition de méthode - interface

Java 8 introduit un mécanisme pour résoudre ce problème en permettant de déclarer des méthodes avec un code d'implémentation :

- des méthodes **static** à l'intérieur des interfaces (non-abstraite et non-default).
- des méthodes par **défaut** permettant de fournir une implémentation par défaut pour les méthodes dans une interface.

Les classes existantes implémentant une interface **hériteront automatiquement** des implémentations par défaut si elles n'en fournissent pas explicitement.

Permet de faire évoluer des interfaces de manière **non intrusive**.

Définition de méthode - interface

Que se passe-t-il si on modifie à posteriori une interface en y ajoutant un nouveau service (méthode) ?

```
interface IContact {  
  
    List<String> standardistes = Arrays.asList("Pierre", "Franck", "Jeremy");  
    int idClass = 23 ;  
    String mailSecours = "secours@info.net" ;  
  
    static int getId() {  
        return idClass ;  
    }  
  
    public void envoieMessage(String message, String destinataire);  
  
    default void forwardMessage (String message) {  
        envoieMessage (message, mailSecours);  
    }  
}
```

Classe abstraite ≠ Interface

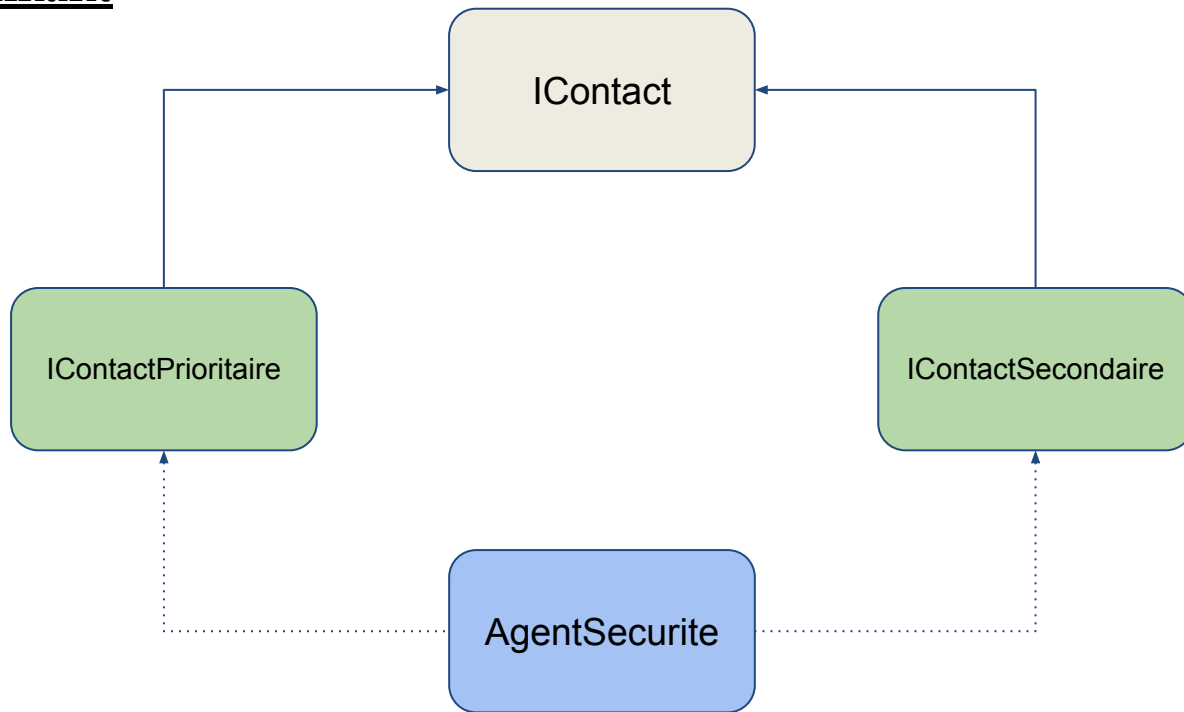
Une classe abstraite :

- peut posséder des **constructeurs** contrairement aux interfaces.
- peut **redéfinir** les **méthodes** de la classe **Object** contrairement à une interface.
- peut déclarer des **variables d'instance** avec tous les modificateurs d'accès possibles accessibles dans les classes dérivées. Une interface contient uniquement des variables **public**, **static** et **final**, et ne peut posséder aucune variable d'instance.
- peut déclarer des **blocs** d'initialisation d'instance et des blocs statiques, alors qu'une interface ne peut en comporter aucun.
- ne **peut pas** faire référence à une **expression lambda**, tandis qu'une interface peut posséder une méthode abstraite unique pouvant faire référence à une expression lambda.

Interface et héritage multiple

Héritage en diamant

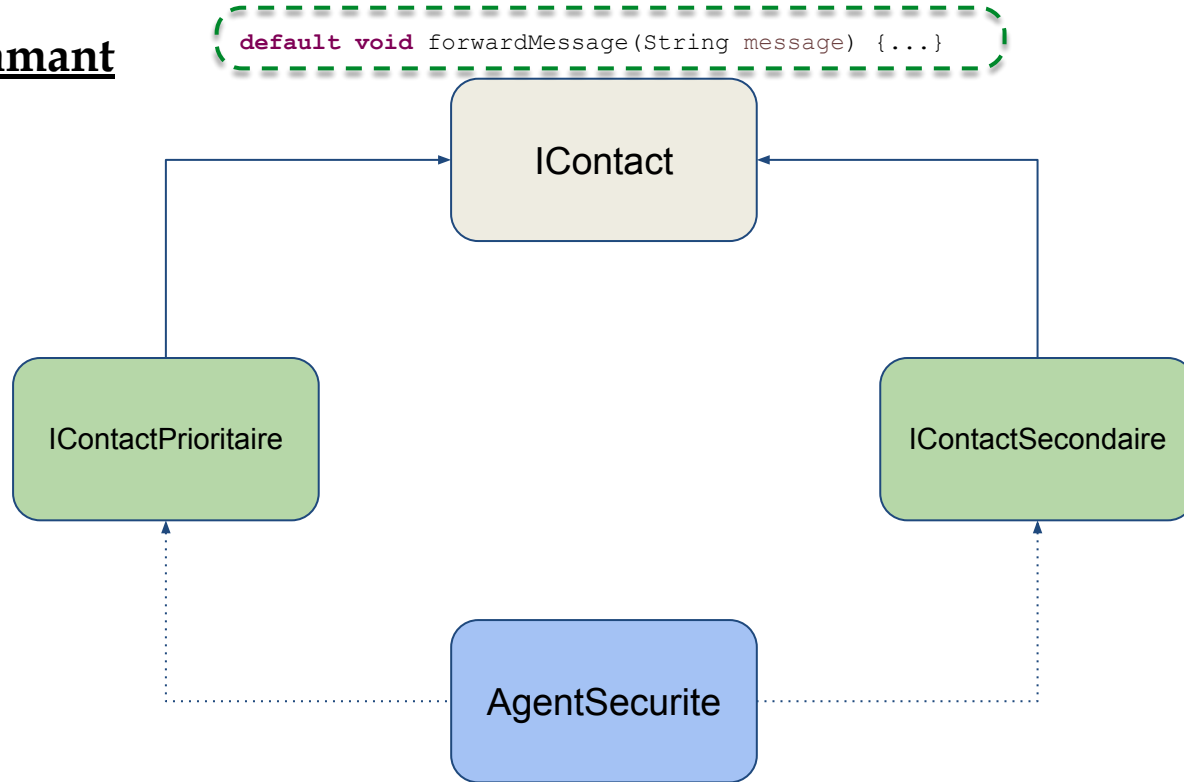
```
default void forwardMessage(String message) {...}
```



Quelle sera la méthode `forwardMessage` utilisée par `AgentSecurite` ?

Interface et héritage multiple

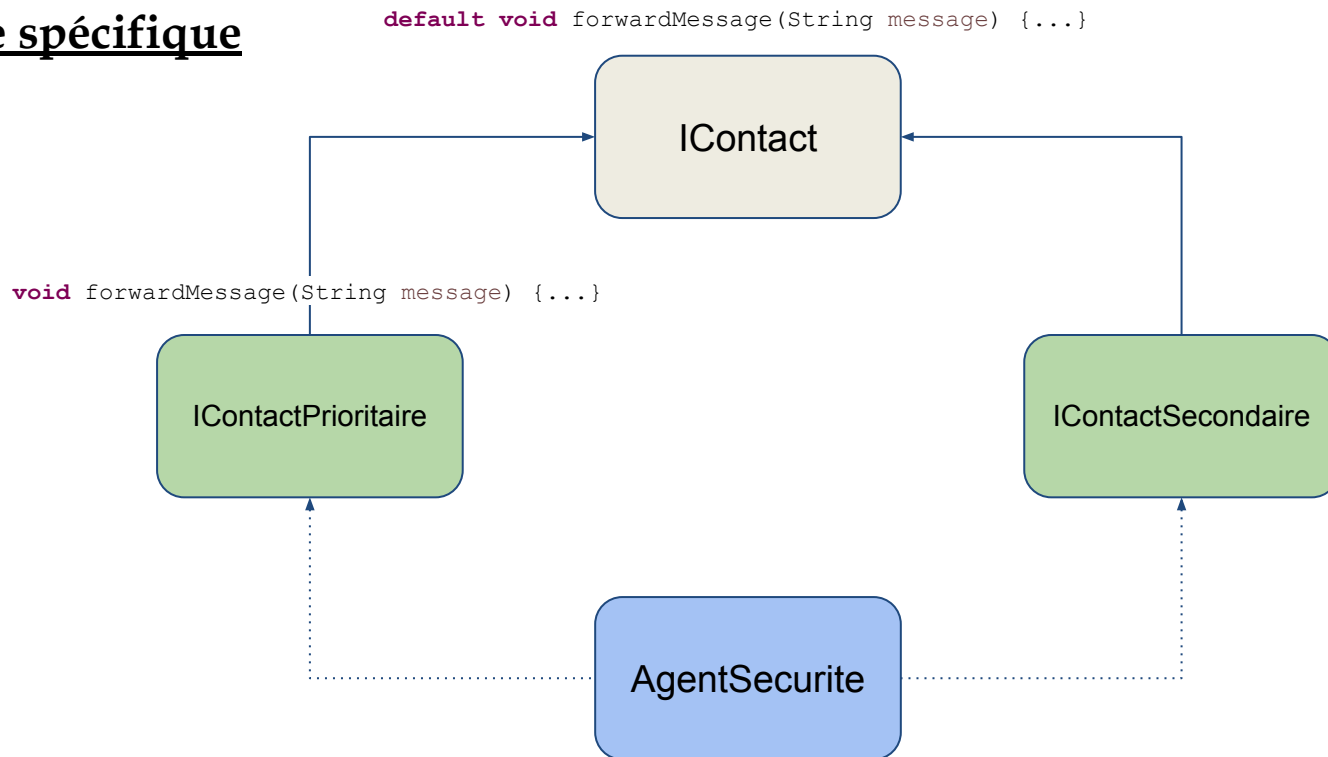
Héritage en diamant



Quelle sera la méthode `forwardMessage` utilisée par `AgentSecurite` ?

Interface et héritage multiple

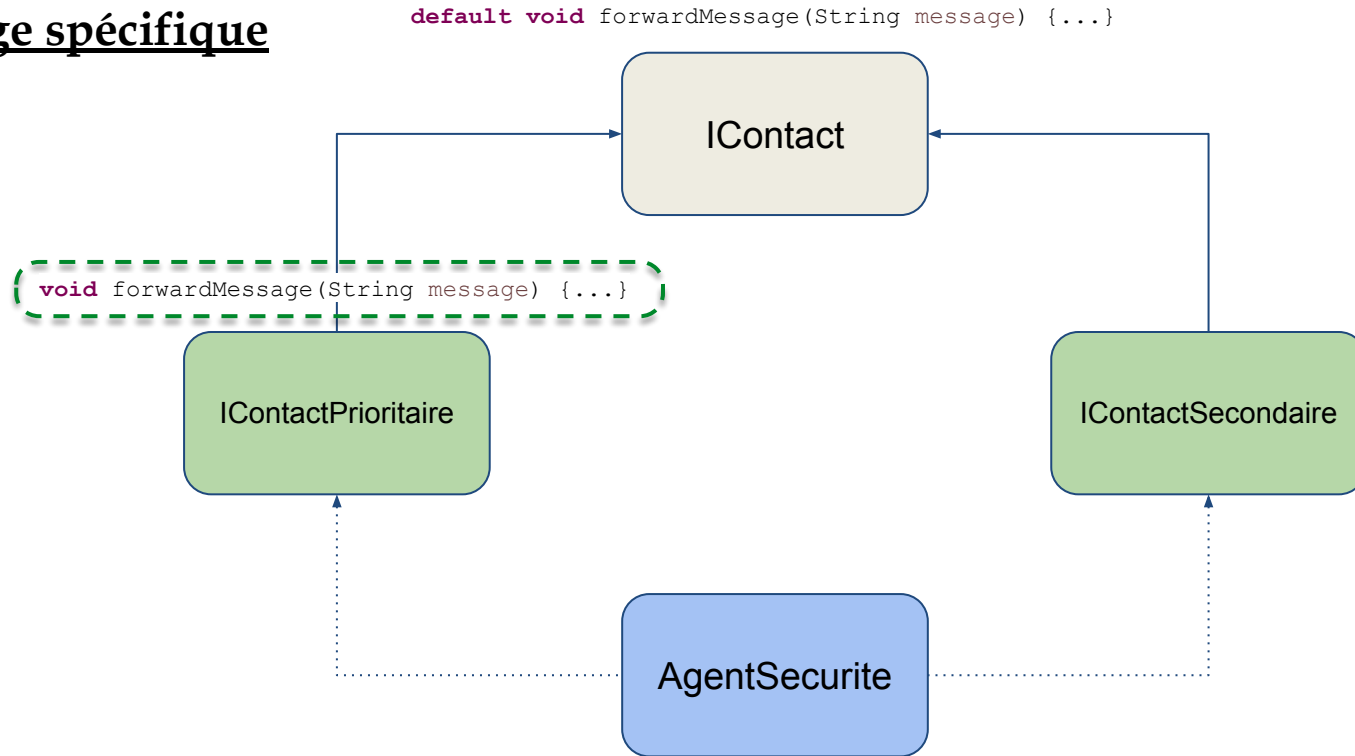
Héritage spécifique



Quelle sera la méthode `forwardMessage` utilisée par `AgentSecurite` ?

Interface et héritage multiple

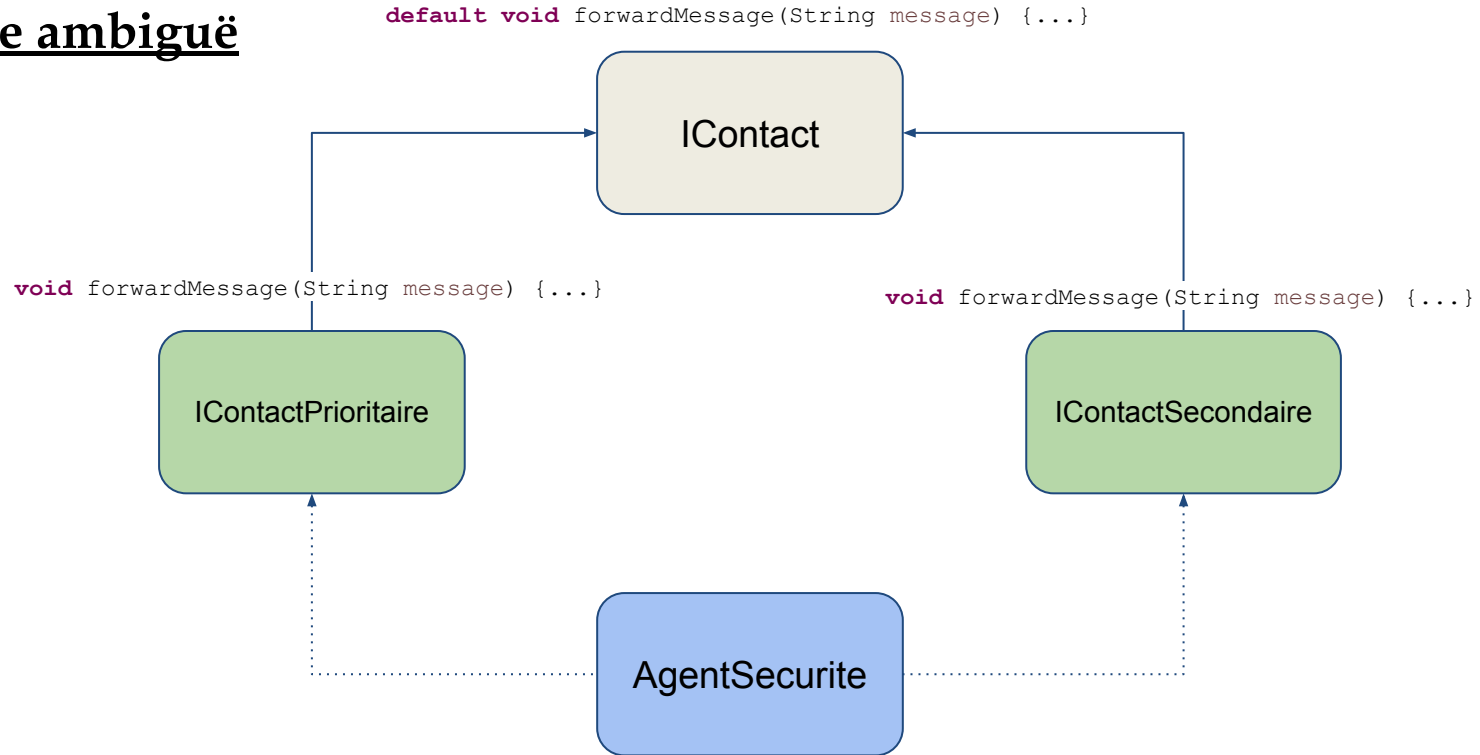
Héritage spécifique



Quelle sera la méthode `forwardMessage` utilisée par **AgentSecurite** ?

Interface et héritage multiple

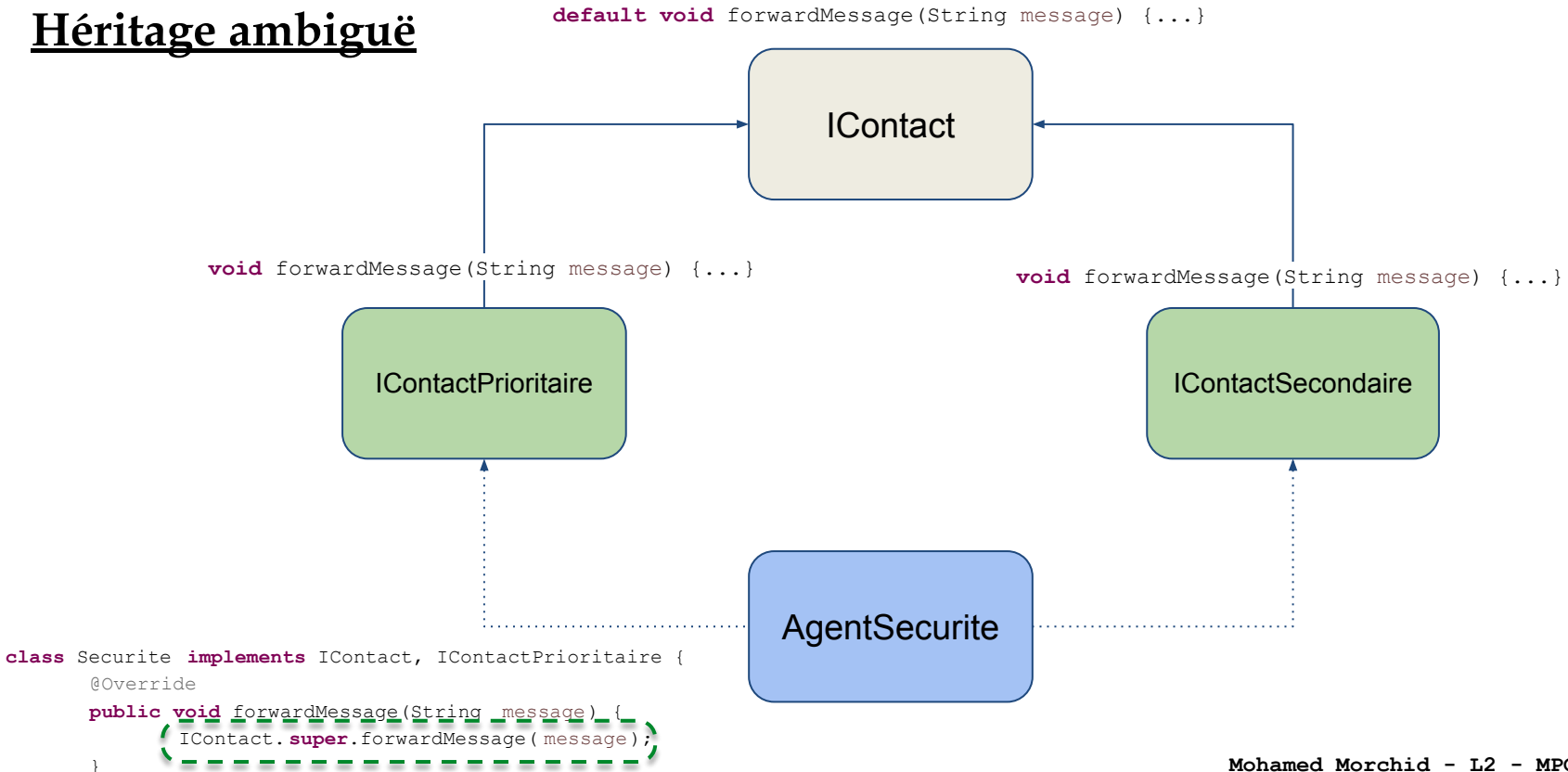
Héritage ambiguë



Quelle sera la méthode `forwardMessage` utilisée par `AgentSecurite` ?

Interface et héritage multiple

Héritage ambiguë



Lambda

Lambda - Motivations

Pour passer des traitements en paramètres d'une méthode, Java 8 propose les expressions lambda nommées aussi closures ou fonctions anonymes.

Le résultat de traitements est décrit mais pas la façon dont ils sont réalisés.

Ceci permet de réduire la quantité de code à écrire pour obtenir le même résultat.

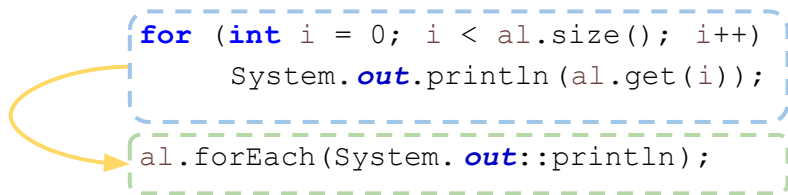
Lambda - Motivations

Pour passer des traitements en paramètres d'une méthode, Java 8 propose les expressions lambda nommées aussi closures ou fonctions anonymes.

Le résultat de traitements est décrit mais pas la façon dont ils sont réalisés.

Ceci permet de réduire la quantité de code à écrire pour obtenir le même résultat.

```
List<String> al = Arrays.asList("Chat", "Chien");
```



```
for (int i = 0; i < al.size(); i++)  
    System.out.println(al.get(i));
```

```
al.forEach(System.out::println);
```

Lambda - Définition

Les expressions lambda permettent d'écrire du code plus concis, donc plus rapide à écrire, à relire et à maintenir.

Une expression lambda :

- est une **fonction anonyme** définie sans déclaration explicite du type de retour, ni de modificateurs d'accès ni de nom.
- est un raccourci syntaxique permettant de **définir** une méthode directement à **l'endroit** où elle est **utilisée** et peut être passé à d'autres traitements.
- est un raccourci syntaxique pour une interface qui ne possède qu'une seule méthode abstraite (**interface fonctionnelle** `@FunctionalInterface`).
- est **évaluée** par le **compilateur**, celui-ci infère le type vers l'interface fonctionnelle. Cela lui permet d'obtenir des informations sur les paramètres utilisés, le type de la valeur de retour, les exceptions qui peuvent être levées.

Lambda - Syntaxe

La syntaxe d'une expression lambda est composée de trois parties :

- un ensemble de paramètres, d'aucun à plusieurs
- l'opérateur ->
- le corps de la fonction

Elle peut prendre deux formes principales : (paramètres) -> { traitements; }

```
(paramètres) -> expression;
```

```
(paramètres) -> {traitements;}
```

```
@FunctionalInterface
interface Op {
    long operation(int a, int b);
}
```

```
Op addition = (a, b) -> a + b;
```

```
Runnable run = () -> { System.out.println("run");
};
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Problème ici ?

```
Consumer afficher = (String param) -> System.out.println(param);
```


Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Problème ici ?

```
Consumer afficher = (String param) -> System.out.println(param);
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Solution 1

```
Consumer afficher = (String param) -> System.out.println(param);
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Solution 1

```
Consumer afficher = param -> System.out.println(param);
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Solution 2

```
Consumer<String> afficher = (String param) ->  
System.out.println(param);
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Solution 3

```
Consumer afficher = System.out::println;
```

Référence de méthode

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Problème ici ?

```
Comparator comp = (c1, String c2) -> Integer.compare(c1.length(), c2.length());
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Problème ici ?

```
Comparator comp = (c1, String c2) -> Integer.compare(c1.length(), c2.length());
```

Lambda - Syntaxe

L'écriture d'une expression lambda doit respecter plusieurs règles générales :

- zéro, un ou plusieurs paramètres dont le **type** peut être déclaré **explicitement** ou **inféré** par le compilateur
- les **paramètres** sont entourés par des **parenthèses** et séparés par des **virgules**. Des parenthèses vides indiquent qu'il n'y a pas de paramètre
- lorsqu'il n'y a qu'un seul paramètre et que son type est inféré alors les parenthèses ne sont pas obligatoires
- Si le corps ne contient d'une seule instruction, les accolades ne sont pas obligatoires et le type de retour correspond à celui de l'instruction.

Solution

```
Comparator<String> comp = (String c1, String c2) -> Integer.compare(c1.length(), c2.length() );
```


Lambda - Paramètres

Les paramètres d'une expression lambda doivent correspondre à ceux définis dans l'interface fonctionnelle et respecter certaines règles :

- une expression peut n'avoir aucun, un seul ou plusieurs paramètres
- le type des paramètres peuvent être explicitement déclaré ou être inféré par le compilateur selon le contexte dans lequel l'expression est utilisée
- les paramètres sont entourés de parenthèses, chacun étant séparé par une virgule
- des parenthèses vides indiquent qu'il n'y a pas de paramètre
- s'il n'y a qu'un seul paramètre dont le type n'est pas explicitement précisé, alors l'utilisation des parenthèses n'est pas obligatoire

Lambda - Corps

Il peut être :

- une expression unique
- un bloc de code composé d'une ou plusieurs instructions entourées par des accolades

Le corps de l'expression doit respecter certaines règles :

- il peut n'avoir aucune, une seule ou plusieurs instructions
- lorsqu'il ne contient qu'une seule instruction, les accolades ne sont pas obligatoires et la valeur de retour est celle de l'instruction si elle en possède une
- lorsqu'il y a plusieurs instructions, elles doivent obligatoirement être entourées d'accolades
- la valeur de retour est celle de la dernière expression ou `void` si rien n'est retourné

Si le corps est simplement une expression, celle-ci est évaluée et le résultat de cette évaluation est renvoyé s'il y en a un.

Lambda - Interface fonctionnelle Java

Le package `java.util.function` propose des interfaces fonctionnelles d'usage courant annotées avec `@FunctionalInterface` :

- **Function** : une fonction **unaire** qui permet de réaliser une **transformation**. Elle attend un ou plusieurs paramètres et renvoie une valeur. La méthode se nomme `apply()`
- **Predicate** : une fonction qui attend un ou plusieurs paramètres et renvoie un **booléen**. La méthode se nomme `test()`
- **Supplier** : une fonction (fabrique) qui renvoie une **instance**. La méthode se nomme `get()`
- **Consumer** : une fonction qui permet de réaliser une **action**. Elle ne renvoie **pas de valeur** et attend un ou plusieurs paramètres. La méthode se nomme `accept()`

Lambda - @FunctionalInterface Consumer

L'interface fonctionnelle `Consumer<T>` définit une fonction qui effectue une opération sur un objet et ne renvoie aucune valeur.

Elle définit la méthode fonctionnelle `accept(T t)` qui ne renvoie aucune valeur.

Elle définit aussi une méthode par défaut `Consumer andThen(Consumer< ? super T> renvoyant un Consumer qui exécute en séquence l'instance courante et celle fournie en paramètre.`

```
Consumer<String> cons = System.out::print;
```

```
cons.andThen(cons).andThen(cons).accept("salut-");
```

> ?

Lambda - @FunctionalInterface Consumer

L'interface fonctionnelle `Consumer<T>` définit une fonction qui effectue une opération sur un objet et ne renvoie aucune valeur.

Elle définit la méthode fonctionnelle `accept(T t)` qui ne renvoie aucune valeur.

Elle définit aussi une méthode par défaut `Consumer andThen(Consumer< ? super T> renvoyant un Consumer qui exécute en séquence l'instance courante et celle fournie en paramètre.`

```
Consumer<String> cons = System.out::print;  
  
cons.andThen(cons).andThen(cons).accept("salut-");  
  
> salut-salut-salut-
```

Lambda - @FunctionalInterface Consumer

L'interface fonctionnelle `Consumer<T>` définit une fonction qui effectue une opération sur un objet et ne renvoie aucune valeur.

Elle définit la méthode fonctionnelle `accept(T t)` qui ne renvoie aucune valeur.

Elle définit aussi une méthode par défaut `Consumer andThen(Consumer< ? super T> renvoyant un Consumer qui exécute en séquence l'instance courante et celle fournie en paramètre.`

```
Consumer<Integer> cons = (v) -> v * 3;
```

```
cons.andThen(cons).andThen(cons).accept(2);
```

> ?

Lambda - @FunctionalInterface Consumer

L'interface fonctionnelle `Consumer<T>` définit une fonction qui effectue une opération sur un objet et **ne renvoie aucune valeur**.

Elle définit la méthode fonctionnelle `accept(T t)` qui ne renvoie aucune valeur.

Elle définit aussi une méthode par défaut `Consumer andThen(Consumer< ? super T>)` renvoyant un `Consumer` qui exécute en séquence l'instance courante et celle fournie en paramètre.

```
Consumer<Integer> cons = (v) -> v * 3;
```

```
cons.andThen(cons).andThen(cons).accept(2);
```

> **Ne compile pas car Consumer ne retourne rien**

Lambda - Référence de méthode

Les références de méthodes permettent d'offrir une syntaxe simplifiée pour invoquer une méthode comme une expression lambda dont le but est d'invoquer une méthode ou un constructeur.

Une expression **lambda** correspond à une **méthode anonyme** dont le type est défini par une interface fonctionnelle alors que la **référence de méthode** permet **d'invoquer** une **méthode** statique ou non ou un constructeur.

Les références de méthodes ou de constructeurs utilisent le nouvel opérateur `::`.

Lambda - Référence de méthode

Type	Syntaxe	Référence de méthode	Expression lambda	Exemple
Méthode statique	nomClasse::nomMethode Statique	System.out::println Math::pow	x -> System.out.println(x) (x, y) -> Math.pow(x, y)	String::valueOf
Méthode sur une instance	objet::nomMethode	monObj::maMethode	x -> monObj.maMethode(x)	personne::toString
Méthode d'un objet arbitraire d'un type donné	nomClasse::nomMethode	String::compareTo	(x, y) -> x.compareTo(y)	Object::toString
Constructeur	nomClasse::new	MaClasse::new	() -> new MaClasse();	Personne::new

Lambda - Exemple de référence de méthode

```
public class TestMethodeReference {
```

```
    public static void main(String[] args) {
```

Création Thread

```
    }
```

```
    public static void executer() {
```

```
        System.out.println("execution: " + Thread.currentThread().getName());
```

```
    }
```

```
}
```

Lambda - Exemple de référence de méthode

```
public class TestMethodReference {  
  
    public static void main(String[] args) {
```

Création Thread

```
        // classe anonyme  
        new Thread(new Runnable() {  
            @Override  
            public void run() {  
                executer();  
            }  
        }).start();  
  
        }  
  
    public static void executer() {  
        System.out.println("execution: " + Thread.currentThread().getName());  
    }  
}
```

Lambda - Exemple de référence de méthode

```
public class TestMethodReference {  
  
    public static void main(String[] args) {
```

Création Thread

```
        // classe anonyme  
        new Thread(new Runnable() {  
            @Override  
            public void run() {  
                executer();  
            }  
        }).start();  
  
        // expression lambda  
        new Thread(() -> executer()).start();  
  
    }  
  
    public static void executer() {  
        System.out.println("execution: " + Thread.currentThread().getName());  
    }  
}
```

Lambda - Exemple de référence de méthode

```
public class TestMethodReference {  
  
    public static void main(String[] args) {
```

Création Thread

```
        // classe anonyme  
        new Thread(new Runnable() {  
            @Override  
            public void run() {  
                executer();  
            }  
        }).start();  
  
        // expression lambda  
        new Thread(() -> executer()).start();  
  
        // référence de méthode statique  
        new Thread(TestMethodReference::executer).start();  
    }  
  
    public static void executer() {  
        System.out.println("execution: " + Thread.currentThread().getName());  
    }  
}
```

Lambda - Exemple de référence de méthode

```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}

@FunctionalInterface
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}

public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

Création ouvrage

Lambda - Exemple de référence de méthode

```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}

@FunctionalInterface
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}

public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

Création ouvrage

?

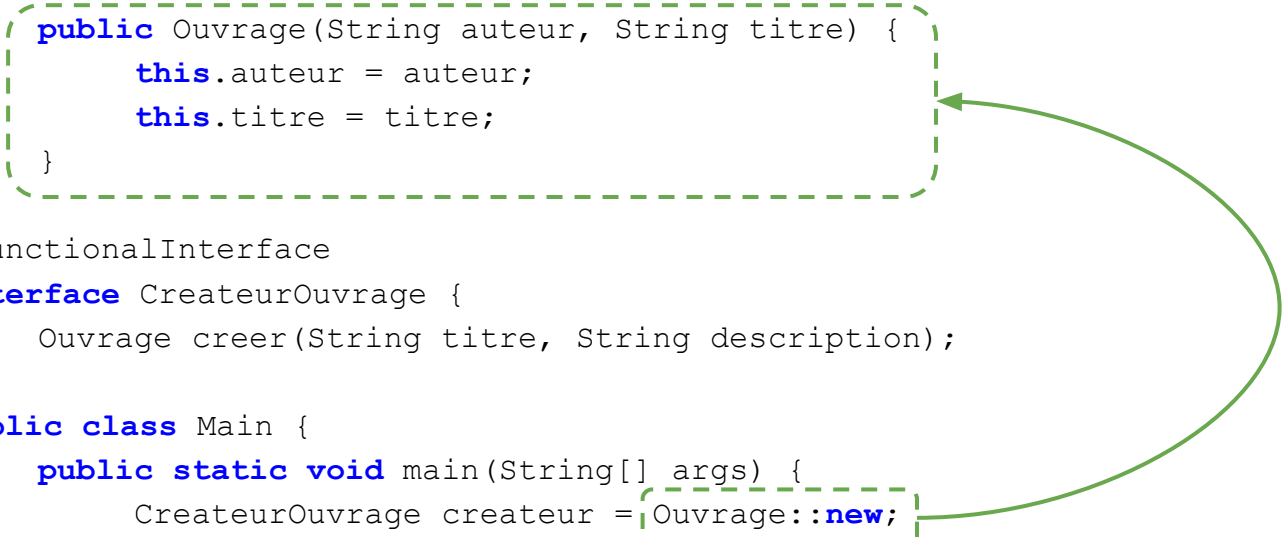
Lambda - Exemple de référence de méthode

```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}

@FunctionalInterface
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}

public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

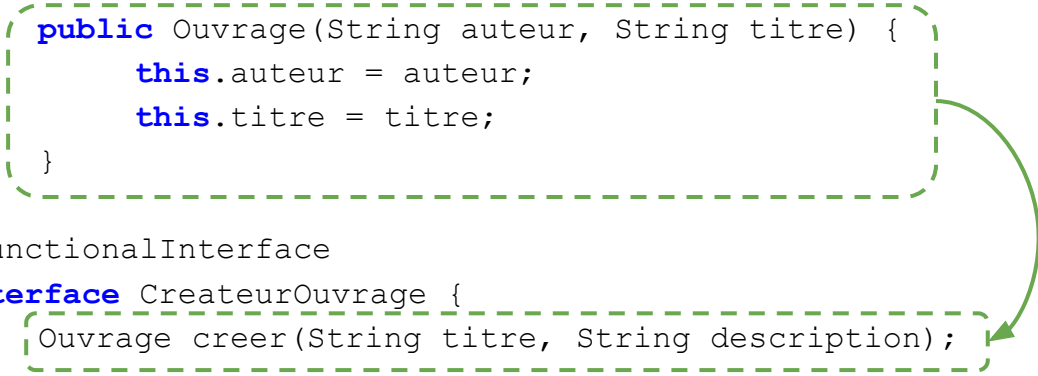
Création ouvrage



Lambda - Exemple de référence de méthode

```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}
@FunctionalInterface
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}
public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

Création ouvrage



Lambda - Exemple de référence de méthode

```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}
```

Création ouvrage

@FunctionalInterface

```
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}
```

```
public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

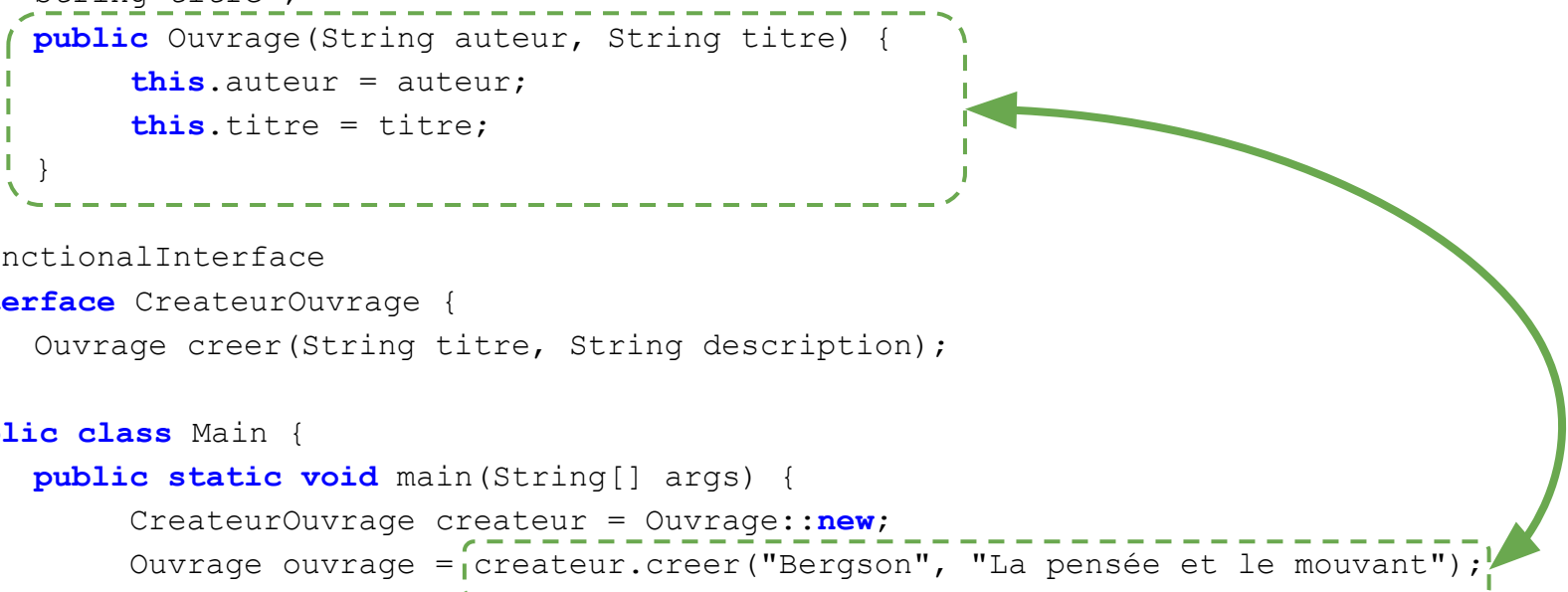
Lambda - Exemple de référence de méthode

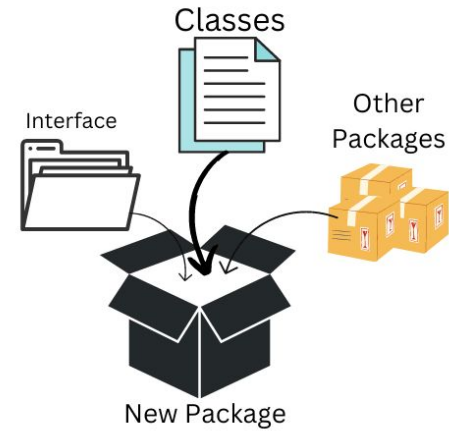
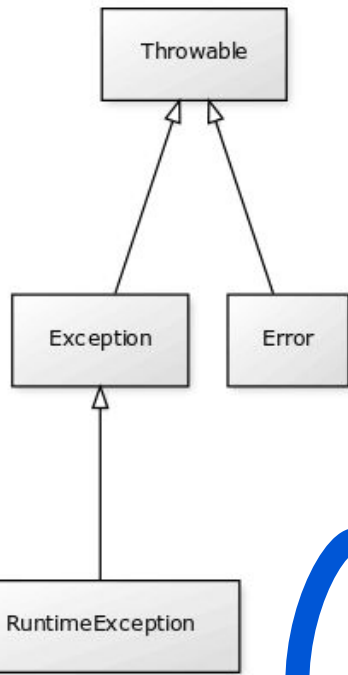
```
class Ouvrage{
    String auteur ;
    String titre ;
    public Ouvrage(String auteur, String titre) {
        this.auteur = auteur;
        this.titre = titre;
    }
}

@FunctionalInterface
interface CreateurOuvrage {
    Ouvrage creer(String titre, String description);
}

public class Main {
    public static void main(String[] args) {
        CreateurOuvrage createur = Ouvrage::new;
        Ouvrage ouvrage = createur.creer("Bergson", "La pensée et le mouvant");
        System.out.println(ouvrage);
    }
}
```

Création ouvrage





Packages et exceptions

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrees-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Packages

- On peut regrouper des classes et des interfaces dans des paquetages ou dossier (packages) pour éviter les conflits de noms et contrôler les accès
- Un package est une collection de classes ou d'interfaces ayant des liens entre elles qui fournit une protection d'accès et une gestion d'espace de noms
- Pour créer un package, il faut utiliser une instruction de package sur la première ligne du fichier source des classes ou interfaces définies dans le package

```
package MyPackage ;
```
- La portée de cette instruction de déclaration de package est le fichier entier

Packages

- On peut regrouper des classes et des interfaces dans des paquetages ou dossier (packages) pour éviter les conflits de noms et contrôler les accès
- Un package est une collection de classes ou d'interfaces ayant des liens entre elles qui fournit une protection d'accès et une gestion d'espace de noms
- Pour créer un package, il faut utiliser une instruction de package sur la première ligne du fichier source des classes ou interfaces définies dans le package

```
package MyPackage ;
```
- La portée de cette instruction de déclaration de package est le fichier entier

Packages

- On peut regrouper des classes et des interfaces dans des paquetages ou dossier (packages) pour éviter les conflits de noms et contrôler les accès
- Un package est une collection de classes ou d'interfaces ayant des liens entre elles qui fournit une protection d'accès et une gestion d'espace de noms
- Pour créer un package, il faut utiliser une instruction de package sur la première ligne du fichier source des classes ou interfaces définies dans le package

package MyPackage ;

- La portée de cette instruction de déclaration de package est le fichier entier

Packages

- On peut regrouper des classes et des interfaces dans des paquetages ou dossier (packages) pour éviter les conflits de noms et contrôler les accès
- Un package est une collection de classes ou d'interfaces ayant des liens entre elles qui fournit une protection d'accès et une gestion d'espace de noms
- Pour créer un package, il faut utiliser une instruction de package sur la première ligne du fichier source des classes ou interfaces définies dans le package

package MyPackage ;

- La portée de cette instruction de déclaration de package est le fichier entier

Utiliser les éléments d'un package

- Seuls les éléments définis publics dans **package** sont accessibles à l'extérieur du package
 - si pas de spécificateur : seuls les éléments du paquetage ont accès à cet élément
- Pour utiliser un élément public à l'extérieur du package, on peut :
 - y faire référence en utilisant le nom en extension (c'est-à-dire incluant le nom des packages qui le définissent)
 - importer l'élément du package
 - utiliser une instruction d'importation au début du fichier avant toute déclaration de classe ou d'interface, mais après l'instruction de package si c'est le cas
 - puis utiliser directement l'élément par son nom 'court' (sans chemin d'accès à travers les packages)
 - importer le package en entier
 - utiliser l'instruction d'importation avec l'astérisque (*)
- Quand on ne définit pas de package, les classes sont définies dans un package par défaut
- La plupart des classes de base sont définies dans le package `java.lang`
 - `java.util` , `java.io`, `java.math`, `javax.swing`, `java.awt`

Utiliser les éléments d'un package

- Seuls les éléments définis publics dans `package` sont accessibles à l'extérieur du package
 - si pas de spécificateur : seuls les éléments du paquetage ont accès à cet élément
- Pour utiliser un élément public à l'extérieur du package, on peut :
 - y faire référence en utilisant le nom en extension (c'est-à-dire incluant le nom des packages qui le définissent)
 - importer l'élément du package
 - utiliser une instruction d'importation au début du fichier avant toute déclaration de classe ou d'interface, mais après l'instruction de package si c'est le cas
 - puis utiliser directement l'élément par son nom 'court' (sans chemin d'accès à travers les packages)
 - importer le package en entier
 - utiliser l'instruction d'importation avec l'astérisque (*)
- Quand on ne définit pas de package, les classes sont définies dans un package par défaut
- La plupart des classes de base sont définies dans le package `java.lang`
 - `java.util` , `java.io`, `java.math`, `javax.swing`, `java.awt`

Utiliser les éléments d'un package

- Seuls les éléments définis publics dans `package` sont accessibles à l'extérieur du package
 - si pas de spécificateur : seuls les éléments du paquetage ont accès à cet élément
- Pour utiliser un élément public à l'extérieur du package, on peut :
 - y faire référence en utilisant le nom en extension (c'est-à-dire incluant le nom des packages qui le définissent)
 - importer l'élément du package
 - utiliser une instruction d'importation au début du fichier avant toute déclaration de classe ou d'interface, mais après l'instruction de package si c'est le cas
 - puis utiliser directement l'élément par son nom 'court' (sans chemin d'accès à travers les packages)
 - importer le package en entier
 - utiliser l'instruction d'importation avec l'astérisque (*)
- Quand on ne définit pas de package, les classes sont définies dans un package par défaut
- La plupart des classes de base sont définies dans le package `java.lang`
 - `java.util` , `java.io`, `java.math`, `javax.swing`, `java.awt`

Utiliser les éléments d'un package

- Seuls les éléments définis publics dans `package` sont accessibles à l'extérieur du package
 - si pas de spécificateur : seuls les éléments du paquetage ont accès à cet élément
- Pour utiliser un élément public à l'extérieur du package, on peut :
 - y faire référence en utilisant le nom en extension (c'est-à-dire incluant le nom des packages qui le définissent)
 - importer l'élément du package
 - utiliser une instruction d'importation au début du fichier avant toute déclaration de classe ou d'interface, mais après l'instruction de package si c'est le cas
 - puis utiliser directement l'élément par son nom 'court' (sans chemin d'accès à travers les packages)
 - importer le package en entier
 - utiliser l'instruction d'importation avec l'astérisque (*)
- Quand on ne définit pas de package, les classes sont définies dans un package par défaut
- La plupart des classes de base sont définies dans le package `java.lang`
 - `java.util` , `java.io`, `java.math`, `javax.swing`, `java.awt`

Gestion des "source" et "compilé"

- Le code d'une classe (ou interface) se trouve dans un fichier portant le nom de la classe (ou interface) et d'extension .java
- Ce fichier source doit être placé dans un répertoire de même nom que celui du package qui contient la classe (ou interface) ; la structure des répertoires doit refléter la structure des packages
- A la compilation, le compilateur crée autant de fichiers de sortie que de classes définies dans le package ; le nom du fichier est celui de la classe (ou interface) et son extension est .class
- Le nom d'un élément de package et son chemin d'accès ont la même structure :
 - nom de classe : `MonPackage.MaClasse`
 - chemin d'accès au fichier : `MonPackage / MaClass.java`
- Comme pour un fichier .java, un fichier .class file devra aussi être dans une série de répertoires qui reflète le nom du package

Gestion des "source" et "compilé"

- Le code d'une classe (ou interface) se trouve dans un fichier portant le nom de la classe (ou interface) et d'extension .java
- Ce fichier source doit être placé dans un répertoire de même nom que celui du package qui contient la classe (ou interface) ; la structure des répertoires doit refléter la structure des packages
- A la compilation, le compilateur crée autant de fichiers de sortie que de classes définies dans le package ; le nom du fichier est celui de la classe (ou interface) et son extension est .class
- Le nom d'un élément de package et son chemin d'accès ont la même structure :
 - nom de classe : `MonPackage.MaClasse`
 - chemin d'accès au fichier : `MonPackage / MaClass.java`
- Comme pour un fichier .java, un fichier .class file devra aussi être dans une série de répertoires qui reflète le nom du package

Gestion des "source" et "compilé"

- Le code d'une classe (ou interface) se trouve dans un fichier portant le nom de la classe (ou interface) et d'extension .java
- Ce fichier source doit être placé dans un répertoire de même nom que celui du package qui contient la classe (ou interface) ; la structure des répertoires doit refléter la structure des packages
- A la compilation, le compilateur crée autant de fichiers de sortie que de classes définies dans le package ; le nom du fichier est celui de la classe (ou interface) et son extension est .class
- Le nom d'un élément de package et son chemin d'accès ont la même structure :
 - nom de classe : `MonPackage.MaClasse`
 - chemin d'accès au fichier : `MonPackage / MaClass.java`
- Comme pour un fichier .java, un fichier .class file devra aussi être dans une série de répertoires qui reflète le nom du package

Gestion des "source" et "compilé"

- Le code d'une classe (ou interface) se trouve dans un fichier portant le nom de la classe (ou interface) et d'extension .java
- Ce fichier source doit être placé dans un répertoire de même nom que celui du package qui contient la classe (ou interface) ; la structure des répertoires doit refléter la structure des packages
- A la compilation, le compilateur crée autant de fichiers de sortie que de classes définies dans le package ; le nom du fichier est celui de la classe (ou interface) et son extension est .class
- Le nom d'un élément de package et son chemin d'accès ont la même structure :
 - nom de classe : `MonPackage.MaClasse`
 - chemin d'accès au fichier : `MonPackage / MaClass.java`
- Comme pour un fichier .java, un fichier .class file devra aussi être dans une série de répertoires qui reflète le nom du package

Gestion des "source" et "compilé"

- Le code d'une classe (ou interface) se trouve dans un fichier portant le nom de la classe (ou interface) et d'extension .java
- Ce fichier source doit être placé dans un répertoire de même nom que celui du package qui contient la classe (ou interface) ; la structure des répertoires doit refléter la structure des packages
- A la compilation, le compilateur crée autant de fichiers de sortie que de classes définies dans le package ; le nom du fichier est celui de la classe (ou interface) et son extension est .class
- Le nom d'un élément de package et son chemin d'accès ont la même structure :
 - nom de classe : `MonPackage.MaClasse`
 - chemin d'accès au fichier : `MonPackage / MaClass.java`
- Comme pour un fichier .java, un fichier .class file devra aussi être dans une série de répertoires qui reflète le nom du package

Exceptions

Renforcer la sécurité

```
public class TestException {  
    public static void main(java.lang.String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
    }  
}
```

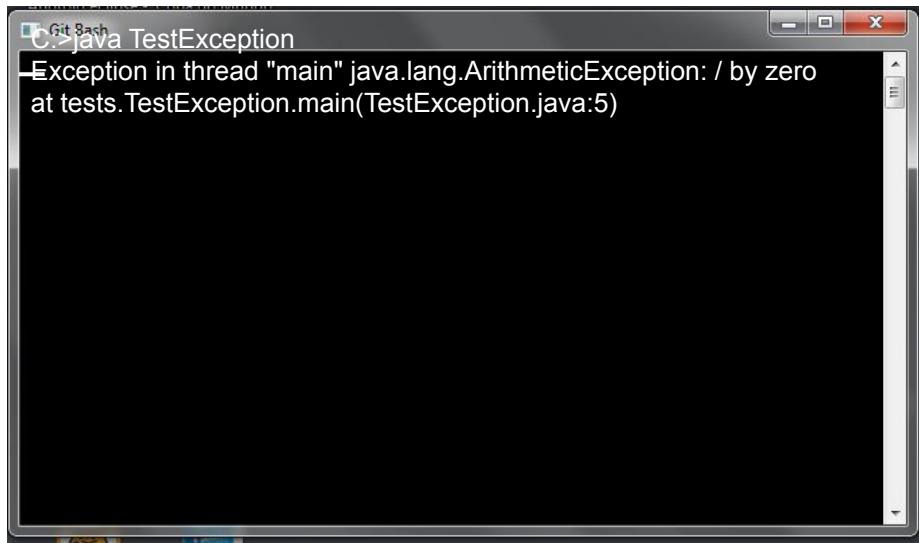
Renforcer la sécurité

```
public class TestException {  
    public static void main(java.lang.String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
    }  
}
```

Résultat ?

Renforcer la sécurité

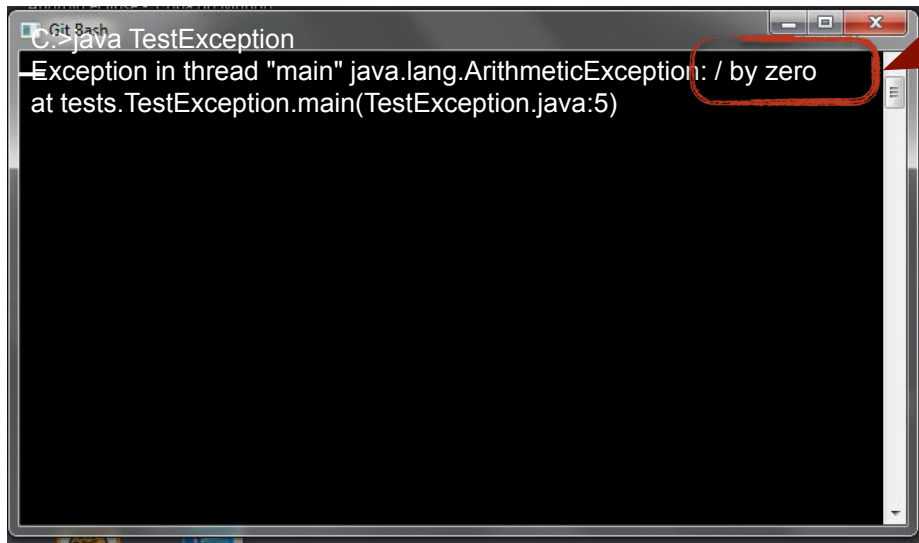
```
public class TestException {  
    public static void main(java.lang.String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
    }  
}
```

A screenshot of a Git Bash terminal window. The title bar shows 'Git Bash'. The command prompt is 'C:\>java TestException'. The output is an exception message: 'Exception in thread "main" java.lang.ArithmeticException: / by zero at tests.TestException.main(TestException.java:5)'. The terminal has a black background with white text. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
C:\>java TestException  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at tests.TestException.main(TestException.java:5)
```

Renforcer la sécurité

```
public class TestException {  
    public static void main(java.lang.String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
    }  
}
```



The screenshot shows a terminal window titled "Git Bash" with the command `C:\> java TestException` entered. The output displays an exception: `Exception in thread "main" java.lang.ArithmeticException: / by zero` followed by the stack trace `at tests.TestException.main(TestException.java:5)`. A red arrow points from the `(i / j)` expression in the code above to the `/ by zero` part of the exception message in the terminal.

Renforcer la sécurité

- Si dans un bloc de code on fait appel à une méthode qui peut potentiellement générer une exception, on doit soit essayer de la récupérer avec **try/catch**, soit ajouter le mot clé **throws** dans la déclaration du bloc.
- Si on ne le fait pas, il y a une erreur à la compilation. Les erreurs et exceptions du paquetage `java.lang` échappent à cette contrainte.
- Le mot réservé **throws** permet de déléguer la responsabilité des erreurs à la méthode appelante
- Ce procédé présente un inconvénient (**throws**): de nombreuses méthodes des packages java indiquent dans leur déclaration qu'elles peuvent lever une exception. Cependant ceci garantit que certaines exceptions critiques seront prises explicitement en compte par le programmeur.

Renforcer la sécurité

- Si dans un bloc de code on fait appel à une méthode qui peut potentiellement générer une exception, on doit soit essayer de la récupérer avec **try/catch**, soit ajouter le mot clé **throws** dans la déclaration du bloc.
- Si on ne le fait pas, il y a une erreur à la compilation. Les erreurs et exceptions du paquetage `java.lang` échappent à cette contrainte.
- Le mot réservé **throws** permet de déléguer la responsabilité des erreurs à la méthode appelante
- Ce procédé présente un inconvénient (**throws**): de nombreuses méthodes des packages java indiquent dans leur déclaration qu'elles peuvent lever une exception. Cependant ceci garantit que certaines exceptions critiques seront prises explicitement en compte par le programmeur.

Renforcer la sécurité

- Si dans un bloc de code on fait appel à une méthode qui peut potentiellement générer une exception, on doit soit essayer de la récupérer avec **try/catch**, soit ajouter le mot clé **throws** dans la déclaration du bloc.
- Si on ne le fait pas, il y a une erreur à la compilation. Les erreurs et exceptions du paquetage `java.lang` échappent à cette contrainte.
- Le mot réservé **throws** permet de déléguer la responsabilité des erreurs à la méthode appelante
- Ce procédé présente un inconvénient (**throws**): de nombreuses méthodes des packages java indiquent dans leur déclaration qu'elles peuvent lever une exception. Cependant ceci garantit que certaines exceptions critiques seront prises explicitement en compte par le programmeur.

Renforcer la sécurité

- Si dans un bloc de code on fait appel à une méthode qui peut potentiellement générer une exception, on doit soit essayer de la récupérer avec **try/catch**, soit ajouter le mot clé **throws** dans la déclaration du bloc.
- Si on ne le fait pas, il y a une erreur à la compilation. Les erreurs et exceptions du paquetage `java.lang` échappent à cette contrainte.
- Le mot réservé **throws** permet de déléguer la responsabilité des erreurs à la méthode appelante
- **Ce procédé présente un inconvénient (**throws**):** de nombreuses méthodes des packages java indiquent dans leur déclaration qu'elles peuvent lever une exception. Cependant ceci garantit que certaines exceptions critiques seront prises explicitement en compte par le programmeur.

Gestion des exceptions

- On utilise les exceptions pour gérer les erreurs en exécution
- Une exception est un événement se produisant en cours d'exécution de programme et qui interrompt le flot normal des instructions
- Lancer une exception consiste à créer un objet Exception et le transmettre à l'environnement d'exécution
- Un objet Exception contient des informations sur l'exception, comme son type et l'état du programme lorsque l'erreur s'est produite
- Avantages
 - séparation entre le code et la gestion des erreurs
 - propagation des erreurs dans la pile d'appel
 - regroupement des erreurs par types et différenciation des erreurs entre elles

Gestion des exceptions

- On utilise les exceptions pour gérer les erreurs en exécution
- Une exception est un événement se produisant en cours d'exécution de programme et qui interrompt le flot normal des instructions
- Lancer une exception consiste à créer un objet Exception et le transmettre à l'environnement d'exécution
- Un objet Exception contient des informations sur l'exception, comme son type et l'état du programme lorsque l'erreur s'est produite
- Avantages
 - séparation entre le code et la gestion des erreurs
 - propagation des erreurs dans la pile d'appel
 - regroupement des erreurs par types et différenciation des erreurs entre elles

Gestion des exceptions

- On utilise les exceptions pour gérer les erreurs en exécution
- Une exception est un événement se produisant en cours d'exécution de programme et qui interrompt le flot normal des instructions
- Lancer une exception consiste à créer un objet Exception et le transmettre à l'environnement d'exécution
- Un objet Exception contient des informations sur l'exception, comme son type et l'état du programme lorsque l'erreur s'est produite
- Avantages
 - séparation entre le code et la gestion des erreurs
 - propagation des erreurs dans la pile d'appel
 - regroupement des erreurs par types et différenciation des erreurs entre elles

Gestion des exceptions

- On utilise les exceptions pour gérer les erreurs en exécution
- Une exception est un événement se produisant en cours d'exécution de programme et qui interrompt le flot normal des instructions
- Lancer une exception consiste à créer un objet Exception et le transmettre à l'environnement d'exécution
- Un objet Exception contient des informations sur l'exception, comme son type et l'état du programme lorsque l'erreur s'est produite
- Avantages
 - séparation entre le code et la gestion des erreurs
 - propagation des erreurs dans la pile d'appel
 - regroupement des erreurs par types et différenciation des erreurs entre elles

Gestion des exceptions

- On utilise les exceptions pour gérer les erreurs en exécution
- Une exception est un événement se produisant en cours d'exécution de programme et qui interrompt le flot normal des instructions
- Lancer une exception consiste à créer un objet Exception et le transmettre à l'environnement d'exécution
- Un objet Exception contient des informations sur l'exception, comme son type et l'état du programme lorsque l'erreur s'est produite
- Avantages
 - séparation entre le code et la gestion des erreurs
 - propagation des erreurs dans la pile d'appel
 - regroupement des erreurs par types et différenciation des erreurs entre elles

Lancer une exception

```
class ConstErr extends Exception  
{ }
```


Lancer une exception

```
class ConstErr extends Exception  
{ }
```

Lancer une exception

```
class ConstErr extends Exception
{
    class Point{

        int x , y ;

        Point(int x, int y) throws ConstErr{
            if ((x<0) || (y<0)) throw new ConstErr();
            this.x=x;
            this.y=y;
        }

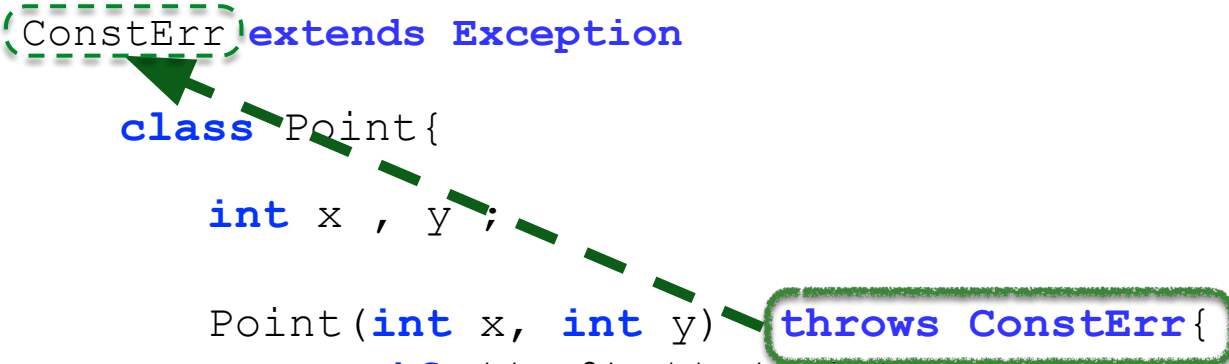
        void display(){
            System.out.println("x : "+x+" , y : "+y);
        }
    }
}
```

Lancer une exception

```
class ConstErr extends Exception
{
    class Point{
        int x , y ;

        Point(int x, int y) throws ConstErr{
            if ((x<0) || (y<0)) throw new ConstErr();
            this.x=x;
            this.y=y;
        }

        void display(){
            System.out.println("x : "+x+" , y : "+y);
        }
    }
}
```



Gérer une exception

```
class Exemple{  
    public static void main(String[] a){  
  
        try{  
            Point p=new Point (-3 , 5) ;  
        }  
  
        catch (ConstErr e){  
            System.out.println("Erreur à la construction");  
        }  
    }  
}
```

Gérer une exception

```
class Exemple{  
    public static void main(String[] a){  
  
        try{  
            Point p=new Point (-3 , 5) ;  
        }  
  
        catch (ConstErr e){  
            System.out.println("Erreur à la construction");  
        }  
    }  
}
```

Mécanisme

- Quand une exception est lancée dans un bloc **try**, l'exécution du bloc s'interrompt immédiatement
- L'environnement d'exécution recherche alors un gestionnaire dont le type correspond à l'exception lancée
- La recherche s'effectue en remontant la pile d'appels, à partir de la méthode dans laquelle l'exception s'est produite, jusqu'à trouver une méthode contenant un gestionnaire d'exception approprié
- Après l'exécution du bloc **catch**, l'exécution continue par l'instruction qui suit les gestionnaires d'exception puis pass dans tous les cas par le bloc **finally**.

Mécanisme

- Quand une exception est lancée dans un bloc **try**, l'exécution du bloc s'interrompt immédiatement
- L'environnement d'exécution recherche alors un gestionnaire dont le type correspond à l'exception lancée
- La recherche s'effectue en remontant la pile d'appels, à partir de la méthode dans laquelle l'exception s'est produite, jusqu'à trouver une méthode contenant un gestionnaire d'exception approprié
- Après l'exécution du bloc **catch**, l'exécution continue par l'instruction qui suit les gestionnaires d'exception puis pass dans tous les cas par le bloc **finally**.

Mécanisme

- Quand une exception est lancée dans un bloc **try**, l'exécution du bloc s'interrompt immédiatement
- L'environnement d'exécution recherche alors un gestionnaire dont le type correspond à l'exception lancée
- La recherche s'effectue en remontant la pile d'appels, à partir de la méthode dans laquelle l'exception s'est produite, jusqu'à trouver une méthode contenant un gestionnaire d'exception approprié
- Après l'exécution du bloc **catch**, l'exécution continue par l'instruction qui suit les gestionnaires d'exception puis pass dans tous les cas par le bloc **finally**.

Mécanisme

- Quand une exception est lancée dans un bloc **try**, l'exécution du bloc s'interrompt immédiatement
- L'environnement d'exécution recherche alors un gestionnaire dont le type correspond à l'exception lancée
- La recherche s'effectue en remontant la pile d'appels, à partir de la méthode dans laquelle l'exception s'est produite, jusqu'à trouver une méthode contenant un gestionnaire d'exception approprié
- Après l'exécution du bloc **catch**, l'exécution continue par l'instruction qui suit les gestionnaires d'exception puis pass dans tous les cas par le bloc **finally**.

Le bloc catch

- Un bloc **try** peut être associé à un ou plusieurs blocs **catch**
- Si on choisit de ne gérer pas une exception dans une méthode, il faut alors spécifier dans son prototype qu'elle peut lancer cette exception. Ceci concerne :
 - les exceptions lancées dans la méthode par une instruction **throw**
 - les exceptions lancées indirectement dans des méthodes appelées
- On utilisera l'héritage entre exceptions afin d'écrire des gestionnaires d'exception généraux qui traiteront plusieurs types d'exception

Le bloc catch

- Un bloc **try** peut être associé à un ou plusieurs blocs **catch**
- Si on choisit de ne gérer pas une exception dans une méthode, il faut alors spécifier dans son prototype qu'elle peut lancer cette exception. Ceci concerne :
 - les exceptions lancées dans la méthode par une instruction **throw**
 - les exceptions lancées indirectement dans des méthodes appelées
- On utilisera l'héritage entre exceptions afin d'écrire des gestionnaires d'exception généraux qui traiteront plusieurs types d'exception

Le bloc catch

- Un bloc **try** peut être associé à un ou plusieurs blocs **catch**
- Si on choisit de ne gérer pas une exception dans une méthode, il faut alors spécifier dans son prototype qu'elle peut lancer cette exception. Ceci concerne :
 - les exceptions lancées dans la méthode par une instruction **throw**
 - les exceptions lancées indirectement dans des méthodes appelées
- On utilisera l'héritage entre exceptions afin d'écrire des gestionnaires d'exception généraux qui traiteront plusieurs types d'exception

Le bloc finally

- C'est un mécanisme qui permet de “nettoyer” l'état d'une méthode avant de continuer l'exécution à la suite du programme
- Toujours associé à un bloc **try**
- Syntaxe :

```
try
{ ... }
catch (Exception e)           //optionnel si un bloc
finally est précisé
{ ... }
finally           //optionnel si un bloc catch est
précisé
{ ... }
```

Le bloc try auto-close

Java 7 a introduit l'interface **AutoCloseable** ainsi qu'une nouvelle syntaxe dénommée **try-with-resources** et les deux codes ci-dessous sont équivalents :

```
try (PrintWriter writer =  
    new PrintWriter(new File("test.txt"))){  
  
    writer.println("Hello World");  
}  
catch (Exception e) {  
    e.printStackTrace();  
}
```



```
try {  
    PrintWriter writer2 = new  
    PrintWriter(new File("test2.txt"));  
    try {  
        writer2.println("Hello World 2");  
    }  
    catch (Exception e) {  
        e.printStackTrace();  
    }  
    finally {  
        writer2.close();  
    }  
}  
catch (Exception e) {  
    e.printStackTrace();  
}
```

La classe `Exception`

- Elle définit un constructeur à un argument de type `String`
- Lorsqu'on définit ses propres classes d'exception, on peut déclarer un constructeur à un argument de type `String` : il sera utilisé pour transmettre un message informant de la nature du problème qui s'est produit
- La méthode `getMessage` retourne la chaîne qui a été renseignée lors de la construction de l'exception

La classe `Exception`

- Elle définit un constructeur à un argument de type `String`
- Lorsqu'on définit ses propres classes d'exception, on peut déclarer un constructeur à un argument de type `String` : il sera utilisé pour transmettre un message informant de la nature du problème qui s'est produit
- La méthode `getMessage` retourne la chaîne qui a été renseignée lors de la construction de l'exception

La classe `Exception`

- Elle définit un constructeur à un argument de type `String`
- Lorsqu'on définit ses propres classes d'exception, on peut déclarer un constructeur à un argument de type `String` : il sera utilisé pour transmettre un message informant de la nature du problème qui s'est produit
- La méthode `getMessage` retourne la chaîne qui a été renseignée lors de la construction de l'exception

Utiliser les méthodes de la base

```
class ConstErr extends Exception{  
    public ConstErr(String s){  
        super(s);  
    }  
}
```

On peut alors utiliser la fonction getMessage()

Si e est un objet ConstErr, e.getMessage() retourne un objet String

→ Exemple : System.out.println(e.getMessage());

Utiliser les méthodes de la base

```
class ConstErr extends Exception{  
    public ConstErr(String s){  
        super(s);  
    }  
}
```



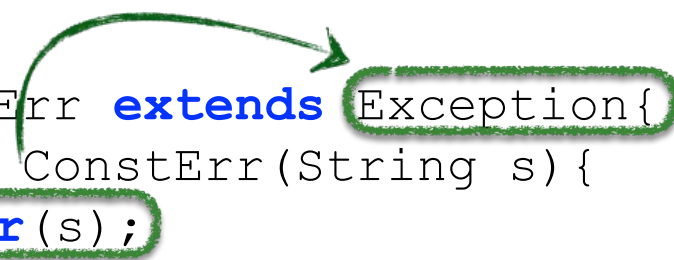
On peut alors utiliser la fonction getMessage()

Si e est un objet ConstErr, e.getMessage() retourne un objet String

→ Exemple : System.out.println(e.getMessage());

Utiliser les méthodes de la base

```
class ConstErr extends Exception{  
    public ConstErr(String s){  
        super(s);  
    }  
}
```

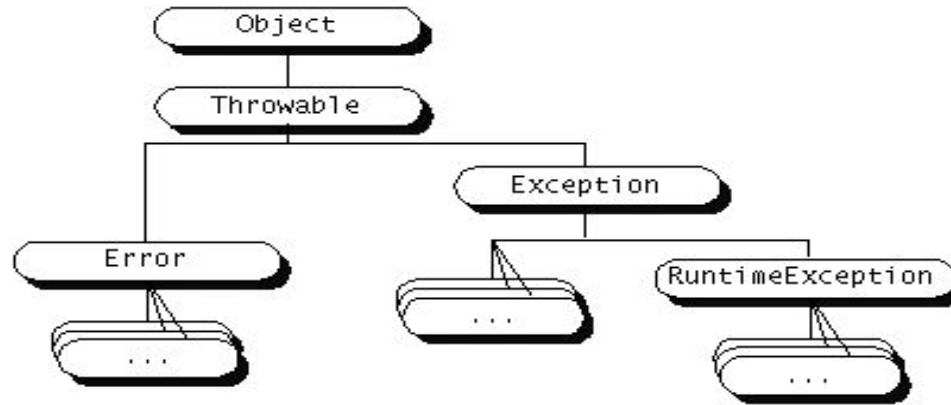
A green curved arrow points from the `Exception` class in the `extends` clause to the `super(s);` call inside the `ConstErr` constructor, illustrating the call to the superclass constructor.

On peut alors utiliser la fonction `getMessage()`

Si `e` est un objet `ConstErr`, `e.getMessage()` retourne un objet `String`

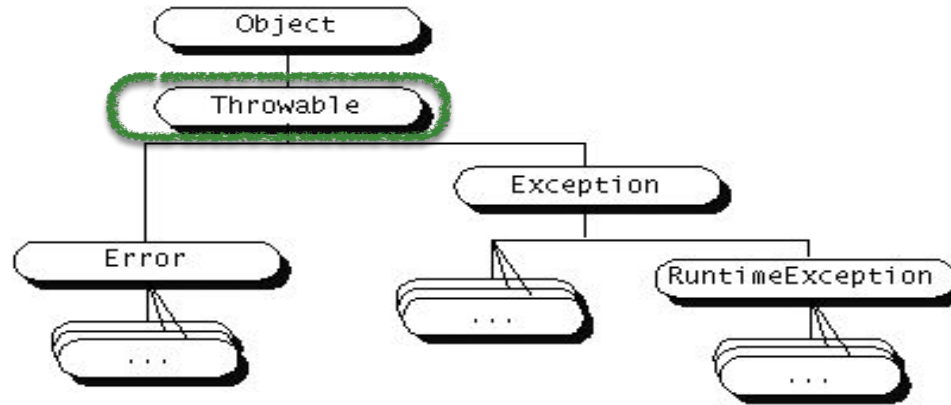
→ Exemple : `System.out.println(e.getMessage());`

La hiérarchie de classes



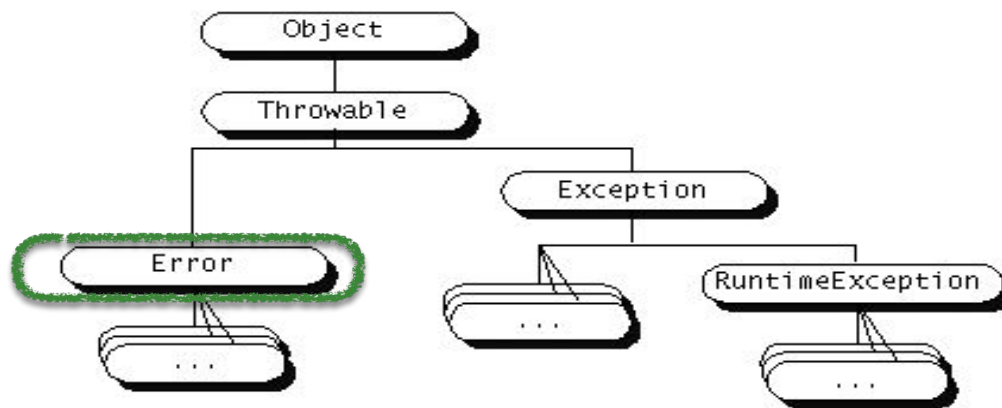
- Les exceptions qu'une méthode peut lancer font partie de son prototype, au même titre que ses paramètres et son résultat

La hiérarchie de classes



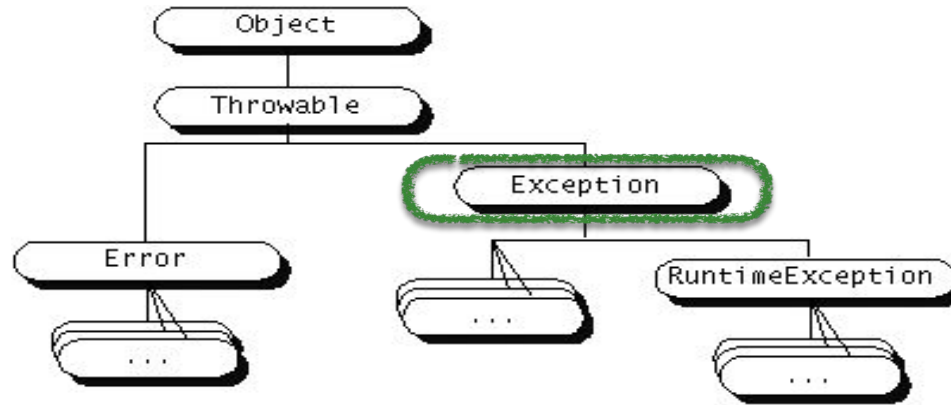
- **Throwable** descend directement de la classe `Object` : c'est la classe de base pour le traitement des erreurs.

La hiérarchie de classes



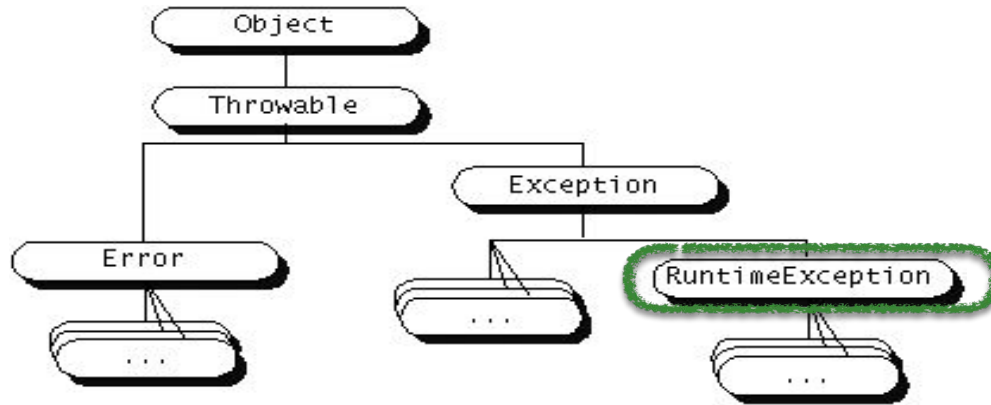
- **Error** relate un problème "hard" *failure* dans la machine virtuelle. L'application Java s'arrête instantanément dès l'apparition d'une exception de la classe `Error`.

La hiérarchie de classes



- La classe **Exception** représente des erreurs moins graves

La hiérarchie de classes



- **RuntimeException** : le compilateur accepte que les exceptions runtime ne soient ni spécifiées ni gérées. Généralement, les exceptions `RuntimeException` peuvent être évitées, comme `NullPointerException` ou `ArrayIndexOutOfBoundsException`.
- Si l'on vérifie la valeur `null` avant d'appeler une méthode, `NullPointerException` ne se produira jamais par exemple.

Exception personnalisée

- Pour générer une exception, il suffit d'utiliser le mot clé **throw**, suivi d'un objet dont la classe dérive de `Throwable`.
- Si l'on veut générer une exception dans une méthode avec **throw**, il faut l'indiquer dans la déclaration de la méthode, en utilisant le mot clé **throws**.
- En cas de nécessité, on peut créer ses propres exceptions. Elles descendent des classes `Exception` ou `RuntimeException` mais pas de la classe `Error`. Il est préférable (par convention) d'inclure le mot « Exception » dans le nom de la nouvelle classe.

Exception personnalisée

- Pour générer une exception, il suffit d'utiliser le mot clé **throw**, suivi d'un objet dont la classe dérive de `Throwable`.
- Si l'on veut générer une exception dans une méthode avec **throw**, il faut l'indiquer dans la déclaration de la méthode, en utilisant le mot clé **throws**.
- En cas de nécessité, on peut créer ses propres exceptions. Elles descendent des classes `Exception` ou `RuntimeException` mais pas de la classe `Error`. Il est préférable (par convention) d'inclure le mot « Exception » dans le nom de la nouvelle classe.

Exception personnalisée

- Pour générer une exception, il suffit d'utiliser le mot clé **throw**, suivi d'un objet dont la classe dérive de `Throwable`.
- Si l'on veut générer une exception dans une méthode avec **throw**, il faut l'indiquer dans la déclaration de la méthode, en utilisant le mot clé **throws**.
- En cas de nécessité, on peut créer ses propres exceptions. Elles descendent des classes `Exception` ou `RuntimeException` mais pas de la classe `Error`. Il est préférable (par convention) d'inclure le mot « Exception » dans le nom de la nouvelle classe.

Exception personnalisée

```
public class SaisieErroneeException extends Exception {  
    public SaisieErroneeException() {super();}  
    public SaisieErroneeException(String s) {super(s);}  
}
```

Exception personnalisée

```
public class SaisieErroneeException extends Exception {  
    public SaisieErroneeException() {super();}  
    public SaisieErroneeException(String s) {super(s);}  
}
```

Constructeur sans argument

Exception personnalisée

```
public class SaisieErroneeException extends Exception {  
    public SaisieErroneeException() {super();}  
    public SaisieErroneeException(String s) {super(s);}  
}
```

Constructeur avec argument

Exception personnalisée

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {super();}
    public SaisieErroneeException(String s) {super(s);}
}

public class TestSaisieErroneeException {
    public static void controle(String chaine) throws SaisieErroneeException {
        if (chaine.equals("") == true)
            throw new SaisieErroneeException("Saisie erronee : chaine vide »);
        else
            System.out.println("Chaine " + chaine);
    }

    public static void main(java.lang.String[] args) {
        String chaine1 = "bonjour";
        String chaine2 = "";
        try {
            controle(chaine1);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine1 saisie erronee");
        }
        try {
            controle(chaine2);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine2 saisie erronee");
        }
    }
}
```


Exception personnalisée

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {super();}
    public SaisieErroneeException(String s) {super(s);}
}

public class TestSaisieErroneeException {
    public static void controle(String chaine) throws SaisieErroneeException {
        if (chaine.equals("") == true)
            throw new SaisieErroneeException("Saisie erronee : chaine vide »);
        else
            System.out.println("Chaine " + chaine);
    }

    public static void main(java.lang.String[] args) {
        String chaine1 = "bonjour";
        String chaine2 = "";
        try {
            controle(chaine1);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine1 saisie erronee");
        }
        try {
            controle(chaine2);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine2 saisie erronee");
        }
    }
}
```



Exception personnalisée

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {super();}
    public SaisieErroneeException(String s) {super(s);}
}

public class TestSaisieErroneeException {
    public static void controle(String chaine) throws SaisieErroneeException {
        if (chaine.equals("")) == true
            throw new SaisieErroneeException("Saisie erronee : chaine vide »");
        else
            System.out.println("Chaine " + chaine);
    }

    public static void main(java.lang.String[] args) {
        String chaine1 = "bonjour";
        String chaine2 = "";
        try {
            controle(chaine1);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine1 saisie erronee");
        }
        try {
            controle(chaine2);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine2 saisie erronee");
        }
    }
}
```

Pas d'exception levée

Exception personnalisée

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {super();}
    public SaisieErroneeException(String s) {super(s);}
}

public class TestSaisieErroneeException {
    public static void controle(String chaine) throws SaisieErroneeException {
        if (chaine.equals("") == true)
            throw new SaisieErroneeException("Saisie erronee : chaine vide »");
        else
            System.out.println("Chaine " + chaine);
    }

    public static void main(java.lang.String[] args) {
        String chaine1 = "bonjour";
        String chaine2 = "";
        try {
            controle(chaine1);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine1 saisie erronee");
        }
        try {
            controle(chaine2);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine2 saisie erronee");
        }
    }
}
```



Exception personnalisée

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {super();}
    public SaisieErroneeException(String s) {super(s);}
}

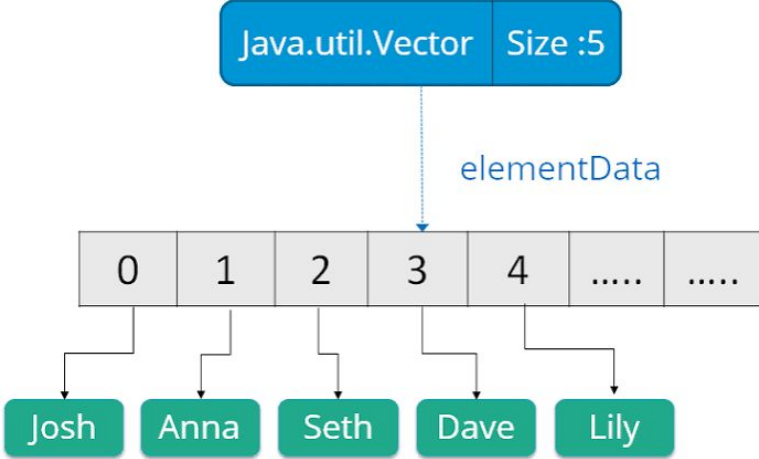
public class TestSaisieErroneeException {
    public static void controle(String chaine) throws SaisieErroneeException {
        if (chaine.equals("") == true)
            throw new SaisieErroneeException("Saisie erronee : chaine vide »);
        else
            System.out.println("Chaine " + chaine);
    }

    public static void main(java.lang.String[] args) {
        String chaine1 = "bonjour";
        String chaine2 = "";
        try {
            controle(chaine1);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine1 saisie erronee");
        }
        try {
            controle(chaine2);
        }
        catch (SaisieErroneeException e) {
            System.out.println("Chaine2 saisie erronee");
        }
    }
}
```

Exception levée de type
SaisieErroneeException

Exception personnalisée

- Les méthodes pouvant lever des exceptions doivent inclure une clause **throws** **nom_exception** dans leur en-tête.
- L'objectif est double : avoir une valeur documentaire et préciser au compilateur que cette méthode pourra lever cette exception et que toute méthode qui l'appelle devra prendre en compte cette exception (traitement ou propagation).
- Si la méthode appelante ne traite pas l'erreur ou ne la propage pas, le compilateur génère l'exception : `nom_exception must be caught or it must be declared in the throws clause of this method`.
- Java n'oblige à déclarer les exceptions dans l'en-tête de la méthode que pour les exceptions dites contrôlées (checked). Les exceptions non contrôlées (unchecked) peuvent être capturées mais n'ont pas à être déclarées. Les exceptions et erreurs qui héritent de `RuntimeException` et de `Error` sont non contrôlées. Toutes les autres exceptions sont contrôlées.



Java I/O (Input/Output Streams)



Reading....

Writing...

Collections et entrées-sorties

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Package et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Collections

- Une collection (parfois appelée conteneur) est un objet qui regroupe plusieurs éléments en une seule unité
 - on l'utilise pour stocker, récupérer et gérer des données

Collections

- Une collection (parfois appelée conteneur) est un objet qui regroupe plusieurs éléments en une seule unité
 - on l'utilise pour stocker, récupérer et gérer des données
- Le package `java.util` contient le framework des collections, les classes collections, ainsi que d'autres classes utilitaires dont celles permettant d'utiliser les dates et l'heure ou l'internationalisation

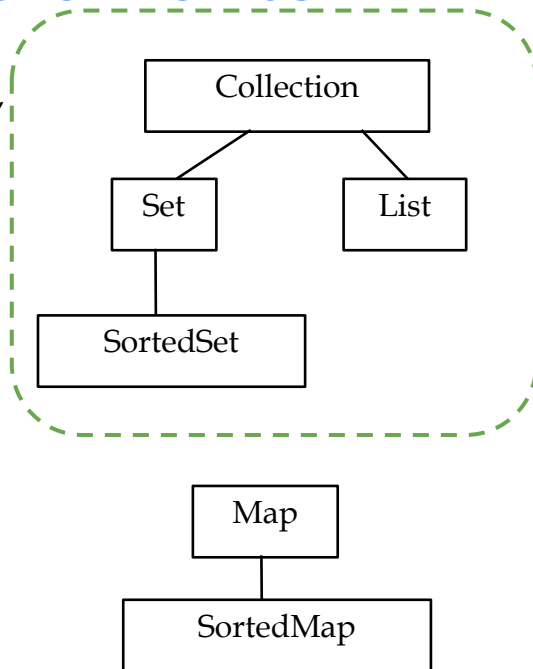
Collections

- Une collection (parfois appelée conteneur) est un objet qui regroupe plusieurs éléments en une seule unité
 - on l'utilise pour stocker, récupérer et gérer des données
- Le package `java.util` contient le framework des collections, les classes collections, ainsi que d'autres classes utilitaires dont celles permettant d'utiliser les dates et l'heure ou l'internationalisation
- Le framework des collections Java est une architecture unifiée de description et de manipulation de collections. Il contient :
 - des interfaces : on peut ainsi manipuler les collections indépendamment des détails de leur implémentation
 - des implémentations, qui sont des structures de données réutilisables
 - des algorithmes : des méthodes qui effectuent les traitements habituels (comme la recherche ou le tri) sur des instances des classes implémentant les interfaces collection (généralement des méthodes statiques de la classe `Collection`)

<https://docs.oracle.com/javase/8/docs/api/overview-summary.html>

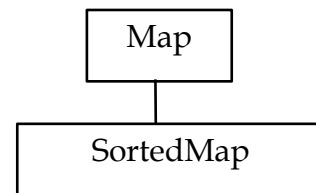
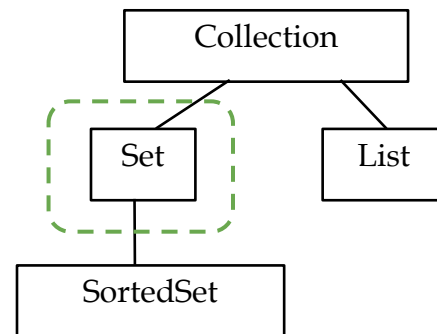
Interfaces de conteneur d'éléments

- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non



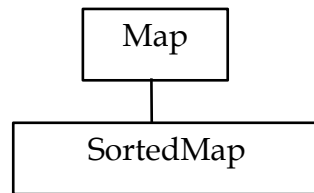
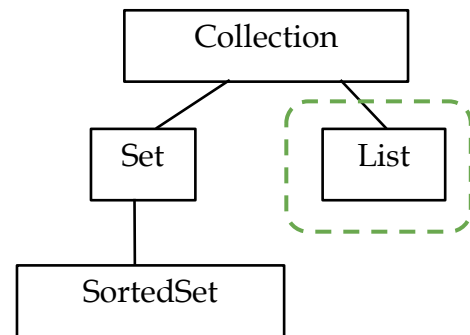
Interfaces de conteneur d'éléments

- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non
- Un **Set** (ensemble) est une collection qui ne contient pas d'élément dupliqué
- Une **List** (parfois appelée séquence) est une collection ordonnée ; elle peut contenir des éléments dupliqués
- Une **Map** est un objet qui associe des clés à des valeurs, elle ne peut pas avoir de clé dupliquée
- Un **SortedSet** est un **Set** qui assure un ordre ascendant de ses éléments
- Une **SortedMap** est une **Map** qui assure un ordre ascendant de ses clés



Interfaces de conteneur d'éléments

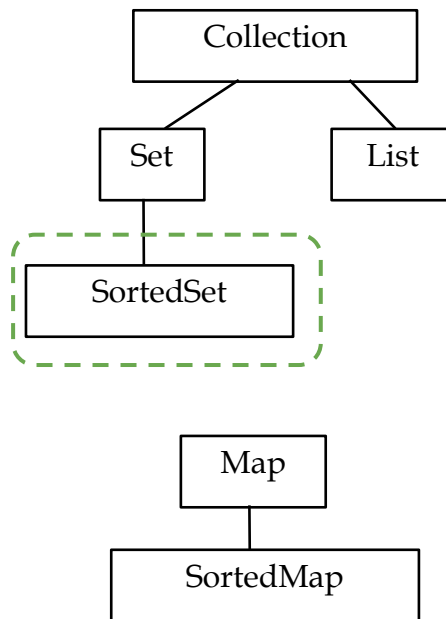
- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non
- Un **Set** (ensemble) est une collection qui ne contient pas d'élément dupliqué
- Une **List** (parfois appelée séquence) est une collection ordonnée ; elle peut contenir des éléments dupliqués



- Une **Map** est un objet qui associe des clés à des valeurs, elle ne peut pas avoir de clé dupliquée
- Un **SortedSet** est un **Set** qui assure un ordre ascendant de ses éléments
- Une **SortedMap** est une **Map** qui assure un ordre ascendant de ses clés

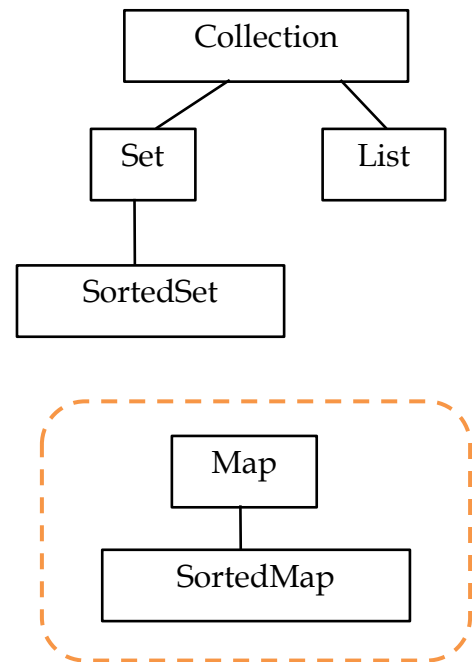
Interfaces de conteneur d'éléments

- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non
- Un **Set** (ensemble) est une a collection qui ne contient pas d'élément dupliqué
- Une **List** (parfois appelée séquence) est une collection ordonnée ; elle peut contenir des éléments dupliqués
- Un **SortedSet** est un **Set** qui assure un ordre ascendant de ses éléments



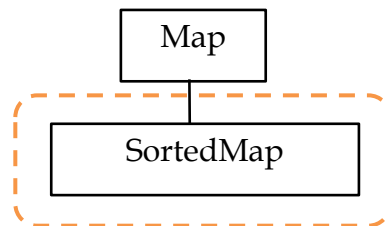
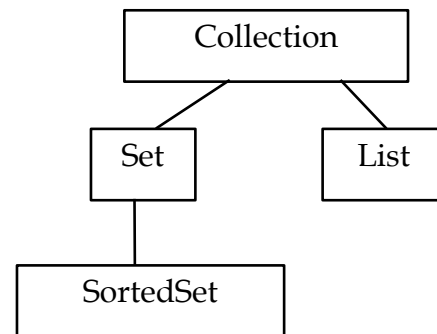
Interfaces de conteneur d'éléments

- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non
- Un **Set** (ensemble) est une a collection qui ne contient pas d'élément dupliqué
- Une **List** (parfois appelée séquence) est une collection ordonnée ; elle peut contenir des éléments dupliqués
- Un **SortedSet** est un **Set** qui assure un ordre ascendant de ses éléments
- Une **Map** est un objet qui associe des clés à des valeurs; elle ne peut pas avoir de clé dupliquée
- Une **SortedMap** est une **Map** qui assure un ordre ascendant de ses clés



Interfaces de conteneur d'éléments

- Une **Collection** représente un groupe d'objets. Certaines implémentations d'une collection dupliquent les éléments, d'autres non. Certaines sont ordonnées, d'autres non
- Un **Set** (ensemble) est une a collection qui ne contient pas d'élément dupliqué
- Une **List** (parfois appelée séquence) est une collection ordonnée ; elle peut contenir des éléments dupliqués
- Un **SortedSet** est un **Set** qui assure un ordre ascendant de ses éléments
- Une **Map** est un objet qui associe des clés à des valeurs; elle ne peut pas avoir de clé dupliquée
- Une **SortedMap** est une **Map** qui assure un ordre ascendant de ses clés



L'interface Collection

```
public interface Collection{

    // Opérations de base
    int size();
    boolean isEmpty();
    boolean contains(Object element);
    boolean add(Object element);
    boolean remove(Object element);
    Iterator iterator();

    // Opérations de masse
    boolean containsAll(Collection c);
    boolean addAll(Collection c);
    boolean removeAll(Collection c);
    boolean retainAll(Collection c);
    void clear();

    // Opérations de tableaux
    Object[] toArray();
    Object[] toArray(Object a[]);
}
```


Implémentations

- Implémentations de Set :
 - HashSet : plus rapide
 - TreeSet : l'ordre est garanti

Implémentations

- Implémentations de Set :
 - HashSet : plus rapide
 - TreeSet : l'ordre est garanti
- Implémentations de List :
 - ArrayList : tableau redimensionnable
 - LinkedList : alloue un nœud par élément
 - Vector

Implémentations

- Implémentations de Set :
 - HashSet : plus rapide
 - TreeSet : l'ordre est garanti
- Implémentations de List :
 - ArrayList : tableau redimensionnable
 - LinkedList : alloue un nœud par élément
 - Vector
- Map implementations :
 - HashMap
 - TreeMap

Itérateurs

- Fournissent des méthodes pour parcourir une collection

```
public interface Iterator{  
    boolean hasNext();  
    Object next();  
    void remove();  
}
```

Tri

- On pourra trier une liste `l` en invoquant la fonction statique:
`Collections.sort(l);`
- Les éléments de la liste devront être des instances d'une classe qui implémente l'interface `Comparable` qui ne contient qu'une seule méthode :

```
public interface Comparable{  
    public int compareTo(Object o);  
}
```

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre
 - `firstElement` : retourne le premier élément (celui à l'index 0)

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre
 - `firstElement` : retourne le premier élément (celui à l'index 0)
 - `lastElement` : retourne le dernier élément

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre
 - `firstElement` : retourne le premier élément (celui à l'index 0)
 - `lastElement` : retourne le dernier élément
 - `removeElementAt` : supprime l'élément à l'index spécifié en paramètre

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre
 - `firstElement` : retourne le premier élément (celui à l'index 0)
 - `lastElement` : retourne le dernier élément
 - `removeElementAt` : supprime l'élément à l'index spécifié en paramètre
 - `add(pos, element)` : ajoute element à la position pos si pos est défini, sinon add augmentant la taille de un et ajoute element à la fin

La classe ArrayList

- Implémente un tableau redimensionnable d'objets
- Méthodes :
 - `capacity` : pour connaître la capacité actuelle
 - `size` : pour connaître le nombre d'éléments
 - `get` : retourne l'élément à l'index spécifié en paramètre
 - `firstElement` : retourne le premier élément (celui à l'index 0)
 - `lastElement` : retourne le dernier élément
 - `removeElementAt` : supprime l'élément à l'index spécifié en paramètre
 - `add(pos, element)` : ajoute element à la position pos si pos est défini, sinon add augmentant la taille de un et ajoute element à la fin
 - `toString` : retourne une chaîne de caractères (String) représentant le ArrayList, contenant la représentation String de chaque élément

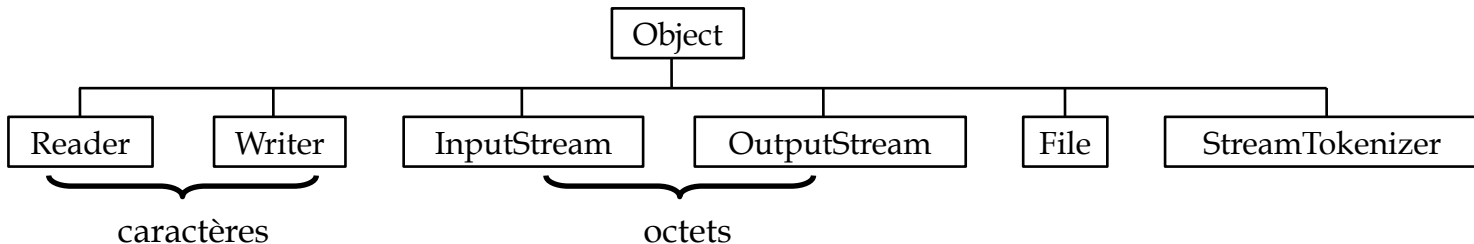
Flow d'entrées-sorties

Les flots (streams)

- Pour obtenir de l'information, un programme ouvre un flot sur une source d'information et lit l'information séquentiellement
- De façon similaire, un programme peut envoyer des informations sur une destination externe en ouvrant un flot
- Le cycle d'utilisation d'un flot est le suivant :
 - ouverture du flot
 - tant que cela est possible, lecture (ou écriture) des données
 - fermeture du flot
- Un flot peut être utilisé pour lire ou écrire à partir d'un fichier, de la mémoire, d'un socket, d'un pipe (canal entre 2 threads)

Le package `java.io`

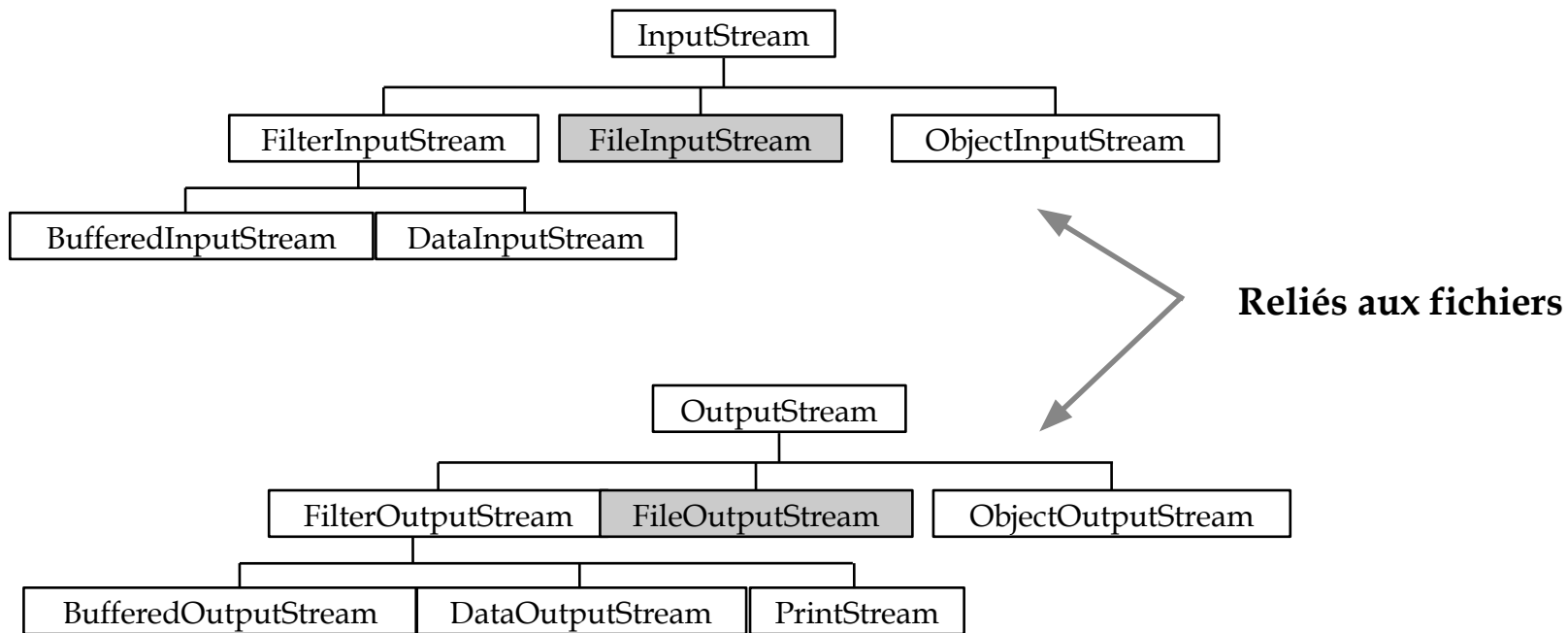
- Plus de 50 classes
- Deux sortes de flots : octets (`bytes`) et caractères (`characters`)
- Les classes de base fournissent les méthodes de lecture et écriture
- Pour les deux sortes de flots, des classes additionnelles proposent les possibilités :
 - d'accéder à la source de données
 - de filtrer l'information



Les superclasses d'entrées-sorties

- Ce sont des classes abstraites
- `Reader` et `InputStream` définissent des APIs similaires
 - deux méthodes `read` : pour lire un ou plusieurs caractères ou octets
 - une méthode de fermeture `close`
- `Writer` et `OutputStream` sont définies parallèlement
 - deux méthodes `write` : pour écrire un ou plusieurs caractères ou octets
 - une méthode de fermeture `close`

Les flots de bytes



Exemples

```
FileOutputStream fos = new FileOutputStream("MyFile.txt");  
fos.write('a');
```

```
BufferedOutputStream bos = new BufferedOutputStream(fos);  
// permet de réduire le nombre d'accès à la source de données :  
// plus efficace
```

```
DataOutputStream dos = new DataOutputStream(bos);  
// pour écrire des données de types primitifs
```

```
dos.writeInt(2); // écriture de la représentation interne (4  
octets)
```

```
// fermeture des flux
```

```
dos.close();    bos.close();    fos.close();
```

La classe `PrintStream`

- La variable de classe (static) `System.out` est un `PrintStream`
- Cette classe fournit des méthodes simples d'affichage

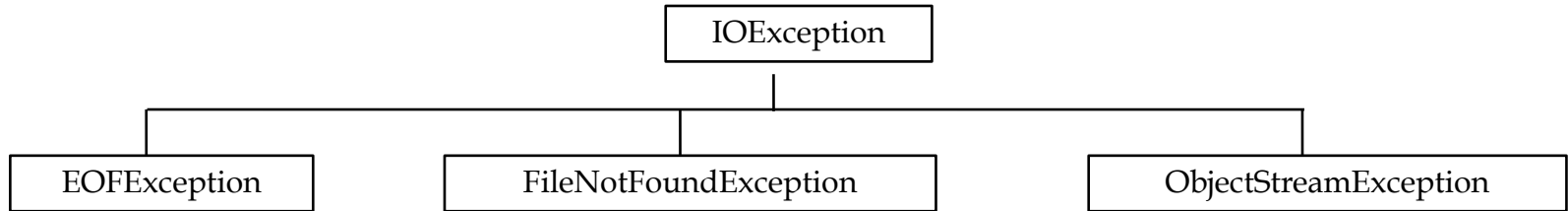
- Exemple :

```
PrintStream p=new PrintStream(new FileOutputStream("MyFile"));  
p.print(12.6);
```

- Un objet de classe `PrintStream` ne lance jamais d'`IOException`.
Les situations exceptionnelles peuvent être testées en utilisant la méthode `checkError` pour vérifier l'état de l'erreur

Exceptions

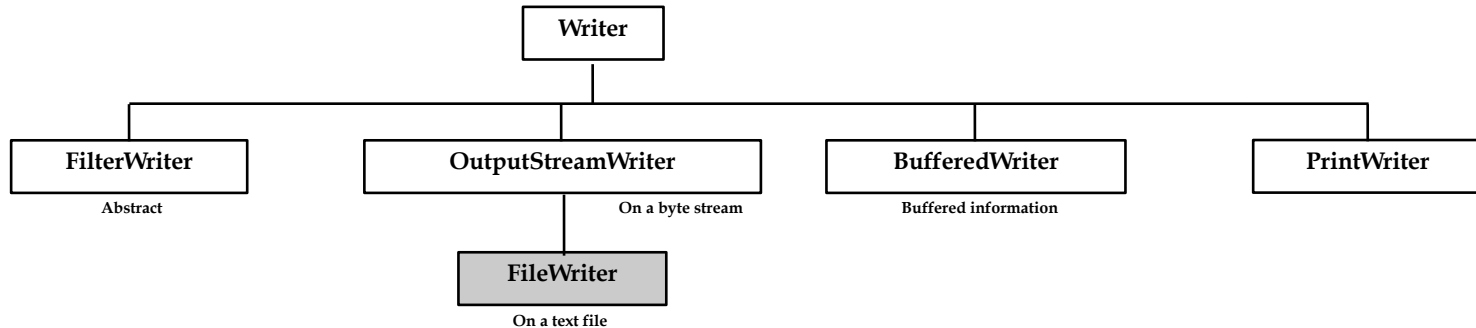
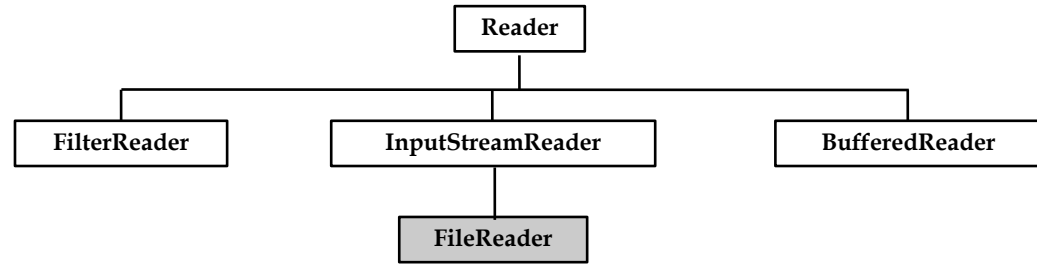
- Toute méthode travaillant avec les entrées-sorties doit soit gérer les exceptions soit propager l'exception dans sa signature
- Le bloc **finally** peut être utilisé pour fermer les flots



Les flots de caractères

En Java, les caractères sont codés en Unicode (2 octets).

Ils sont convertis de/en un caractère d'un octet avant/après l'accès au fichier



Conversions entre flots

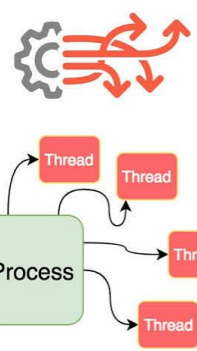
- Deux classes fournissent des moyens de conversion entre bytes et caractères
- Un `InputStreamReader` lit des octets (bytes) à partir d'un `InputStream` et les convertit en caractères
- Un `OutputStreamWriter` convertit des caractères en bytes et les écrit sur un `OutputStream`

```
byte b=0;      char c=0; int i=0; long l=0;

float f=0; double d=0;
BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
try{
    b=Byte.parseByte(in.readLine());
    i=Integer.parseInt(in.readLine());
    l=Long.parseLong(in.readLine());
    f=Float.parseFloat(in.readLine());
    d=Double.parseDouble(in.readLine());
}
catch (IOException e) {}
```

Divers

- Un `StreamTokenizer` lit des “tokens” sur un flot d’entrée, un à la fois
- La classe `File` propose des fonctionnalités pour obtenir des informations sur le système de fichier
 - vérifier si un fichier est de type ordinaire ou répertoire, lister les fichiers du répertoire courant, effacer, créer ou renommer un fichier, ...
- La classe `RandomAccessFile` permet un accès non-séquentiel ou aléatoire au contenu d’un fichier en utilisant la méthode `seek` qui positionne le pointeur de fichier sur un octet spécifié



Sérialisation, Base de données, Multi-processus et Sockets

Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation, Base de données, Thread et Socket	377	419
4	Evaluation		

Sérialisation

- On nomme sérialisation la sauvegarde et la relecture d'objets
 - l'état d'un objet (ses variables d'instance uniquement) est converti en une forme sérialisée (octets) de façon à le reconstruire plus tard
- Les deux flots `ObjectInputStream` et `ObjectOutputStream` sont des flots d'octets et peuvent lire et écrire des objets

- Ecrire dans un `ObjectOutputStream`

```
ObjectOutputStream s =  
new ObjectOutputStream(new FileOutputStream("MyFile"));  
s.writeObject(unObject);
```

- Lire à partir d'un `ObjectInputStream`

```
ObjectInputStream s =  
new ObjectInputStream(new FileInputStream("MyFile"));  
aClass anotherObject = (aClass)s.readObject();
```

- Un objet est sérialisable si sa classe implémente l'interface `Serializable` qui est une interface vide

Base de données relationnelles avec JDBC

Base de données relationnelles avec JDBC

- JDBC (Java DataBase Connectivity) est l'API permettant d'interagir avec les bases données relationnelles en Java.

Base de données relationnelles avec JDBC

- JDBC (Java DataBase Connectivity) est l'API permettant d'interagir avec les bases données relationnelles en Java.
- JDBC est donc directement dans le JDK.

Base de données relationnelles avec JDBC

- JDBC (Java DataBase Connectivity) est l'API permettant d'interagir avec les bases données relationnelles en Java.
- JDBC est donc directement dans le JDK.
- Les ressources de base de données doivent impérativement être fermées par l'appel à `close()`. `try` (`java.sql.Connection connection = dataSource.getConnection()`) {...}

Base de données relationnelles avec JDBC

- JDBC (Java DataBase Connectivity) est l'API permettant d'interagir avec les bases données relationnelles en Java.
- JDBC est donc directement dans le JDK.
- Les ressources de base de données doivent impérativement être fermées par l'appel à `close()`. `try (java.sql.Connection connection = dataSource.getConnection()) {...}`
- JDBC est une API indépendante de la base de données : d'un côté, les interactions avec une base de données à partir de cette API et de l'autre côté un pilote JDBC (JDBC driver comme Oracle DB, MySQL, PostgreSQL, Apache Derby, SQLServer ou SQLite)

Base de données relationnelles avec JDBC

```
public static Connection connectR() {  
    try {  
        String dataBase = "ams";  
        String port = "5454";  
        String address = "localhost";  
  
        String url = "jdbc:postgresql://" + address + ":" + port  
+ "/" + dataBase;  
  
        Properties props = new Properties();  
        props.setProperty("user", "john");  
        props.setProperty("password", "doe");  
        connection = DriverManager.getConnection(url, props);  
    }  
    catch (Exception e) {}  
    return connection ;  
}
```


Base de données relationnelles avec JDBC

```
public static void executeCreate(String query) {  
    try {  
        Statement stmt = connection.createStatement();  
        System.out.println("Query: " + query + " execution  
...");  
        stmt.executeUpdate(query);  
        stmt.close();  
    }  
    catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

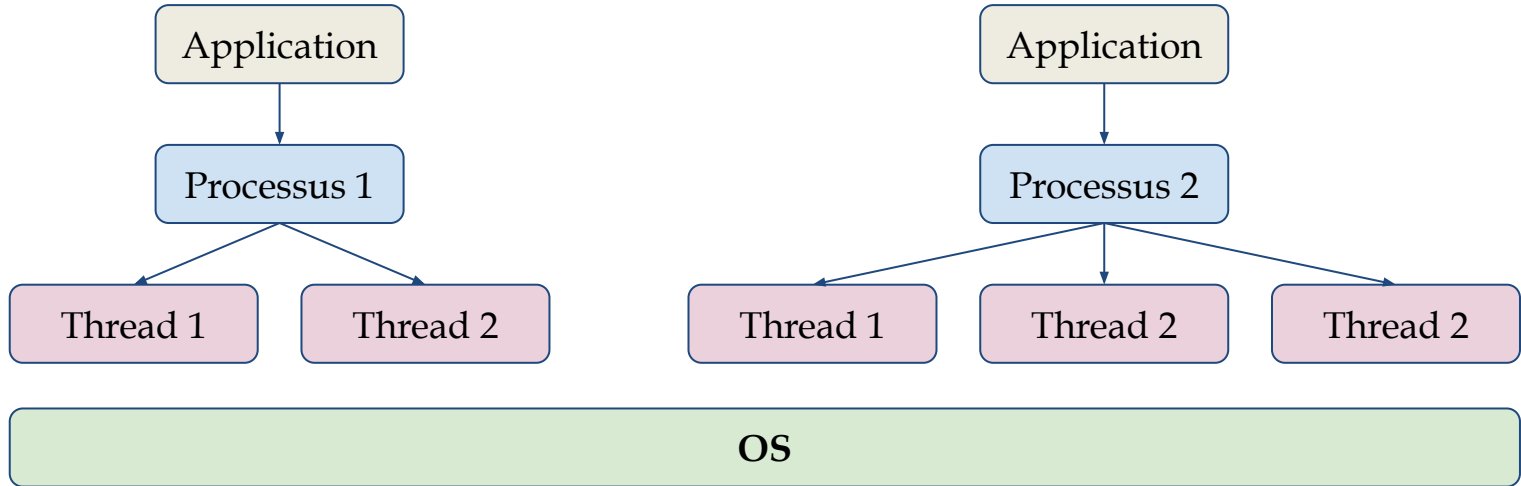
Base de données relationnelles avec JDBC

```
public static int add(Product product) {  
    String sql = "INSERT INTO product(name, price) VALUES(?,?)";  
    try {  
        Connection conn = DB.connectR();  
        PreparedStatement pstmt = conn.prepareStatement(sql,  
                                                         Statement.RETURN_GENERATED_KEYS)  
        pstmt.setString(1, product.getName());  
        pstmt.setDouble(2, product.getPrice());  
        int insertedRow = pstmt.executeUpdate();  
        if (insertedRow > 0) {  
            ResultSet rs = pstmt.getGeneratedKeys();  
            if (rs.next()) {  
                return rs.getInt(1);  
            }  
        }  
    }  
    catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Multi-processus : Threads

Les Threads - Définition

Java permet de d'exécuter plusieurs blocs d'instruction à la fois appelé Thread (tâche).



Les Threads - Classe Thread

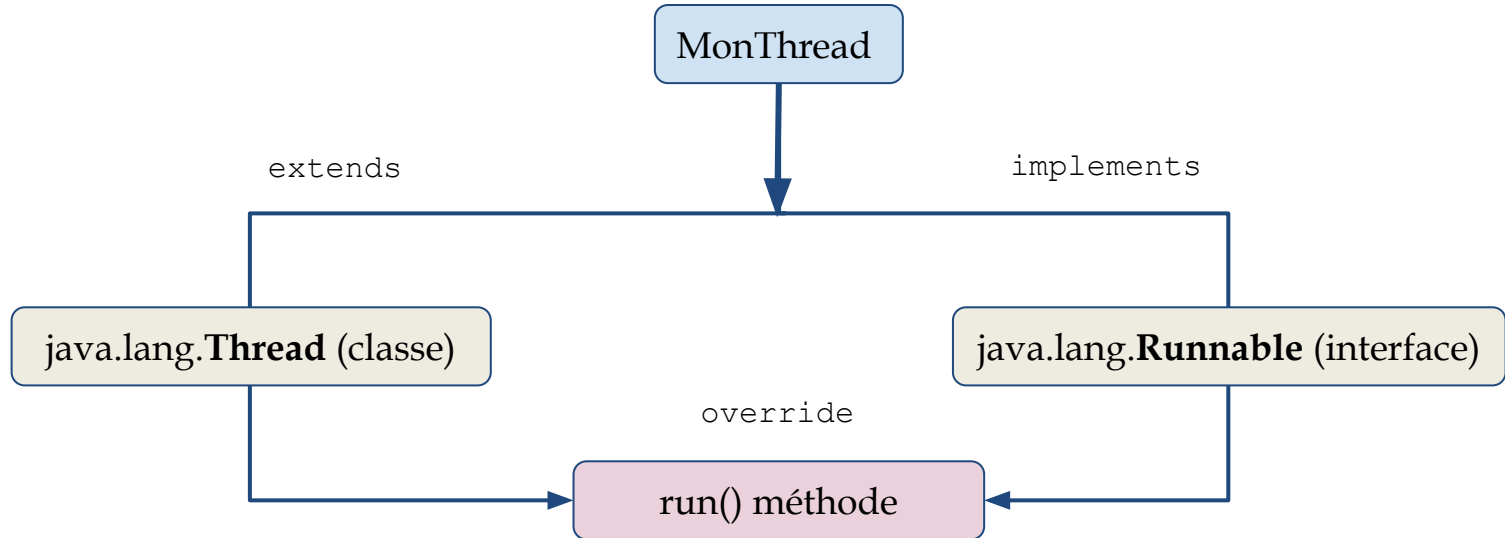
- Méthodes principales de la classe Thread (Consulter [ce lien](#) pour l'ensemble de la documentation) et `wait`, `notify` et `notifyAll` évite le **polling** (interrogation en boucle) :

Méthode	Description
<code>void join()</code>	Attend la fin de l'exécution du thread
<code>void resume()</code>	Reprend l'exécution du thread préalablement suspendue par <code>suspend()</code> .
<code>void run()</code>	Contient les traitements à exécuter
<code>static void sleep(long millis)</code>	Endort le thread pour le délai exprimé en millisecondes
<code>void start()</code>	Associe des ressources systèmes pour l'exécution et invoquer la méthode <code>run()</code>
<code>void suspend()</code>	Suspend le thread jusqu'au moment où il sera relancé par la méthode <code>resume()</code>
<code>void wait()</code>	Amène le thread courant à attendre indéfiniment jusqu'à ce qu'un autre thread appelle <code>notify()</code> ou <code>notifyAll()</code> sur cet objet.
<code>static void yield()</code>	Demande au scheduler de laisser la main aux autres threads
<code>void notify</code>	Réveille un seul thread en attente sur le moniteur de l'objet
<code>void notifyAll</code>	Cette méthode réveille tous les threads qui attendent sur le moniteur de l'objet
<code>void stop()</code>	Arrête le thread

Les Threads - Définition

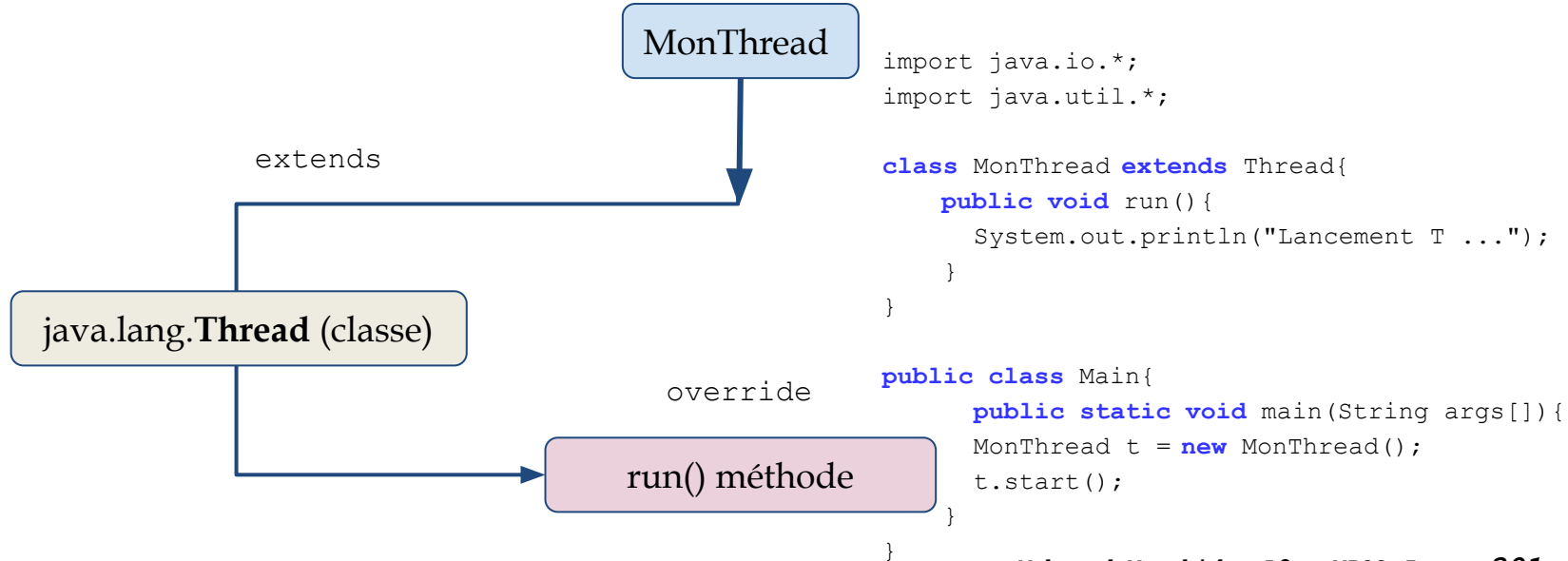
Il existe en Java deux manières de procéder pour mettre en place un nouveau Thread :

- En dérivant le thread depuis la classe `java.lang.Thread`
- En implémentant l'interface `java.lang.Runnable`



Les Threads - Dérivation

- Créer dans un premier temps une classe qui dérive (étend) la classe `java.lang.Thread`.
- Redéfinir ensuite la méthode `run()` (placer le code que le thread doit exécuter).
- Créer enfin une instance de cette nouvelle classe et appeler la méthode `start()` pour lancer le code contenu dans la méthode `run()` dans un nouveau thread.



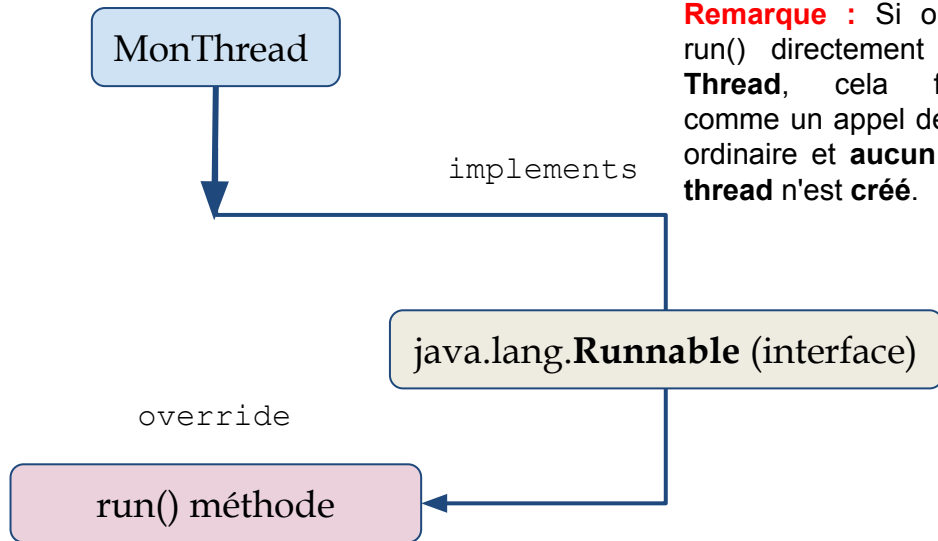
Les Threads - Implémentation

- Créer une classe qui implémente l'interface Runnable.
- Redéfinir la méthode `run()` contenant le code du thread.
- Créer une nouvelle instance de la classe Thread avec un objet Runnable comme paramètre.
- Appeler la méthode `start()` de votre instance de Thread.

```
import java.io.*;
import java.util.*;
```

```
class MonThread implements Runnable
{
    public void run() {
        System.out.println("Lancement R ...");
    }
}
```

```
public class Main{
    public static void main(String args[]){
        MonThread r = new MonThread();
        Thread t = new Thread(r);
        t.start();
    }
}
```



Remarque : Si on appelle `run()` directement **sans** le **Thread**, cela fonctionne comme un appel de méthode ordinaire et **aucun nouveau thread** n'est créé.

Les Threads - Accès ressources

La concurrence en Java permet à **plusieurs threads** de s'exécuter simultanément, améliorant ainsi les performances et l'utilisation des ressources.

Toutefois, une **gestion inadéquate** des données partagées peut entraîner des comportements **imprévisibles** et des **erreurs**.

- Permet l'exécution parallèle de tâches pour de meilleures performances
- Améliore la réactivité et l'efficacité de l'application
- Nécessite une gestion rigoureuse des ressources partagées pour éviter tout problème

Les Threads - Accès ressources

Accès simultané à une même données - Problème

Une concurrence sur des données partagées survient lorsque plusieurs threads accèdent simultanément à des données partagées et que le résultat dépend de l'ordre dans lequel les threads s'exécutent.

```
class Compteur {
    int compt = 0;
    void incremente() { compt++;}
}

public class ConcurrencySynchronized {
    public static void main(String[] args) throws InterruptedException {
        Compteur c = new Compteur();
        Thread t1 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});
        Thread t2 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});
        t1.start(); t2.start();
        t1.join(); t2.join();           // Attends la fin des deux threads ...
        // Le résultat devrait être < 10000
        System.out.println("Compteur final: " + c.count);
    }
}
```

Les Threads - Accès ressources

Accès simultané à une même données - Solution 1

Une concurrence sur des données partagées survient lorsque plusieurs threads accèdent simultanément à des données partagées et que le résultat dépend de l'ordre dans lequel les threads s'exécutent.

```
class Compteur {
    int compt = 0;
    synchronized void incremente() { compt++;}
}

public class ConcurrencySynchronized {
    public static void main(String[] args) throws InterruptedException {
        Compteur c = new Compteur();
        Thread t1 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});
        Thread t2 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});
        t1.start(); t2.start();
        t1.join(); t2.join();           // Attends la fin des deux threads ...
        // Le résultat devrait être < 10000
        System.out.println("Compteur final: " + c.count);
    }
}
```

Un seul thread exécute la méthode à la fois, garantissant ainsi la cohérence des données.

Les Threads - Accès ressources

Accès simultané à une même données - Solution 2

Une concurrence sur des données partagées survient lorsque plusieurs threads accèdent simultanément à des données partagées et que le résultat dépend de l'ordre dans lequel les threads s'exécutent.

```
class Compteur {  
    int compt = 0;  
    void incremente() { synchronized(this) {compt++;}}  
}
```

Seul le bloc mettant à jour le compteur est synchronisé.

```
public class ConcurrencySynchronized {  
    public static void main(String[] args) throws InterruptedException {  
        Compteur c = new Compteur();  
        Thread t1 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});  
        Thread t2 = new Thread(() ->{ for (int i = 0; i < 5000; i++) c.incremente();});  
        t1.start(); t2.start();  
        t1.join(); t2.join(); // Attends la fin des deux threads ...  
        // Le résultat devrait être < 10000  
        System.out.println("Compteur final: " + c.count);  
    }  
}
```

Les Threads - Accès ressources

Accès simultané à une même données - Remarques

- Un thread entrant dans une méthode ou un bloc `synchronized` acquiert un verrou et le libère à la sortie de la méthode.
- Méthodes/blocs d'instance : acquisition d'un verrou au niveau de l'objet.
- Méthodes statiques : acquisition d'un verrou au niveau de la classe.
- La synchronisation sur des objets `null` provoque une `NullPointerException`.
- `wait()`, `notify()` et `notifyAll()` sont des méthodes essentielles à la synchronisation.
- Évitez de synchroniser des champs non `final` pour empêcher les threads de verrouiller des objets différents.
- Le mot-clé `synchronized` ne peut pas être appliqué aux variables.

Les Threads - Accès ressources

Accès simultané à une même données - **Avantages et limitations**

Avantages

- Garantit l'exclusion mutuelle pour les ressources partagées dans les programmes multithread.
- Les méthodes d'instance et les méthodes statiques synchronisées peuvent s'exécuter simultanément, car elles verrouillent des objets différents.

Limitations

- Empêche les lectures simultanées, ce qui limite la concurrence.
- Les méthodes synchronisées peuvent réduire les performances. Privilégiez la synchronisation par bloc pour les sections critiques uniquement.

Les Threads - Accès ressources

Visibilité des variables - Problème

Cela se produit lorsqu'un thread modifie une variable, mais que les autres threads ne voient pas la valeur mise à jour en raison de la mise en cache ou d'un manque de synchronisation => **Cela peut empêcher la boucle de se terminer.**

```
class TestVisibilite {
    private boolean running = true;
    void start() {
        new Thread(() -> {
            while (running) {}
            System.out.println("Stopped!");
        }).start();
    }
    void stop() { running = false;} //Modification potentiellement invisible pour les autres threads ...
    public static void main(String[] args) throws InterruptedException {
        TestVisibilite t = new TestVisibilite();
        t.start();
        Thread.sleep(100); // Courte pause avant l'arrêt ...
        t.stop();          // Le processus ne le verra pas ...
    }
}
```

Les Threads - Accès ressources

Visibilité des variables - Solution

Cela se produit lorsqu'un thread modifie une variable, mais que les autres threads ne voient pas la valeur mise à jour en raison de la mise en cache ou d'un manque de synchronisation => Cela peut empêcher la boucle de se terminer.

```
class TestVisibilite {
    private volatile boolean running = true;
    void start() {
        new Thread(() -> {
            while (running) {}
            System.out.println("Stopped!");
        }).start();
    }
    void stop() { running = false;} //Modification potentiellement invisible pour les autres threads ...
    public static void main(String[] args) throws InterruptedException {
        TestVisibilite t = new TestVisibilite();
        t.start();
        Thread.sleep(100); // Courte pause avant l'arrêt ...
        t.stop();          // Le processus ne le verra pas ...
    }
}
```


Les Threads - Accès ressources

Visibilité des variables - Quand utiliser volatile ?

- Pour les variables de type booléen (par ex. signal d'arrêt de thread)
- Pour les variables d'état
- Lorsqu'un seul thread écrit et que les autres lisent
- En l'absence d'opérations composées (comme l'incrémentatation)

Les Threads - Accès ressources

Différences entre `synchronized` et `volatile`

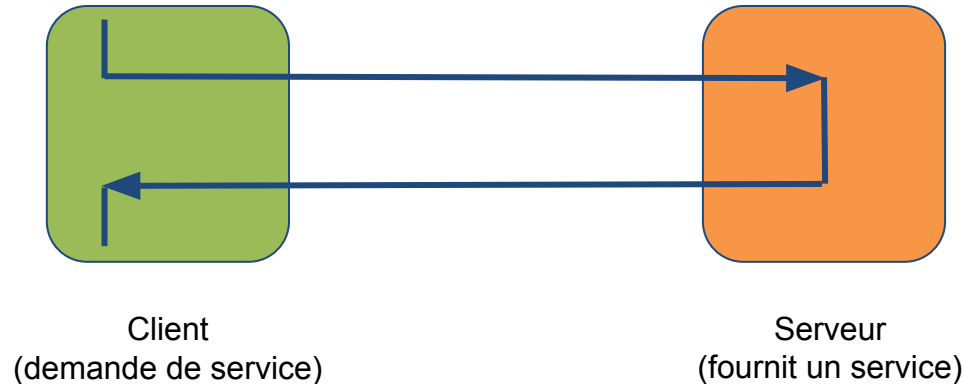
Caractéristique	<code>volatile</code>	<code>synchronized</code>
Objectif	Assure la visibilité des variables	Assure la visibilité et l'atomicité *
Sécurité des threads	Partielle (visibilité uniquement)	Totale
Atomicité	Non garantie	Garantie
Verrouillage	Aucun verrouillage	Utilise un verrou intrinsèque (moniteur)
Performance	Plus rapide (léger)	Plus lente (surcoût lié au verrouillage)
Cas d'utilisation	Drapeaux, variables d'état	Sections critiques, ressources partagées

* une opération atomique est composée éventuellement de plusieurs actions, qui se déroule sans pouvoir être interrompue, en particulier par un autre *thread*.

Communication TCP : Socket

Communication TCP

Dans le schéma **client-serveur**, pour le client, un service est souvent désigné par un nom symbolique converti en une adresse interprétable à la charge du service DNS.



Communication TCP

Pour programmer une application client-serveur, il est commode d'utiliser les *sockets* fournissant une interface qui permet d'utiliser facilement les protocoles de transport (TCP à besoin d'une liaison établie au préalable et UDP).

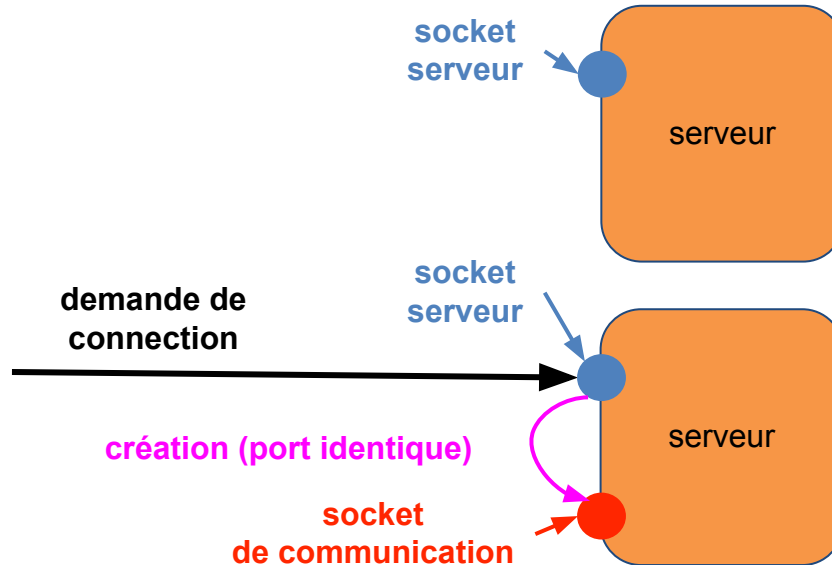
Une *socket* est un moyen de désigner l'extrémité d'un **canal de communication bidirectionnel** associé à un port.

Une fois le canal de communication établi entre processus client et serveur, ceux-ci peuvent communiquer..

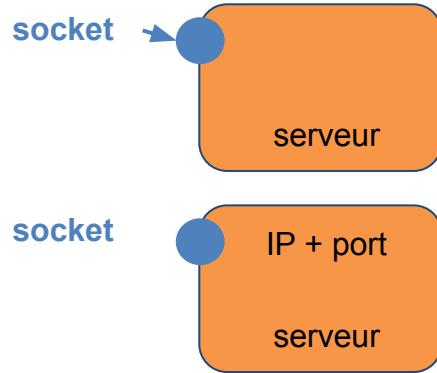


Communication TCP - Serveur

Un **serveur** en mode connecté doit **attendre** une **nouvelle demande** de connexion de la part d'un **client**, puis traiter la (ou les requêtes) envoyée(s) sur cette connexion par le client.
Les fonctions **d'attente** et de traitement sont **séparées**, pour permettre au serveur d'attendre de nouvelles demandes de connexion pendant qu'il traite des requêtes en cours.



Communication TCP - Serveur



Étape 1 : créer une socket serveur TCP indépendante du système :

```
ServerSocket servSock = new ServerSocket();
```

Étape 2 : associer la socket à une adresse IP + port TCP

```
int myPort = ...
```

```
int tailleFileAttente = ...
```

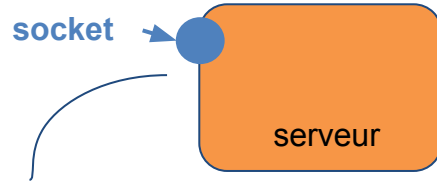
```
InetSocketAddress ipAddrAndPort;
```

```
ipAddrAndPort = new InetSocketAddress(myPort);
```

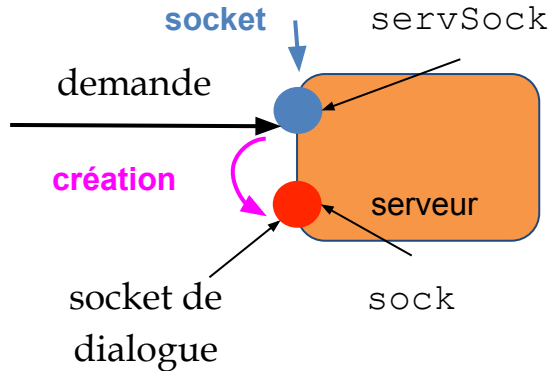
```
servSock.bind(ipAddrAndPort, tailleFileAttente);
```

Cette version du constructeur indique implicitement qu'il faut considérer l'ensemble des adresses IP associées à la machine locale ("*wildcard*"), plutôt qu'une adresse particulière

Communication TCP - Serveur



prête à accepter les
demandes de connexion



Étape 3a : permettre à l'application (côté serveur) de prendre connaissance d'une nouvelle connexion (la primitive `accept` est bloquante si la file est vide)

```
Socket sock;
```

```
...
```

```
sock = servSock.accept();
```

Étape 3b : obtention d'un canal de communication/dialogue par l'application (côté serveur)

```
sock = servSock.accept();
```

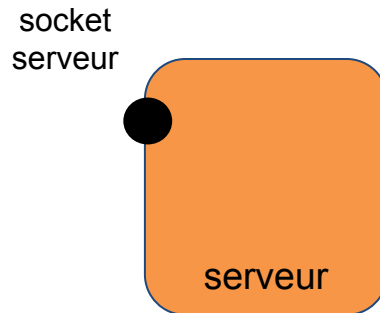
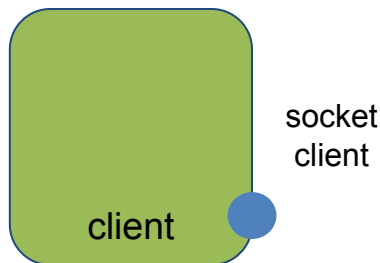
Au retour de l'appel à `accept`, cette variable contient une référence vers un objet `Socket` permettant de dialoguer avec le client

Communication TCP - Client

On doit disposer de l'adresse d'un serveur et le numéro de port (TCP) d'une socket serveur sur celui-ci (sachant qu'un processus serveur est en attente sur ce port)

Étape 1 : créer une *socket* :

```
Socket sock = new Socket();
```



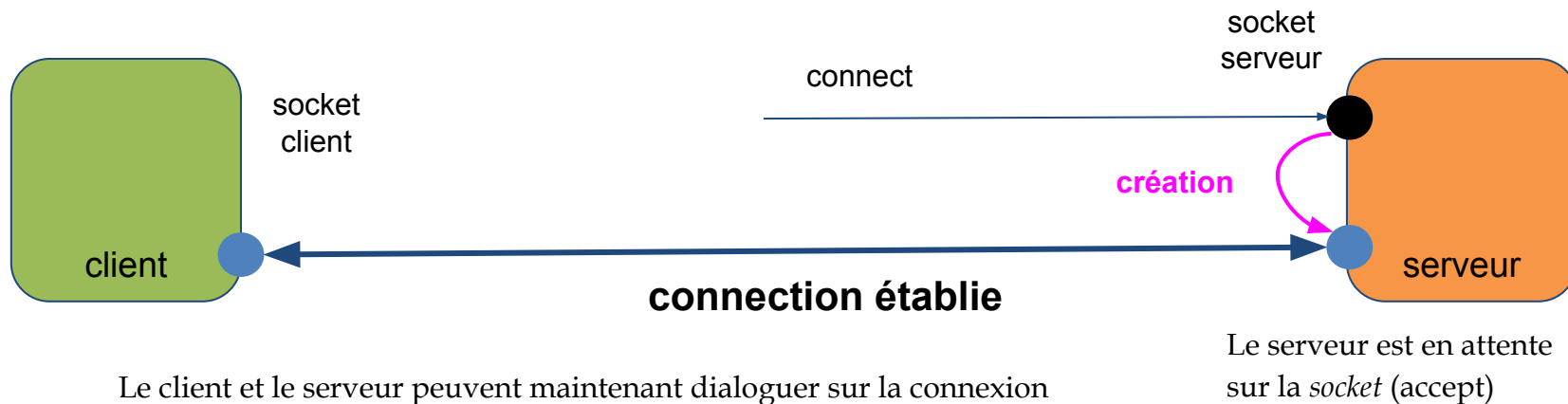
Le serveur est en attente sur la *socket* (accept)

Communication TCP - Client

`connect` envoie une demande de connexion vers la *socket* serveur (une exception `IOException` est levée en cas d'échec de la connexion)

Étape 2 : établir une connexion entre la socket client et le serveur

```
InetSocketAddress serverIpAddrAndPort = new InetSocketAddress (...);  
sock.connect(serverIpAddrAndPort);
```



Communication TCP - Communication

Une fois la connexion établie, le client et le serveur disposent chacun d'un descripteur (pseudo-fichier) vers l'extrémité correspondante de la connexion.

À partir de chaque extrémité, on peut créer 2 flux pour interagir avec l'interlocuteur distant :

- Un flux d'entrée (en lecture) : réception/consommation des données émises par l'interlocuteur
- Un flux de sortie (en écriture) : émission/production de données vers l'interlocuteur

Java fournit différents types de flux d'entrée/sorties à partir d'un objet Socket, comme les deux flux d'octets bruts (binaires)

- **InputStream**(flux d'entrée) et **OutputStream**(flux de sortie)

Communication TCP - Exemple de serveur

```
servSock = new ServerSocket();  
servSock.bind(ipAddrAndPort, backlogSize); // variables à définir  
  
while (true) {  
    sock = servSock.accept();  
  
    // le code « métier » du serveur proprement dit,  
    // qui implémente le dialogue avec un client  
    this.handleDialogWithClient(sock);  
    // lorsque ce dialogue se termine, on ferme la connexion  
    // (si pas déjà fait dans la méthode précédente)  
  
    sock.close();  
    // maintenant on peut accepter la prochaine connexion  
}
```

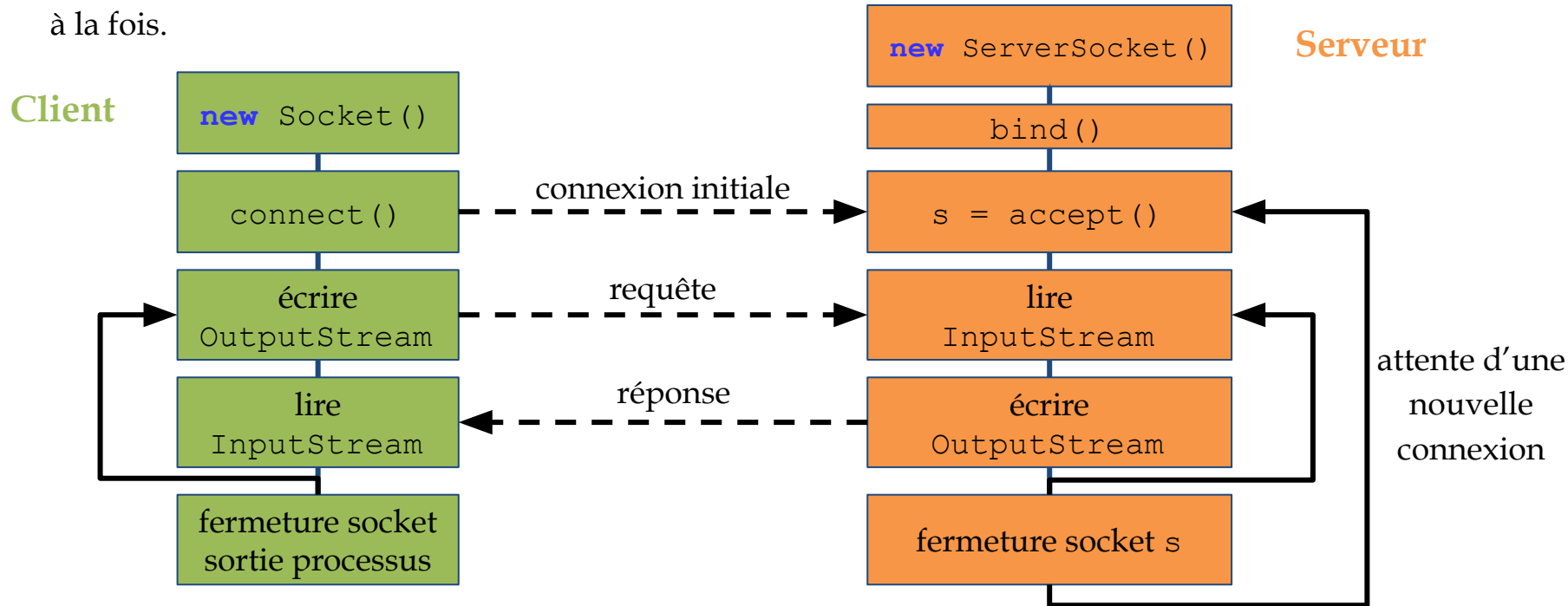
Communication TCP - Exemple de client

```
clientSock= new Socket();  
clientSock.connect(serverIpAddrAndPort); // à définir  
  
// le code « métier » du client proprement dit,  
// qui implémente le dialogue avec le serveur  
this.handleDialogWithServer(clientSock);  
  
// lorsque ce dialogue se termine, on ferme la connexion  
// (si pas déjà fait dans la méthode précédente)  
clientSock.close();
```

Remarque : On n'a pas prévu d'arrêter le serveur (il faut tuer le processus qui l'exécute). Dans une application réelle, il faut prévoir un mécanisme pour arrêter proprement le serveur puis les différents clients.

Communication TCP - Mode itératif

Les programmes précédents réalisent un serveur en **mode itératif** : un seul client est servi à la fois.



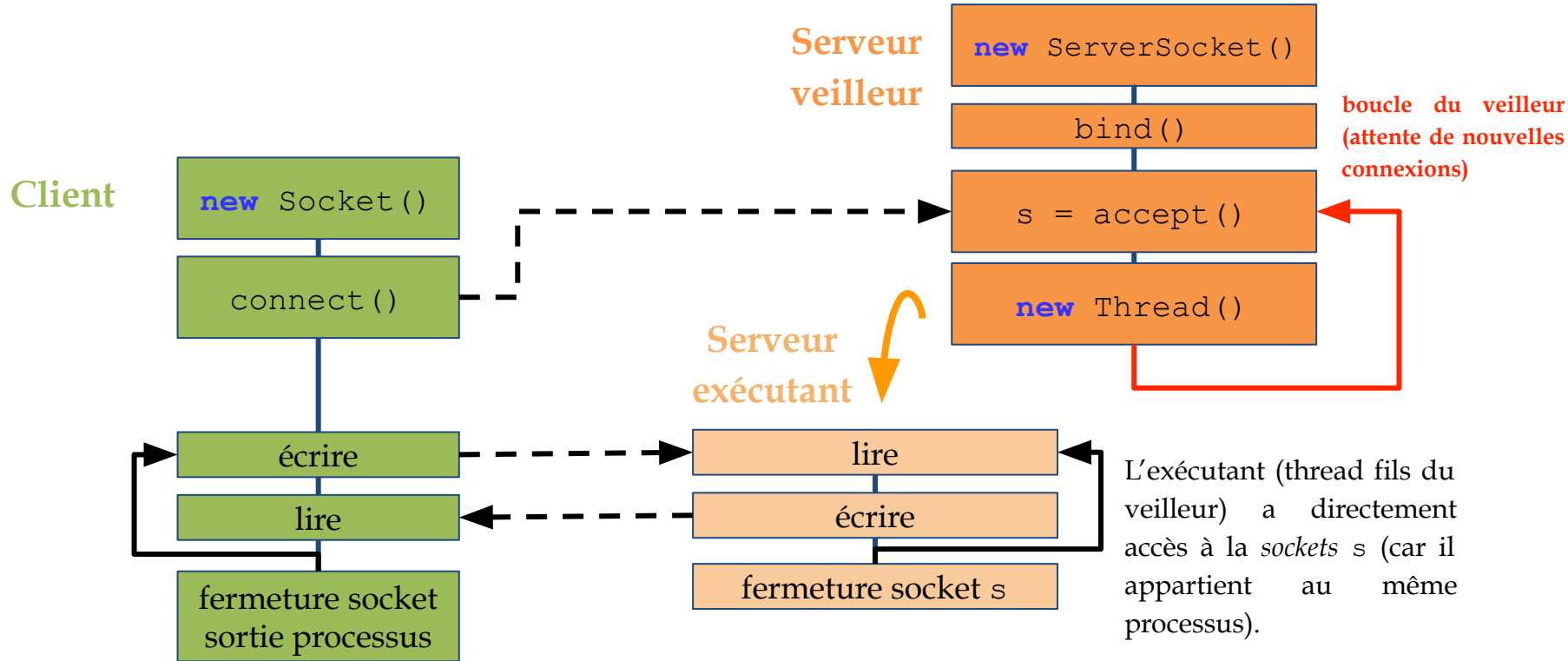
Communication TCP - Mode concurrent

Le **mode concurrent** consiste à **créer un nouveau flot d'exécution** pour chaque nouvelle connexion en utilisant **plusieurs *threads***.

Architecture du serveur concurrent :

- Il y a un flot d'exécution principal (appelé *veilleur*) qui attend sur `accept ()` .
- Lorsqu'il reçoit une demande de connexion, le veilleur crée un nouveau flot (appelé *exécutant*) qui va interagir avec le nouveau client.
- Après la création de l'exécutant, le veilleur revient se mettre en attente sur `accept ()` .
- Plusieurs exécutants peuvent coexister simultanément.

Communication TCP - Mode concurrent



Communication TCP - Suivie

Commande **netstat** : `netstat -t` (équivalent à : `netstat -A inet --tcp`)

Options utiles :

- `-aou --all` : permet d'afficher toutes les sockets existantes sur la machine (par défaut, seules les sockets connectées sont listées)
- `-lou --listen` : affiche uniquement les sockets serveurs
- `-p` : affiche le pid du processus propriétaire d'une socket
- `-e` : permet de connaître l'utilisateur associé au processus propriétaire d'une socket
- `--numeric-hosts` : désactiver la résolution des noms de machines (affichage des adresses IP)
- `--numeric-ports` : désactiver la résolution des numéros de ports (par défaut, les numéros de ports utilisés par les services usuels sont remplacés par le nom du service correspondant, à partir des informations disponibles dans le fichier `/etc/services`)

Communication TCP - Suivie

Commande : `ss -t`

Options utiles :

- `-aou --all` : permet d'afficher toutes les sockets existantes sur la machine (par défaut, seules les sockets connectées sont listées)
- `-lou --listening` : affiche uniquement les sockets serveurs
- `-p`: affiche le pid du processus propriétaire d'une socket
- `-e`: permet de connaître l'utilisateur associé au processus propriétaire d'une socket
- `-nou --numeric u-rou --resolve` : désactiver la résolution des noms de services : activer la résolution des noms de machines et de services

Prochaine séance : Evaluation



Séance	Chapitre	début	fin
1	Principes de base de l'OO et introduction à Java	3	86
1	Classes, objets et classes utilitaires	87	129
2	Héritage, généricité et polymorphisme	130	207
2	Interfaces et lambda	208	270
3	Packages et exceptions	276	341
3	Collections, entrées-sorties	342	376
3	Sérialisation. Base de données. Thread et Socket	377	419
4	Evaluation		